

SKALA

AI Campus 생성형AI

# 7. AI Agent 설계 및 구축

2026.10

배기주 / SK AX

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX




1. AI Agent 이해
2. AI Agent 핵심 구성
3. AI Agentic Workflow 설계
4. AI Agent 구현

[Appx. A] AI Agent 평가

[Appx. B] Harness Engineering

AI Campus 생성형AI

# 1. AI Agent 이해

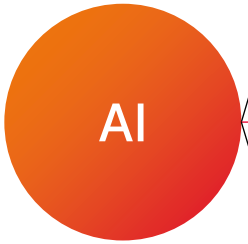
- AI Evolution
- AI Agent = LLM + TOOLS
- AI Service Spectrum
- Prompt to Harness Engineering

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

# 1. AI Evolution

Model to Intelligence




## ANI

Artificial Narrow Intelligence

**특정 업무(분야)에 특화**된 알고리즘 기반의 학습된 모델

- 규칙 기반 Expert System
- Machine Learning (SVM, XGBoost, Ensemble 등)
- 딥러닝 기반 서비스 (얼굴인식, 바둑AI 등)

## G.AI

Generative AI

### Single Modal

- (텍스트) Large Language Models
- (이미지) DALL-E, Stable Diffusion, Midjourney 등
- (음성) Text-to-Speech, Voice Conversion, Compositiion 등

### Multi Modal

- GPT-4V(ision), Flamingo, Kosmos-2 등
- 텍스트+이미지+오디오+비디오 종합 이해 및 생성

### Actuation with Perception

Perception-Action Loop, Embodied AI, Physical AI, Cognitive Robotics

- 로봇 제어 AI (휴머노이드, 산업용 로봇 등)
- 자율주행, 드론 : 센터 데이터 기반 판단 및 제어

## AGI

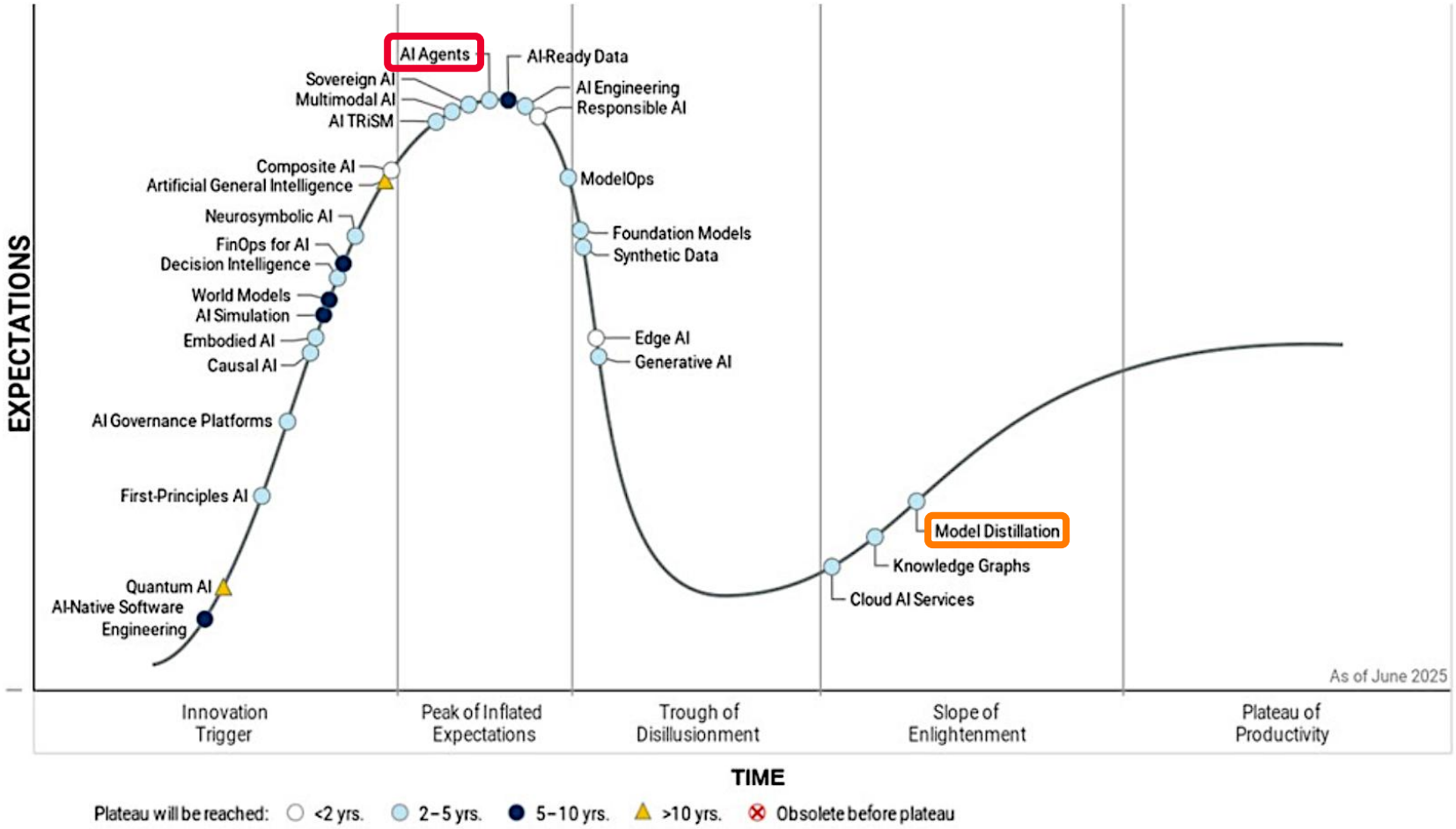
Artificial General Intelligence

(인간 수준의) **광범위하고 유연한 학습 및 추론** 능력 확보

- AI의 미래적 목표로 아직 구체적으로 실현된 예는 없음
- 더 나아가면 ASI (Artificial Super Intelligence) 가능성도 거론

# 2. AI, 어디까지 왔나?

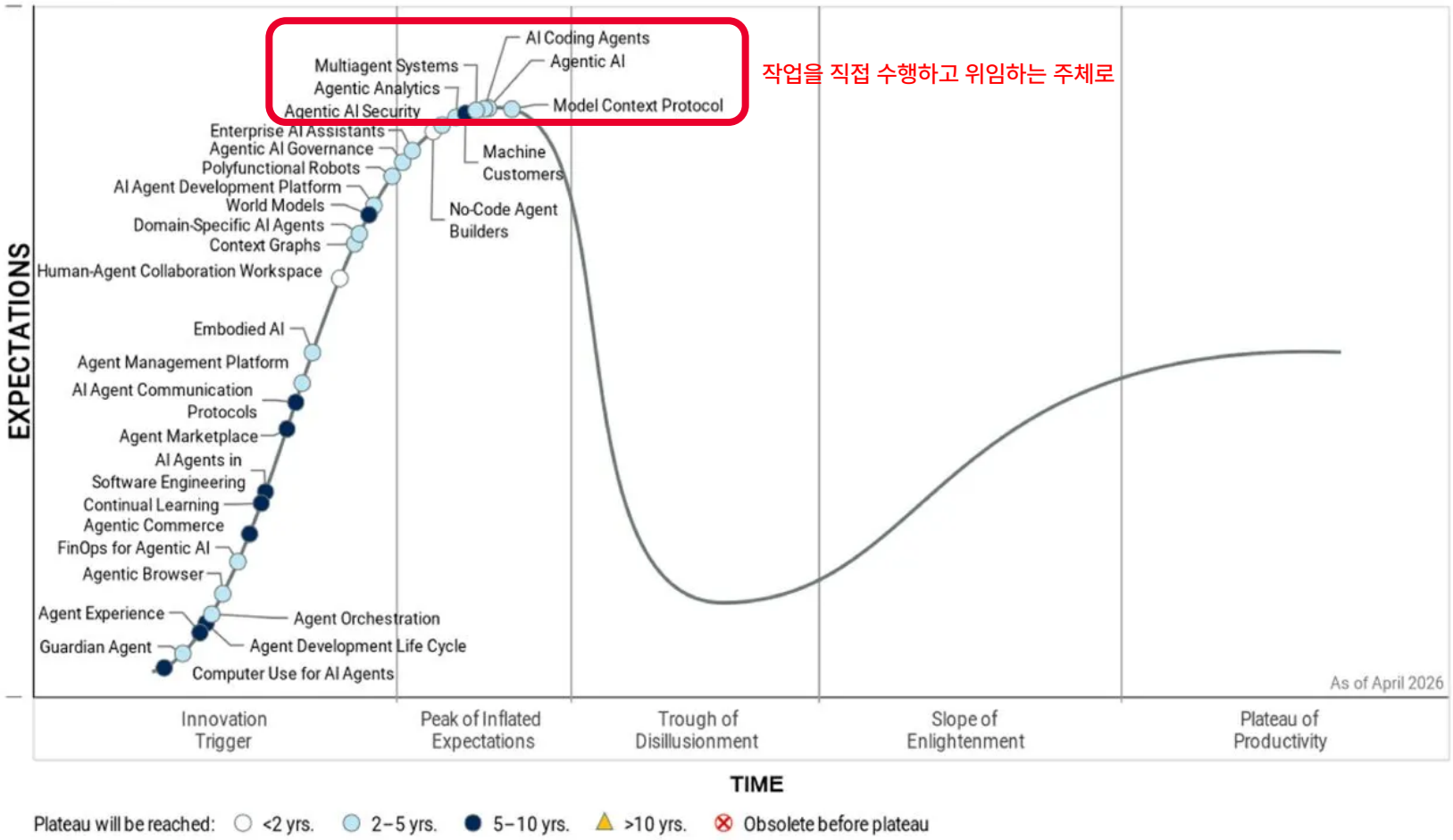
2025, Expectation to Enlightenment in Generative AI




Gartner

# 3. Agent, 어디를 향하고 있나?

2026, Peak of Inflated Expectations in Agentic AI




*Source : https://onereach.ai/resources/gartner-2026-hype-cycle-for-agentic-ai/*

# 2. AI Agent

UPDATE

## Agent

- 계약이나 협상 등에 관한 업무를 대신하여 처리하고 그에 대한 수수료를 받는 법정 대리인
- 특정 분야에서 전문적인 지식이나 기술을 갖추고 다른 사람이나 기업을 대신해 일을 처리하는 역할

## AI Agent – System

- LLM 기반 추론으로 스스로 도구를 선택, 실행 하여 목표를 완수하는 소프트웨어 시스템
- Single : MCP/API 기반 도구 연동 기반 – 코드 실행, 웹브라우징, 파일시스템 등
- Multi : 다단계 Task를 수행하기 위해 역할별 에이전트가 협업/조율 하는 구조

## Agentic AI – Paradigm

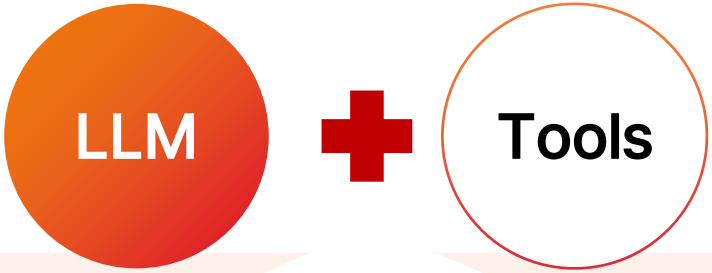
- Agentic Behavior : 외부 자극에 반응하는 것이 아니라, 스스로 목표를 설정하고 행동하는 자기 주도적 행동 (심리학)
- 최소한의 인간 개입으로 스스로 계획/판단/실행하는 자율성의 정도 – 얼마나 Agentic한가?
- Agentic 구성 능력 : Reasoning, Memory/State, Tool Use, Self-Reflection, etc.

# 2. AI Agent

Key Point

두뇌와 행동을 하나로!!

*AI Agent의 지능과 판단 담당 (두뇌)*




*AI Agent가 실제로 작업을 수행하거나 환경과 상호작용 하는 역할 담당 (행동)*

**LLM은 맥락을 기반으로 계획을 수립하고, Agent는 도구를 활용하여 자율적으로 수행**

### ❑ 자연어 처리 및 기억, Memory

- 사용자의 질문과 의도를 정확히 이해하고 적합한 응답 생성
- 이전 대화 맥락을 유지(단기 기억, State)하고, 사용자 선호도나 과거 데이터를 저장/조회(장기 기억)하여 맞춤형 응답 제공

### ❑ 작업 계획 및 추론, Planning

- 사전에 학습된 데이터로부터 지식을 생성하고 판단 수행
- 복잡한 요청을 달성 가능한 여러 하위 단계로 쪼개고 (Task Decomposition), 스스로의 결과를 평가 (Self-Reflection)하여 다음 행동을 결정

### ❑ 필요 도구 연결, Tool Use

- 외부 시스템(DB, Cloud, 인터넷 등)과 통합되어 실질적인 행동 실행
- 예: 스케쥴 시스템에 접근해 회의 일정 저장, RAG 검색 등

### ❑ 자율적 작업 자동화, Autonomous Action

- 정의된 프로세스 따라 반복 업무를 자동화하여 편의성 증대
- LLM이 수립한 계획(Planning) 따라 적절한 도구를 순차적으로 호출하여 작업을 완수하고, 필요 시 사람의 승인 거쳐 안전하게 완수

# [참고] AI Agent for Travel

2024-06

## My First Trip to Norway, With A.I. as a Guide

Can artificial intelligence devise a bucket-list vacation that checks all the boxes: culture, nature, hotels and transportation? Our reporter put three virtual assistants to the test.

"4박 5일로 노르웨이 여행을 가고 싶어요. 제 여행 예산에 맞춰 호텔 예약이 되면 좋겠어요."

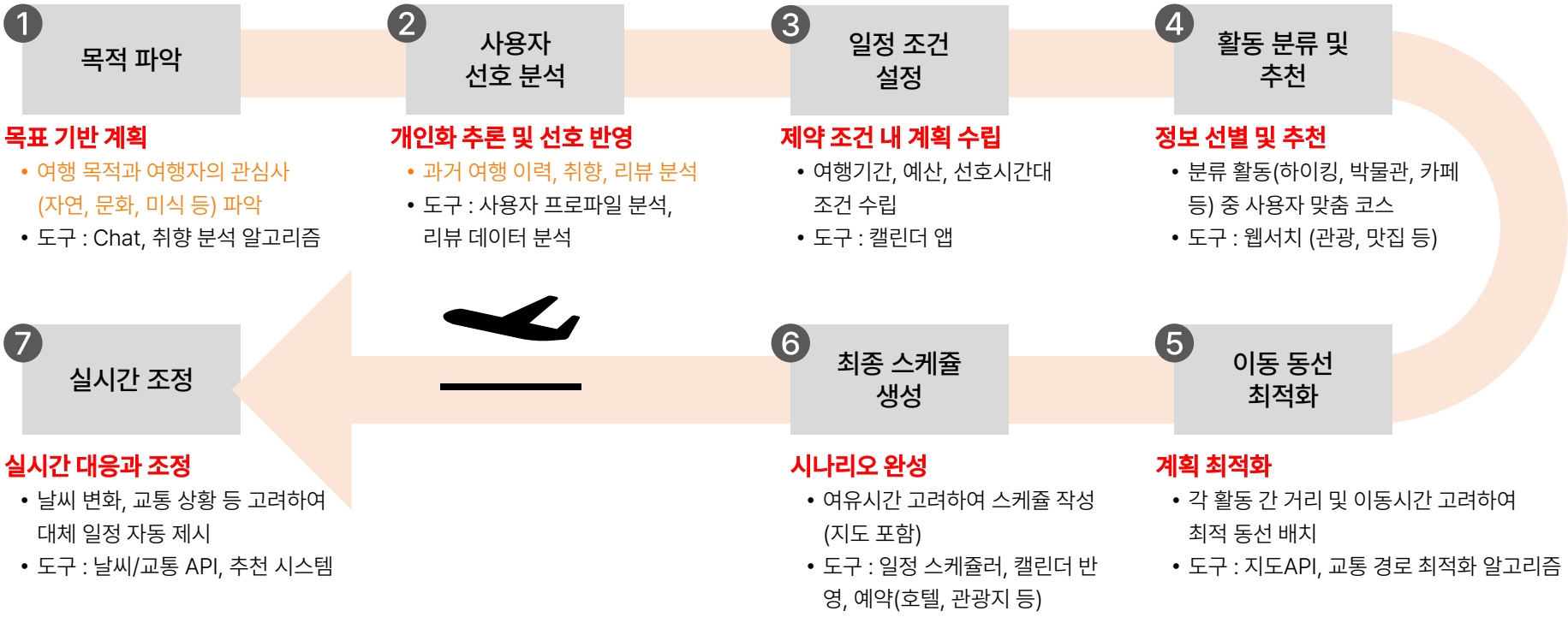


The train between Flam and Myrdal, Norway, goes through some of the most scenic spots in the country. The ride was recommended by the author's virtual assistants. norwaysbest.com/Sverre Hjørnevik

*Source : https://www.nytimes.com/2024/06/26/travel/norway-artficial-intelligence-planners.html*

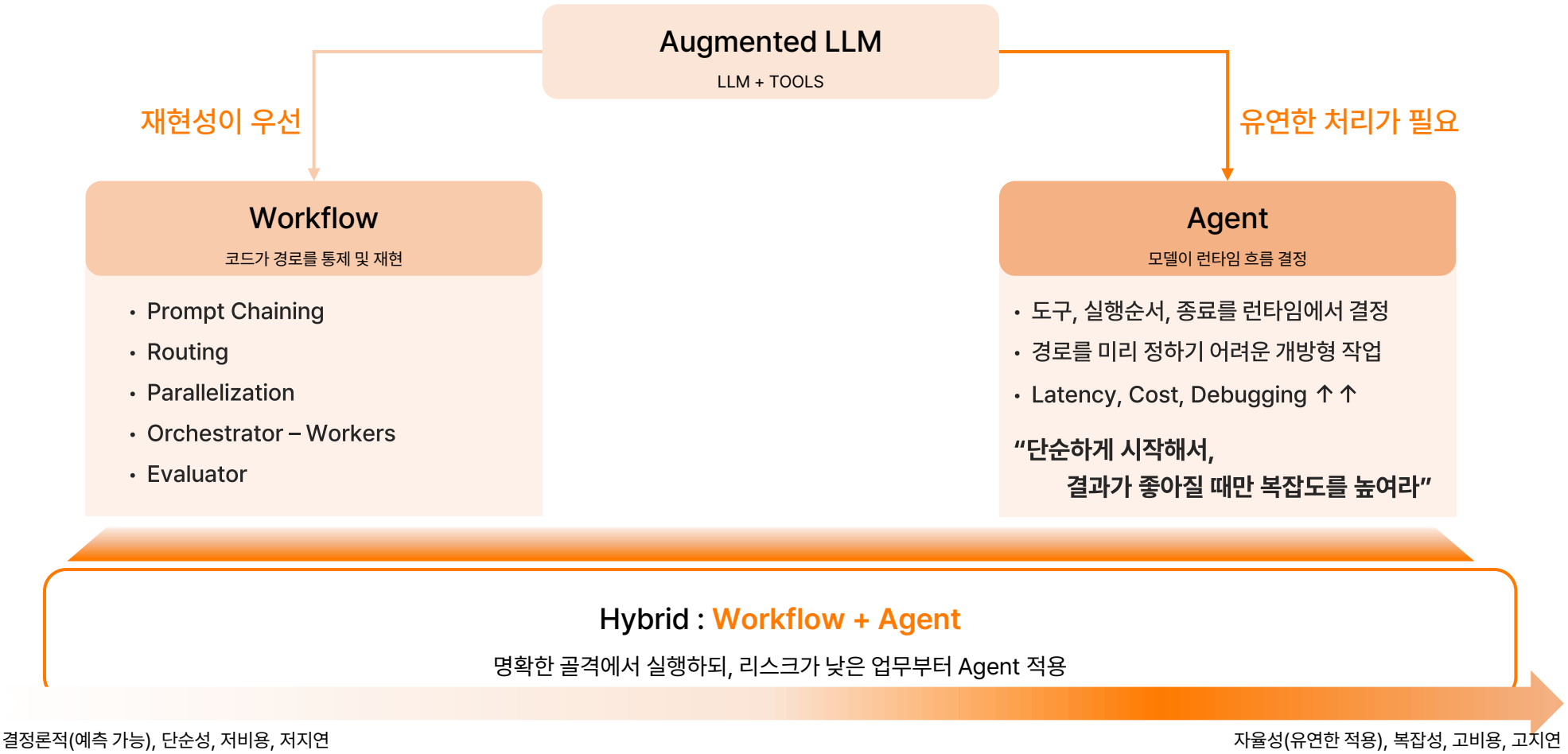
# [참고] AI Agent for Travel

"4박 5일로 노르웨이 여행을 가고 싶어요. 제 여행 예산에 맞춰 호텔 예약이 되면 좋겠어요."



# 3. AI Service, Spectrum

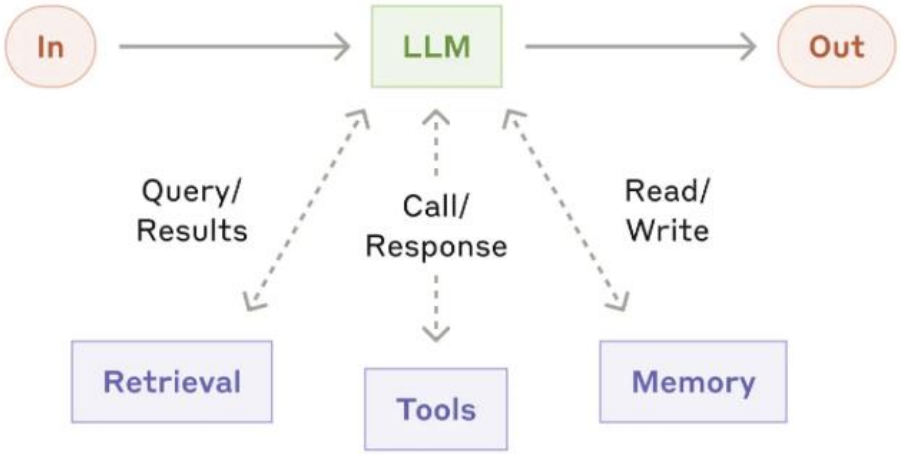
2026.08




*Source : https://www.anthropic.com/engineering/building-effective-agents, https://shellypalmer.com/2026/04/how-anthropic-thinks-about-agents-workflows-and-tasks/*

# [참고] Augmented LLM

The basic building block of agentic systems




- 단순히 LLM에 RAG, 도구, 메모리를 붙여 놓은 상태는 기능을 확장한 기본 컴포넌트에 불과 → Augmented LLM

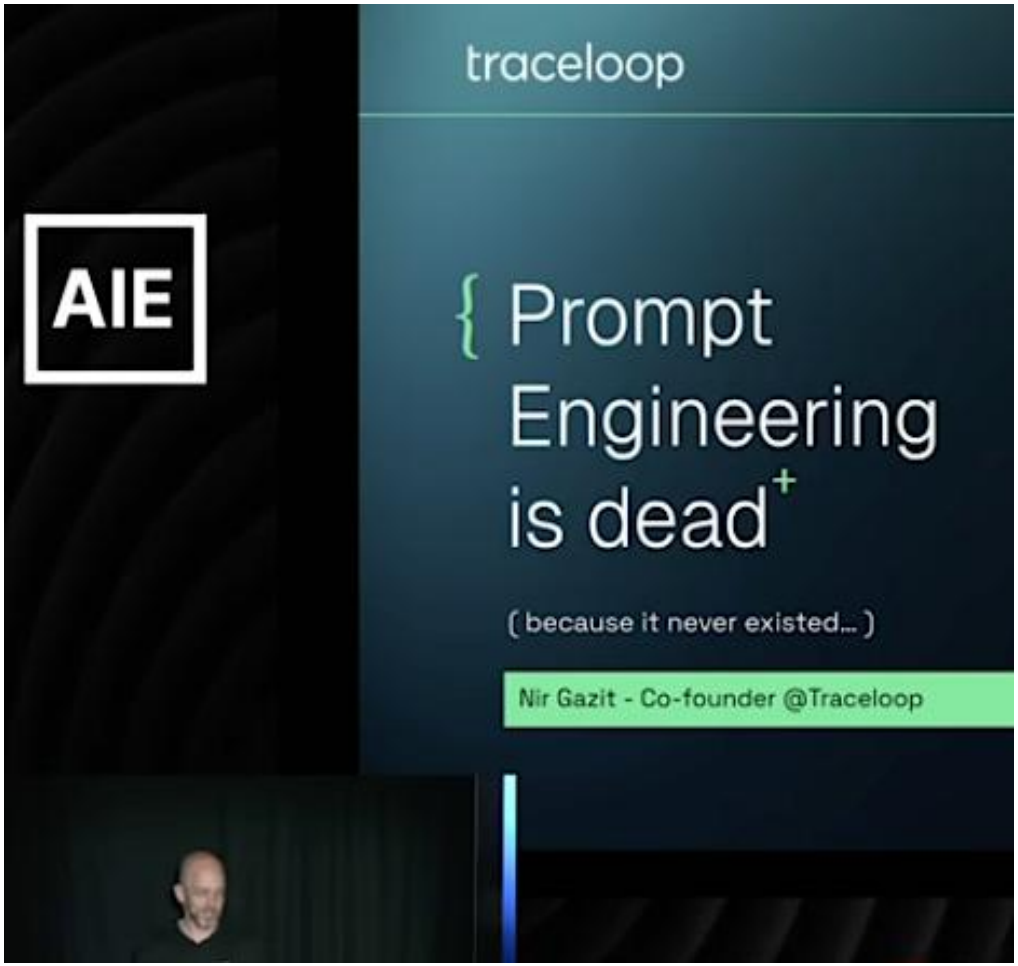
- 이 증강된 LLM이 "어떤 제어 구조로 실행되느냐"에 따라 시스템 성격이 달라짐
  - Workflow : 개발자가 코드로 정해둔 고정된 순서에 따라 Augmented LLM이 호출되는 방식
  - Agent : LLM 스스로 다음에 어떤 도구를 쓸지, 몇 번 반복할지, 언제 작업을 끝낼지를 런타임에 동적으로 결정하는 루프 방

※ Enterprise에서는 Hybrid, Guardrailed Agent 패턴으로 구성
– 안정성, 보안, 비용 등의 문제 고려

*Source : https://www.anthropic.com/engineering/building-effective-agents*

# 1. Prompt Engineering is Dead

2025-06



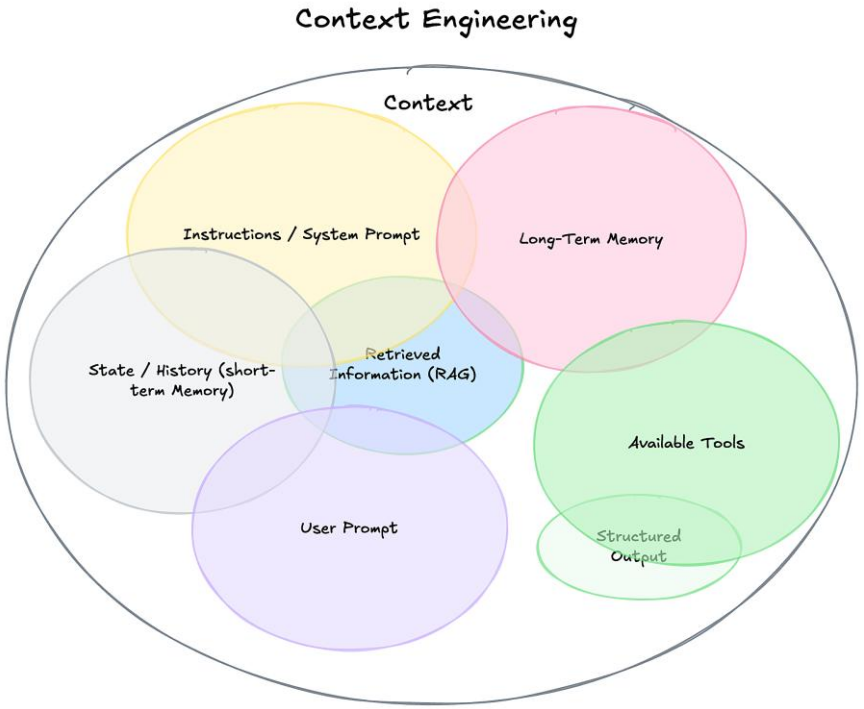

- Manual prompt crafting doesn't scale.
- If you're still hand-tuning prompts, you're doing it wrong.

더 이상 잘 쓰는 프롬프트가 경쟁력이 아니다.
잘 작동하는 **AI 에이전트/시스템**을 설계해야 한다.

*Source : https://www.youtube.com/watch?v=jvKf6zXrNO4*

# 2. Context Engineering

(2025) 에이전트가 어떤 작업을 합리적으로 수행하기 위해서는 그 작업에 적합한 컨텍스트가 필요




Prompt :

- **Instruction/System Prompt** : 대화 중 모델의 동작 방식을 정의하는 지침
- **User Prompt** : 사용자로부터 받은 작업 요청

Context :

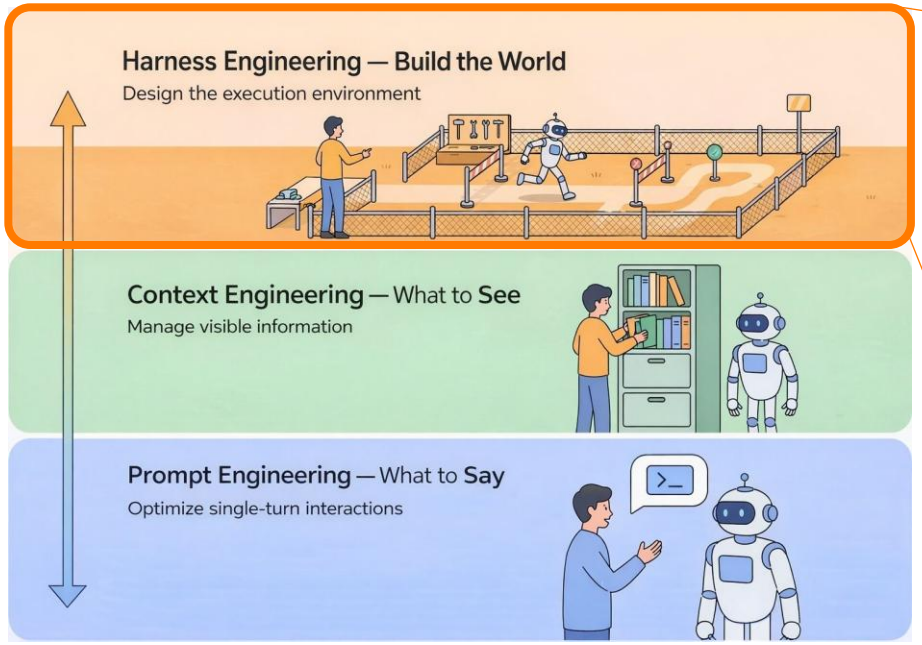
- **State / Short-term Memory** : 지금까지의 사용자와 모델 간의 대화를 포함하여, 현재 시점까지 이어진 맥락
- **Long-term Memory** : 이전 여러 대화를 통해 축적된 지속적인 지식
- **RAG** : 외부 최신 정보. 특정 질문에 대한 정확한 답변을 제공하기 위해 사용

Agent :

- **Available Tools** : 모델이 호출 할 수 있는 모든 함수나 도구 정의
- **Structured output** : 모델의 응답 형식에 대한 정의

# 3. Harness Engineering

(2026. H1) Agent = Model + Harness



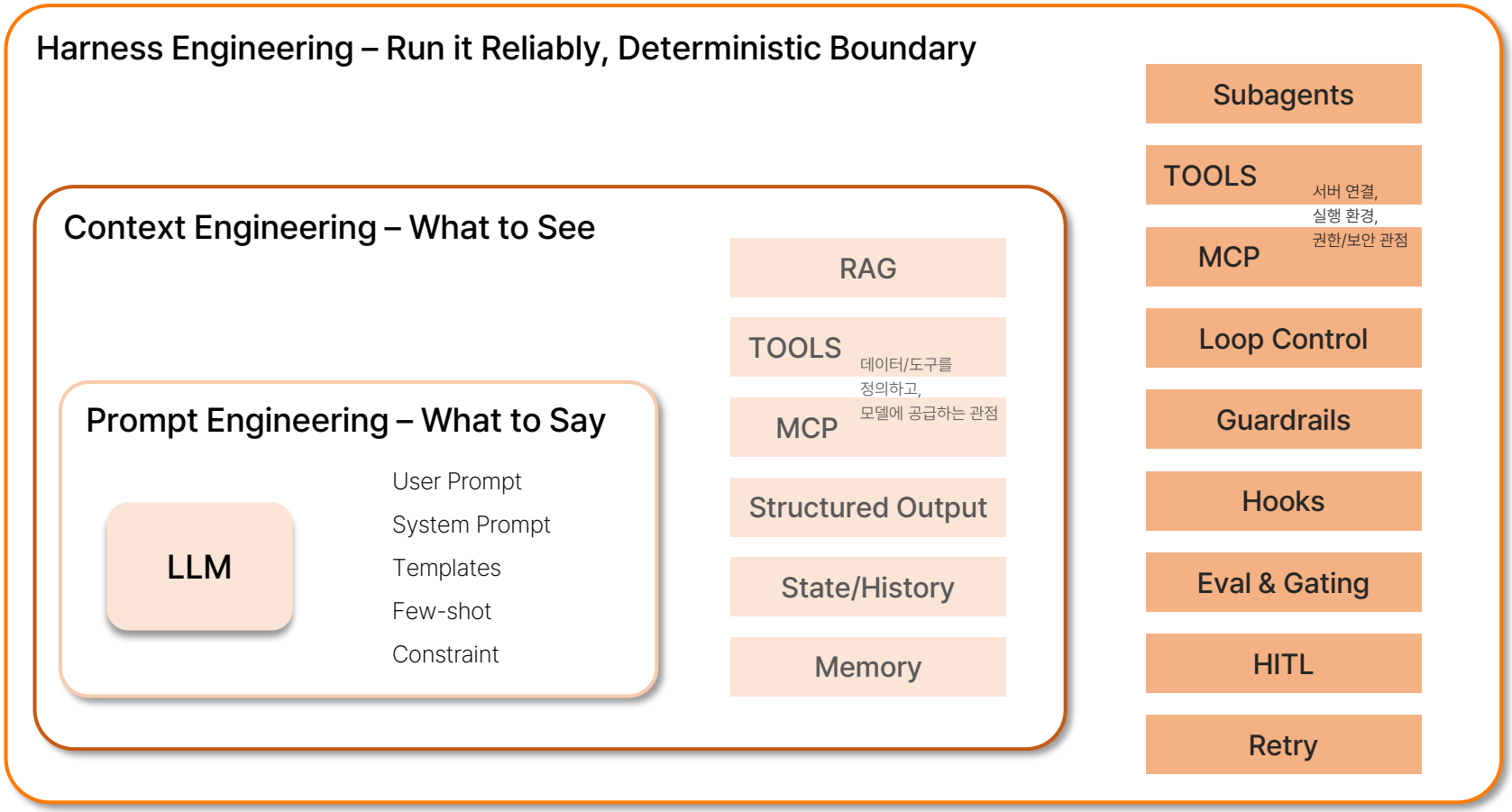

- Goal :
  확률적이고 불안정한 LLM을
  Production에서 신뢰할 수 있도록 **Deterministic Environment** 구축
  → 이를 위한 제어 시스템 ➔ **HARNESS**

- 핵심 역할 :
  - 상태 및 흐름 제어 : State 관리, Tool 실행 루프, HITL
  - Guardrail, 보안 : 권한 격리, 비정상 루프 방지, 안전 장치
  - 결정론적 강제, Deterministic constraint
    : 프롬프트의 확률적 준수를 넘어,
    코드로 규칙을 강제하여 재현성 확보

*Source : https://milvus.io/blog/harness-engineering-ai-agents.md*

# 4. Prompt to Harness Engineering

2026.H1



AI Campus 생성형AI

# 2. AI Agent 핵심 요소

- LLM - Reasoning
- TOOLS - ReAct Framework
- AGENT - Agent Protocol

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

# [참고] Inference vs Reasoning

**Inference :**

- A conclusion reached on the basis of evidence and reasoning
- 증거나 추론을 바탕으로 도달한 결론
- 결론 또는 추론된 결과, 그 자체를 가리키는 경우가 많음

**Reasoning :**

- The action of thinking about something in a logical way in order to form a conclusion or judgment
- 어떤 결론이나 판단에 도달하기 위해 논리적으로 사고하는 행위
- 전제 → 사고 과정 → 결론 도출로 구분되는 생각의 전개 과정에 초점

# [참고] Inference

모델 실행 단계로 예측 수행 → "논리적 추론, Reasoning" 으로 보기 어려움

**ML/DL :**

- 학습이 끝난 모델을 이용해서 새로운 데이터에 대한 출력을 생성하는 단계
- Inference는 말 그대로 "모델의 생성 결과"이자 "예측 결과"

**LLM :**

- 결국은 딥러닝 모델로, 여전히 모델 실행(토큰 생성)이라는 기술적 관점에서 해석 필요
  ← 모델 입장에서는 inference 단계에서 결과값 출력
- inference = 확률적으로 가장 가능성 높은 단어를 예측하는 과정
- LLM에서는 inference 단계 ≠ 논리적 reasoning 단계

# [참고] LLM의 확률적 추론의 불안정성

애플, "대규모 언어 모델의 수학적 추론 능력에 한계 있어"

- LLM의 수학적 추론 능력을 평가하기 위한 새로운 벤치마크를 소개하는 논문 (GSM-Symbolic Benchmark)
- 동일한 문제의 수치값만 변경해도 모델 성능이 크게 변하는 불안정성 확인됨

  → LLM이 수치값의 변화에 민감하며 일관된 수학적 추론을 수행하는데 한계가 있음

**GSM Symbolic Template**

When {name} watches her {family}, she gets out a variety of toys for him. The bag of building blocks has {x} blocks in it. The bin of stuffed animals has {y} stuffed animals inside.The tower of stacking rings has {z} multicolored rings on it.{name} recently bought a tube of bouncy balls, bringing her total number of toys she bought for her {family} up to {total}. How many bouncy balls came in the tube?

#variables:
- name = sample(names)
- family = sample(["nephew", "cousin", "brother"])
- x = range(5, 100)
- y = range(5, 100)
- z = range(5, 100)
- total = range(100, 500)
- ans = range(85, 200)

#conditions:
- x + y + z + ans == total

---

Let T be the number of bouncy balls in the tube. After buying the tube of balls, {name} has {x} + {y} + {z} + T = { x + y + z } + T = {total} toys for her {family}.

Thus, T = {total} - { x + y + z } = <<{total}-{ x + y + z }={ans}>>{ans} bouncy balls came in the tube.

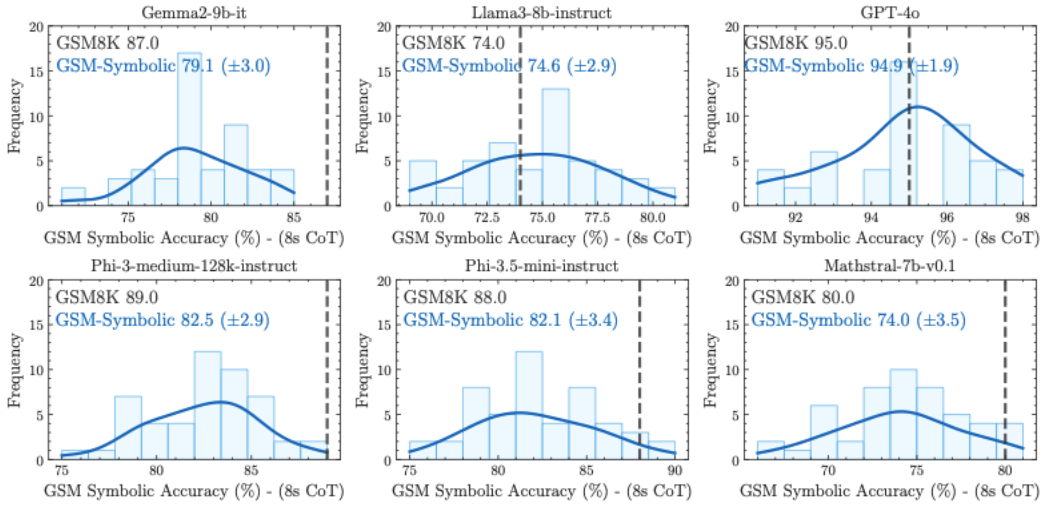



Figure 2: The distribution of 8-shot Chain-of-Thought (CoT) performance across 50 sets generated from `GSM-Symbolic` templates shows significant variability in accuracy among all state-of-the-art models. Furthermore, for most models, the average performance on `GSM-Symbolic` is lower than on `GSM8K` (indicated by the dashed line). Interestingly, the performance of `GSM8K` falls on the right side of the distribution, which, statistically speaking, should have a very low likelihood, given that `GSM8K` is basically a single draw from `GSM-Symbolic`.

*Source : https://aimatters.co.kr/news-report/ai-report/5823/, https://arxiv.org/pdf/2410.05229*

# 1. Reasoning

문제를 푸는 사고의 여정

사전적 의미:

- 논리적 사고를 통해 결론을 도출하는 과정
- 주어진 정보나 사실을 바탕으로 원인 → 결과, 전제 → 논리 전개 → 결론으로 나아가는 생각의 흐름
- (e.g.) 비가 오면 길이 젖는다 → 지금 길이 젖었다 → 아, 비가 왔겠구나..

**LLM**에서의 활용:

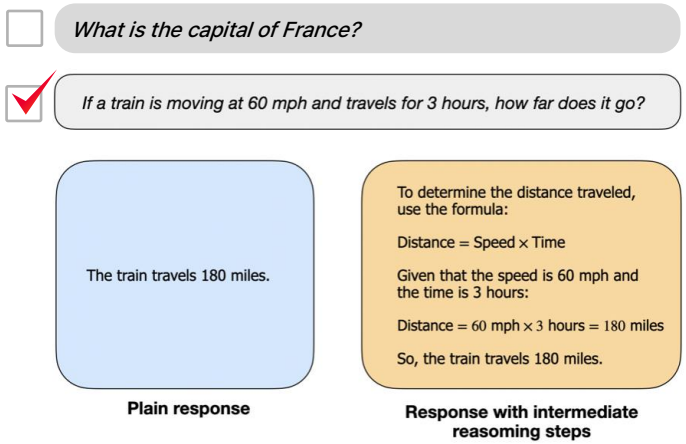
- 사고 능력이나 문제 해결 능력
- LLM이 단순한 암기나 패턴 응답이 아닌, **논리 구조를 따라가면 정답에 도달하는 과정**
  - (e.g.) 한국의 수도는 어디입니까? → non-reasoning
  - (e.g.) A는 B 보다 무겁고, B는 C 보다 무겁다. 누가 가장 무겁나? → reasoning

# 1. LLM Reasoning

Key Point

단순 답변 생성이 아닌 목표 달성 위해 "판단하고 계획하는 **사고의 과정, Process of Planning**"으로 봐야 함

- Reasoning : 주어진 정보로부터 논리적으로 결론을 도출하거나, 다음 행동을 결정하는 사고 과정
- AI Agent가 "왜 이 행동을 해야 하는가"를 판단하게 하는 과정




A regular LLM may only provide a short answer (as shown on the left), whereas reasoning models typically include intermediate steps that reveal part of the thought process. (Note that many LLMs who have not been specifically developed for reasoning tasks can also provide intermediate reasoning steps in their answers.)

AI Agent의 Reasoning 역할 (현재 기준) :

| Agent 역할 | 내용 |
|---|---|
| 행동 판단 | • 도구를 쓸 것인지, 응답을 생성할 것인지를 결정 |
| 작업 계획 | • 단계를 나누고 어떤 순서로 처리할지 구상 |
| 결과 검토 및 수정 | • 행동 결과를 평가하고 다시 사고(재추론)하여 행동 수정 |
| 목표 일치 판단 | • 현재 정보가 사용자의 목표를 충족하는지 확인 |

➔ 단순한 응답 생성기를 넘어,
상황/행동을 판단하고 실행을 조정하는 기능 수행

※ Remark
- System 1 : 빠르고 본능적인 판단. 기존 LLM의 즉각적인 패턴 매칭 응답 (Fast Thinking)
- System 2 : 느리고 논리적인 연산. AI Agent가 수행하는 추론(Reasoning) 및 계획(Planning) 과정 (Slow Thinking)
- 과거(2023) LLM은 즉각적 패턴 매칭(System 1) 위주였으나, 현재는 강화학습 기반의 내부 사고 과정(System 2)이 통합된 추론 모델로 확장됨

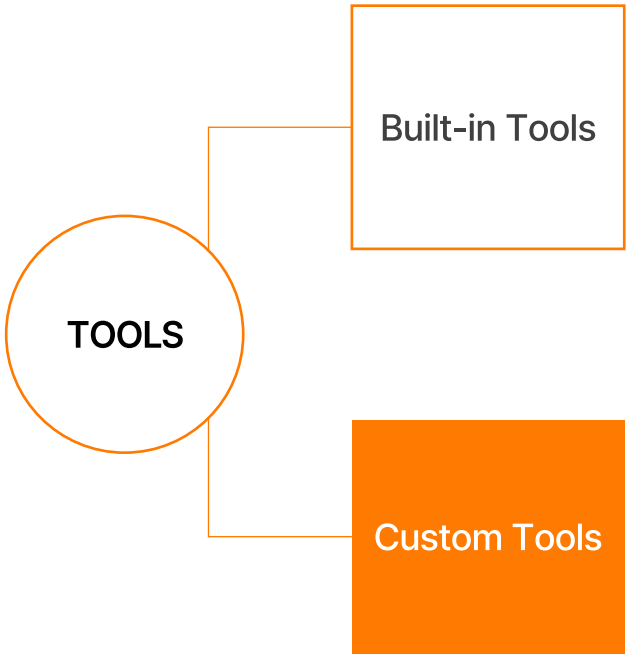
*Source : https://www.linkedin.com/pulse/understanding-reasoning-llms-sebastian-raschka-phd-1tshc/*

# 2. 전략적 도구

## LLM Reasoning 위한 보완 방식

| 구분 | 보완 방식 | 핵심 개념 | 역할/목적 |
|---|---|---|---|
| Model | Inference-time Scaling | 추론 시간에 탐색 연산량을 동적으로 투입/조절 | 복잡한 문제 해결 위한 심층 추론 고려 |
| | Reasoning Tuning | 전문가 추론 궤적 데이터(SFT)와 보상 모델(RL) 기반 튜닝 | 모델 자체의 논리적 사고 및 역추적 내재화 |
| | Reasoning Distillation | 대형 추론 모델의 사고 토큰/경로를 소형 모델 형태로 증류 | 경량 모델의 추론 효율성 및 비용 최적화 |
| | Scratchpad | 중간 계산 및 추론 메모를 컨텍스트 내에 기록하도록 유도 | 다단계 추론의 투명성과 디버깅 확보 |
| Context | Chain-of-Thought | 중간 사고 과정을 단계별로 서술하도록 유도하는 프롬프트 | 단순 응답을 넘어 추론 단계 명시화 |
| | Self-Consistency Decoding | 여러 추론 경로를 병렬 실행 후 다수결 집계 | 추론의 일관성 및 정답 신뢰도 향상 |
| Tool | Tool Augmentation | 계산기, 웹검색, 코드 실행기 등의 외부 도구와 연동 | 지식 최신화 및 결정론적 연산 한계 극복 |
| | Agentic RAG | 검색 쿼리 재작성, 검색 결과 평가 및 반복 검색 결합 | 정보 왜곡 방지 및 고품질 근거 기반 추론 |
| Harness | Feedback Loop | 실행기의 피드백을 받아 계획 재수립 | 실제 환경 기반의 자가 수정(Self-Correction) |
| | Cross-Validation | 생성자와 평가자의 교차 확인 | 단일 모델 편향 및 환각 제어 |

# 2. Tools



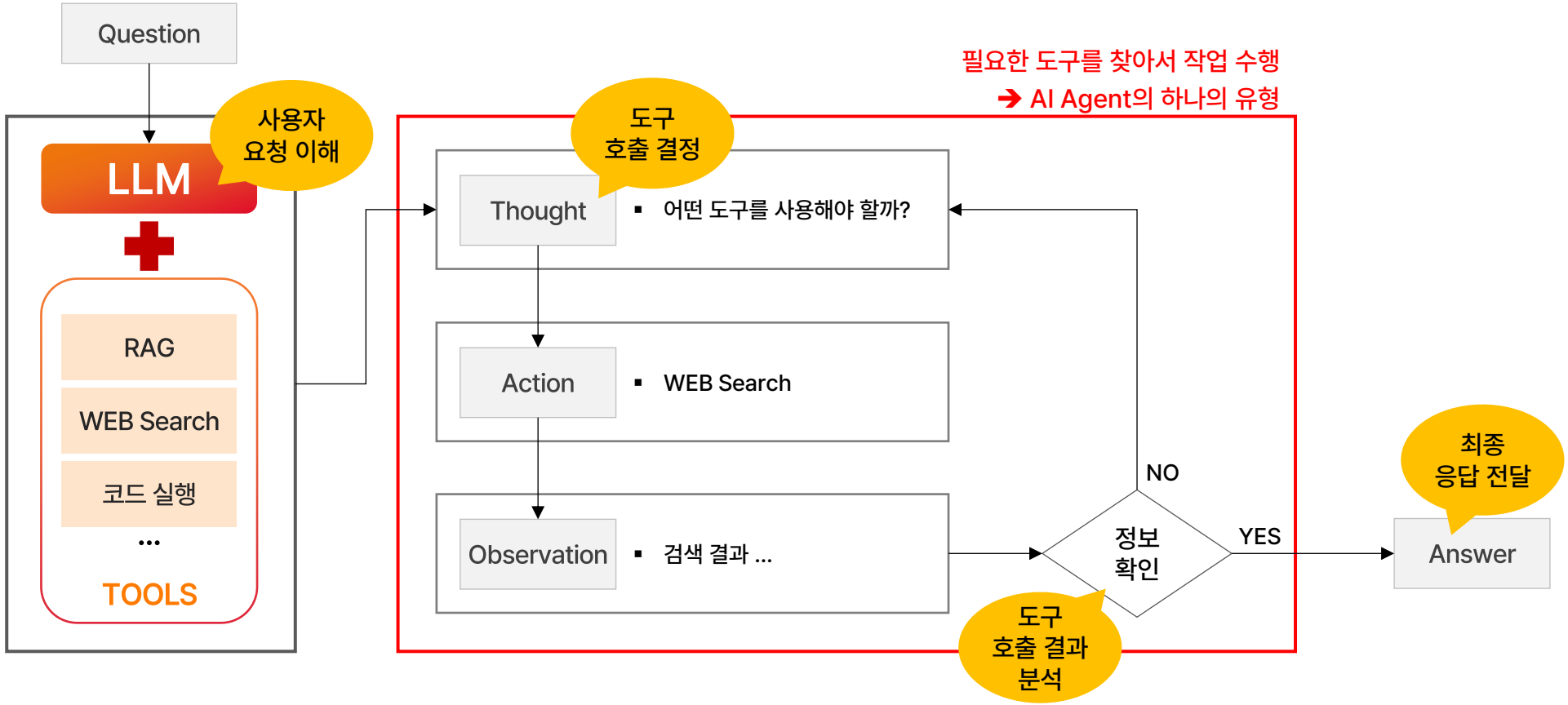

- LangChain에서 제공하는 사전에 정의된 도구(tool)와 툴킷(toolkit)
- Tool : 하나의 기능만 제공하는 도구
- Toolkit : 특정 목적에 맞게 연관된 도구를 묶어놓은 도구 셋트
- https://docs.langchain.com/oss/python/integrations/tools

- 사용자가 직접 도구를 사전에 정의하여 사용
- langchain_core.tools 모듈에서 제공하는 @tool 데코레이터를 사용하여 함수를 도구로 변환
- 데코레이터를 통해 일반 파이썬 함수를 도구로 변환하여 자동화된 업무 프로세스에 반영
- LLM이 함수 목적과 매개변수를 이해할 수 있도록 명확한 Type Hint 및 Docstring 작성 필수

# 1. Tool Calling

모델이 적절한 도구를 반복적으로 호출하고 결과를 받아 문제 해결

- 도구 호출을 하면 모델이 하나 이상의 도구가 호출되어야 하는 시기를 감지하고 해당 내용을 전달할 수 있어야 함
- 목적에 맞는 유효하고 유용한 도구 호출(Tool Calling) 결정 및 실행

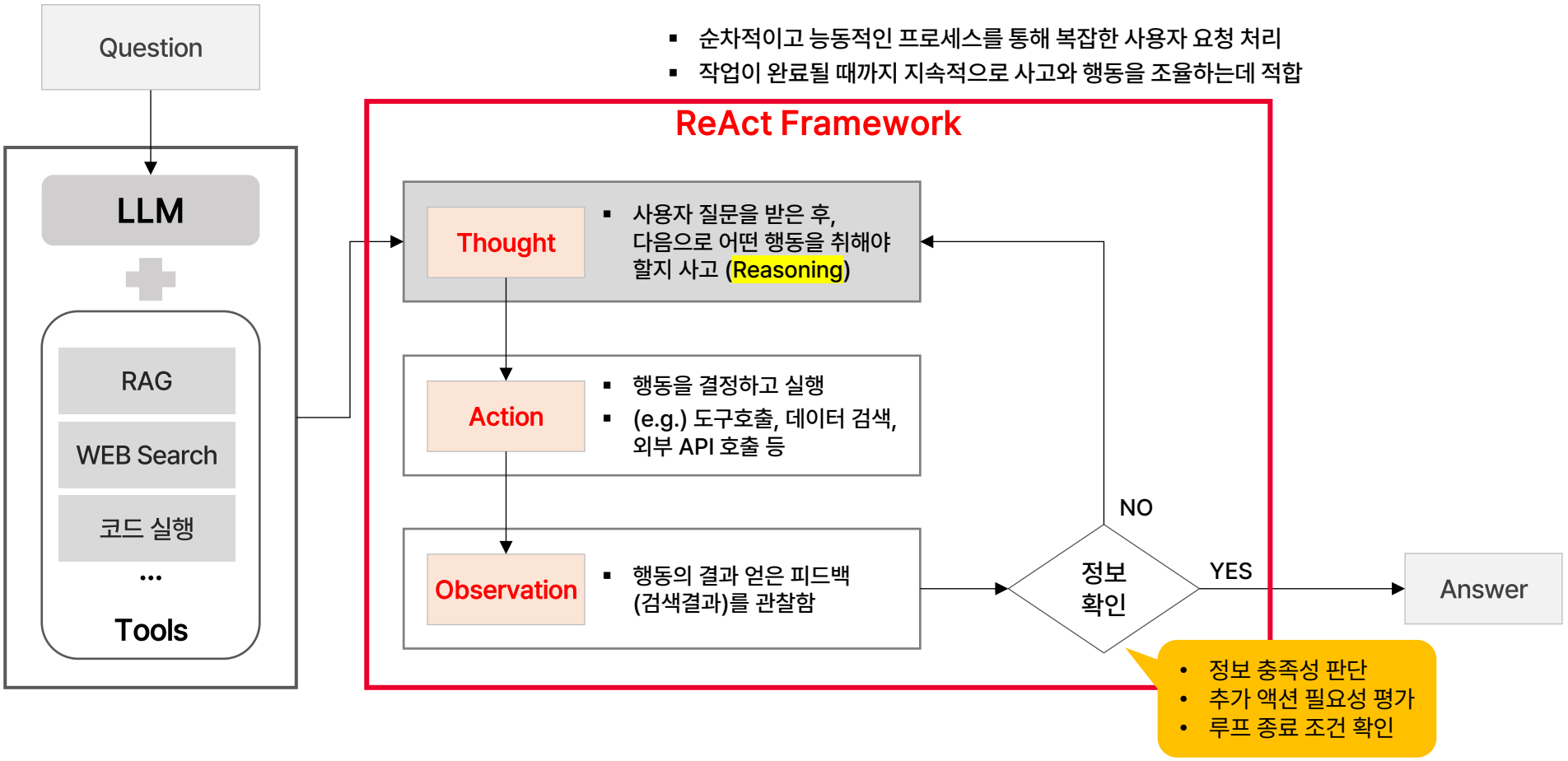



*Reference : https://python.langchain.com/v0.1/docs/modules/agents/agent_types/tool_calling/*

# 2. ReAct Framework

Key Point

Workflow : ReAct = **Reasoning** + Acting



# 3. ReAct Framework 확대/발전

**최초 제안 (2022):**

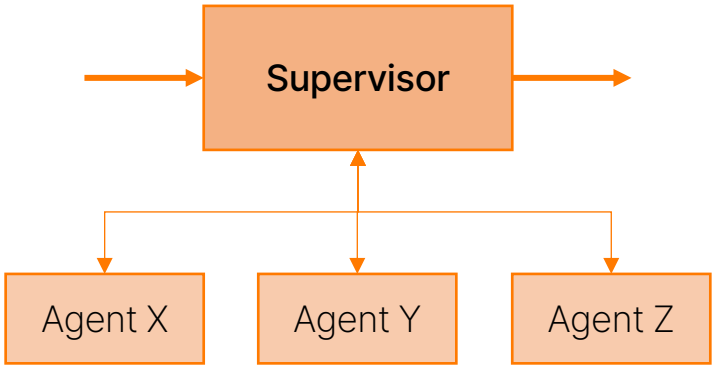
- ReACT: Synergizing Reasoning And Acting in Language Models (Princeton Univ, Google Research)
- Naïve React : 순차적 실행 (Thought → Action → Observation → Thought)
- Contribution :
  - Reasoning과 Acting을 최초로 하나의 시스템으로 결합한 연구
  - ReAct 이후 Tool-use, Memory, Multi-Agent 설계 프레임워크들의 토대
  - 이후 개발 프레임워크 성숙 가속화(LangGraph 대표적)

**Multi-Agent로의 확대 (2024~Current) :**

- 단일 에이전트의 프로세스 확장/확대
- Multi-Agent 설계 :
  - Orchestration : Hierarchical, Handoff, Supervisor 등 다양한 동적 협업 패턴
  - Graph-based Collaboration : 단순 순차 실행이 아닌, 순환(Loop)과 분기 처리(Conditional Edge)가 가능한 워크플로우 설계
- Context, Memory 관리 필요성 :
  - 단기 메모리 : 에이전트 루프 내 메시지 목록 및 상태 스키마 관리(Checkpointer)
  - 장기 메모리 : 세션 간 사용자 프로필 및 과거 실행 내역을 별도 DB 저장/조회

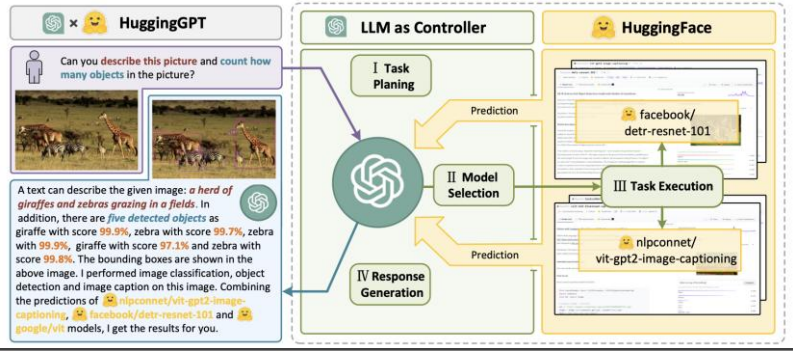
# [참고] Supervisor

입력된 요청을 분석해 가장 적합한 에이전트에게 실행을 위임하고, 그 결과를 종합하여 답변을 완성하는 Orchestration 패턴



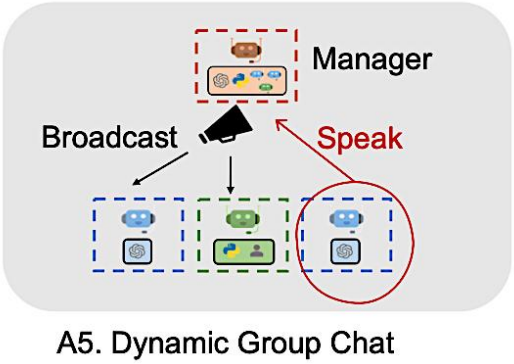

개념적 기원 : HuggingGPT (2023.03)

LLM이 직접 문제를 풀지 않고, 관리자(Controller)로서
하위 모델(Experts Models)에게 일을 시킨다




구조적 기원 : AutoGen (2023.08)

Multi-Agent 간의 협업 및 순서 제어를 프레임워크 단위로 구현한 최초 사례




A5. Dynamic Group Chat

*Source : https://arxiv.org/pdf/2303.17580, https://arxiv.org/pdf/2308.08155*

# 4. Tool Calling Process

**도구 생성**
**@tool**

- LLM에게 제공할 도구 정의
- 파이썬 함수 위에 @tool decorator로 구분
- 여러 개의 도구는 리스트로 묶어서 정의함

```
tools = [A, B, C, D]
```

**Agent 생성**

- create_agent 활용하여 정의
- 실행에 필요한 모델, 도구 및 시스템 프롬프트로 구성

```
llm = ChatOpenAI()
agent = create_agent(
        model=llm,
        tools=tools,
        system_prompt=(
                "You are a helpful assistant."
                "Answer the user's question using the provided tools.")
)
```

**Agent 실행**

- 실행 시 필요한 역할과 user query를 함께 전달하여 실행
- invoke or stream으로 실행

```
result = agent.invoke({
        "messages": [{
                "role" : "user",
                "content" : user_input,
        }]
})

Print(result["messages"][-1]["content"])
```

# 3. Agent Protocol

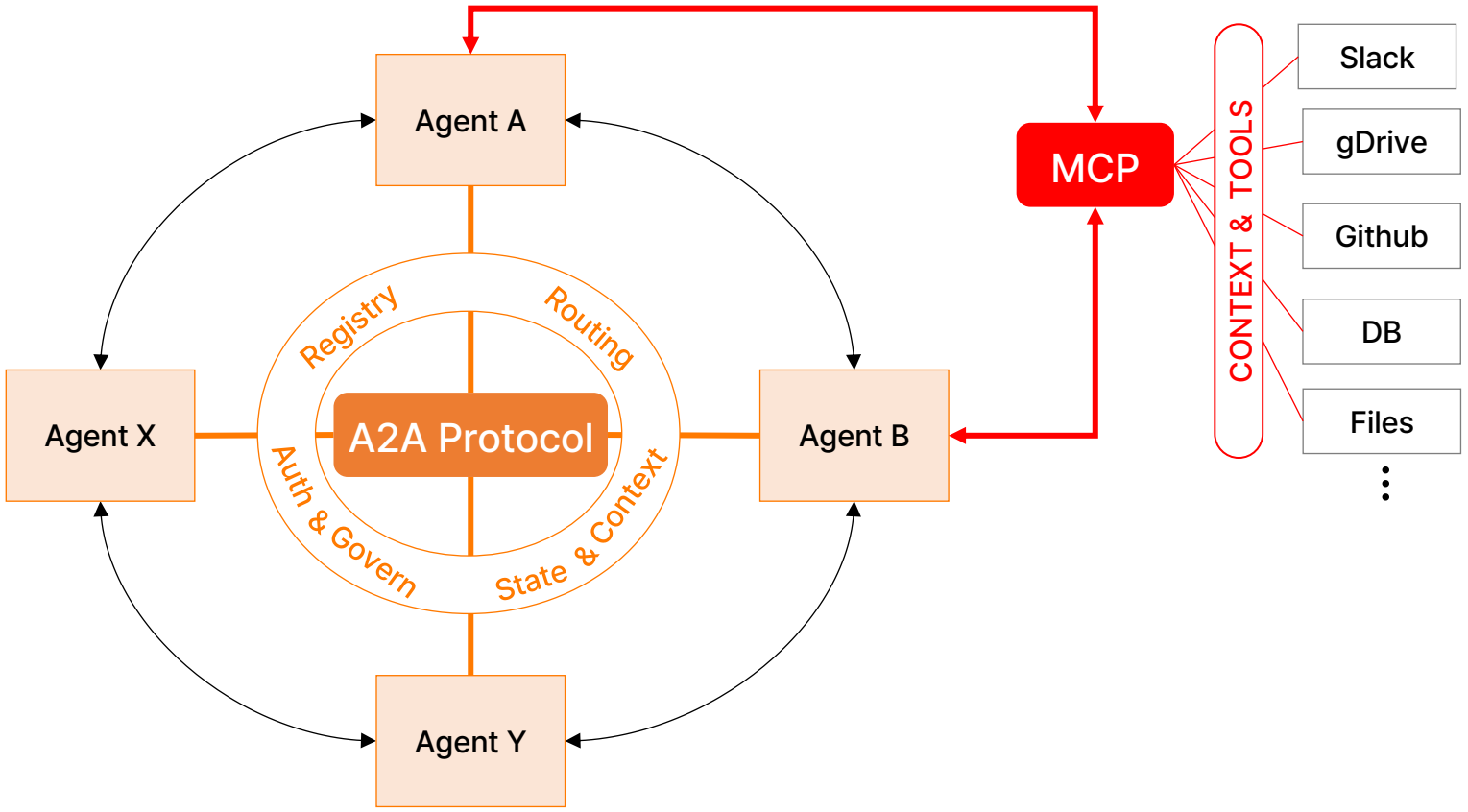
모델은 자연어로 소통한다.

모델이 맥락을 파악하고 이를 바탕으로 작업을 수행하려면,
먼저 그 맥락을 모델이 이해할 수 있는 형식으로 변환하는 과정이 필요하다.

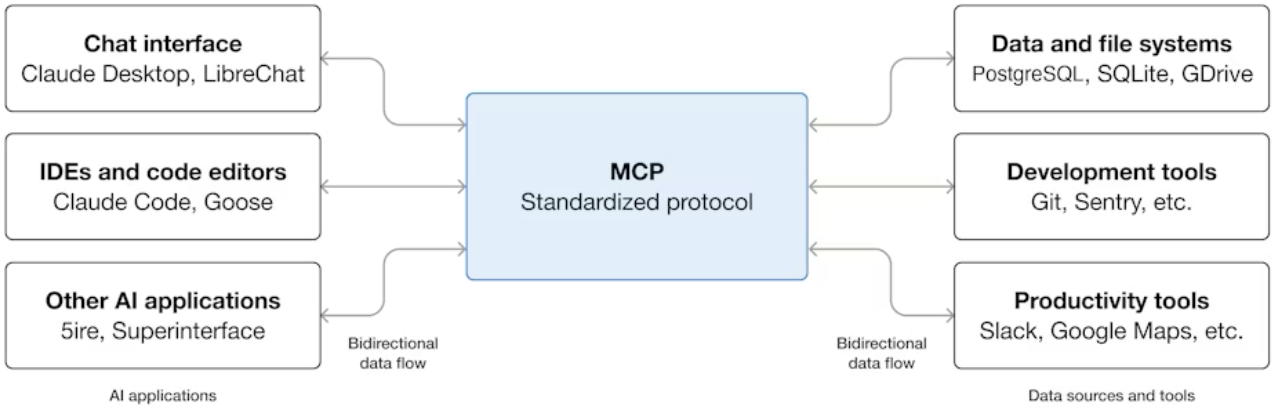
2024.11
Theodora Chu, Anthropic Product Manager

# 3. Agent Protocol

2026.H1



# 1. MCP, Model Context Protocol




## 목적 :

- 에이전트와 다양한 데이터베이스를 쉽게 연결 할 수 있는 오픈소스 프로토콜
- 이를 통해 AI 시스템이 외부 정보에 직접 접근하여 성능과 활용도를 높일 수 있음

## 기능 및 특징 :

- 표준화된 인터페이스 : 다양한 AI시스템과 데이터 소스 간의 상호 운용성을 높이기 위해 공통의 규칙과 형식 정의
- 통합의 용이성 : 각 데이터 소스에 맞춰 별도의 인터페이스 코드를 작성할 필요 없이, 통합된 프로토콜로 여러 데이터 소스와 연결 할 수 있음

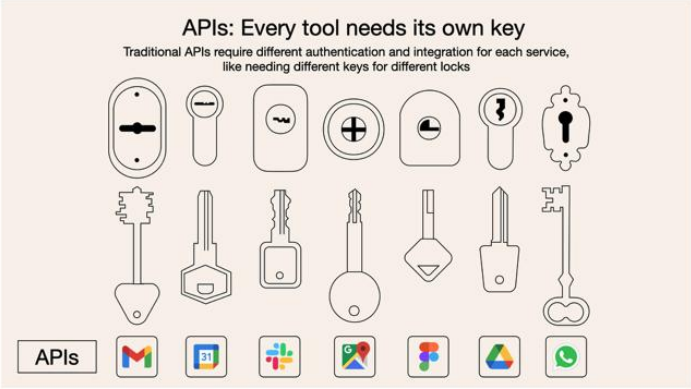
Key Milestones :

- Phrase 1. 초기 런칭 (2024.11)
  : Spec 공개, Python/TypeScript SDK 배포
- Phrase 2. 생태계 확산 (2025.1H)
  : Java, Kotlin, Go 등 주요 언어 SDK 지원 확대 및 Community Server 활성화
- Phrase 3. 표준 정착 (2025.2H)
  : 주요 IDE(Cursor, VS Code) 및 Agent Framework (LangChain, LlamaIndex) 네이티브 통합 완료

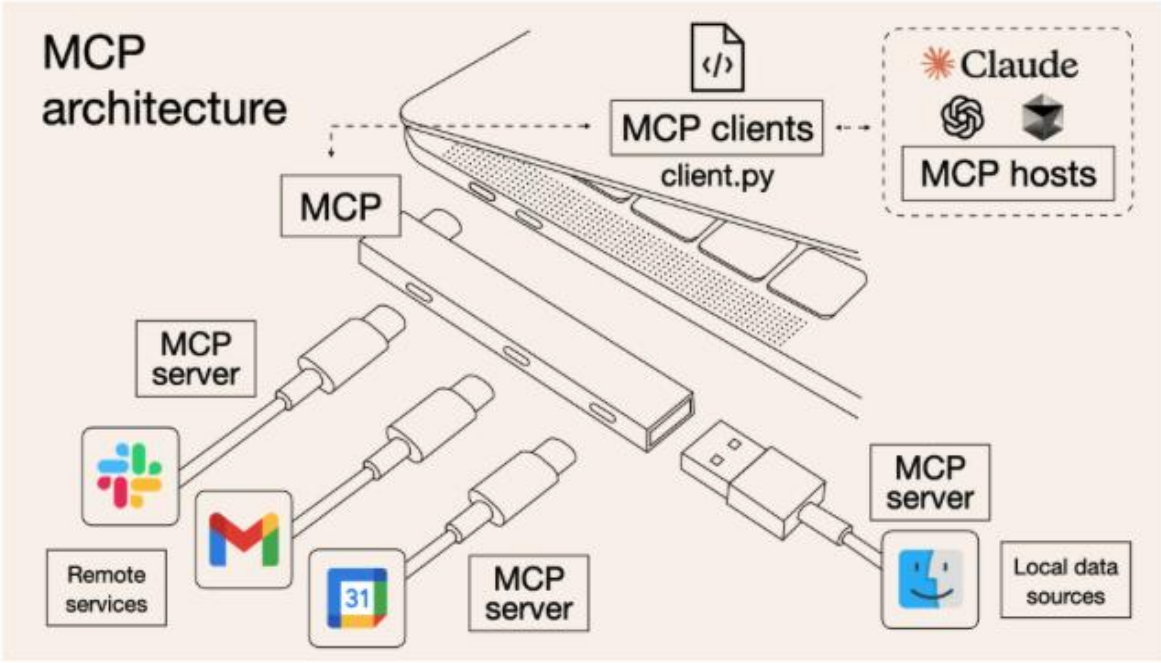
*Source : https://modelcontextprotocol.io/docs/getting-started/intro, https://github.com/modelcontextprotocol*

# 1-(1). MCP Architecture

Standardized protocol that connects AI agents to various external tools and data sources




Traditional APIs require developers to write custom integrations for each service or data source



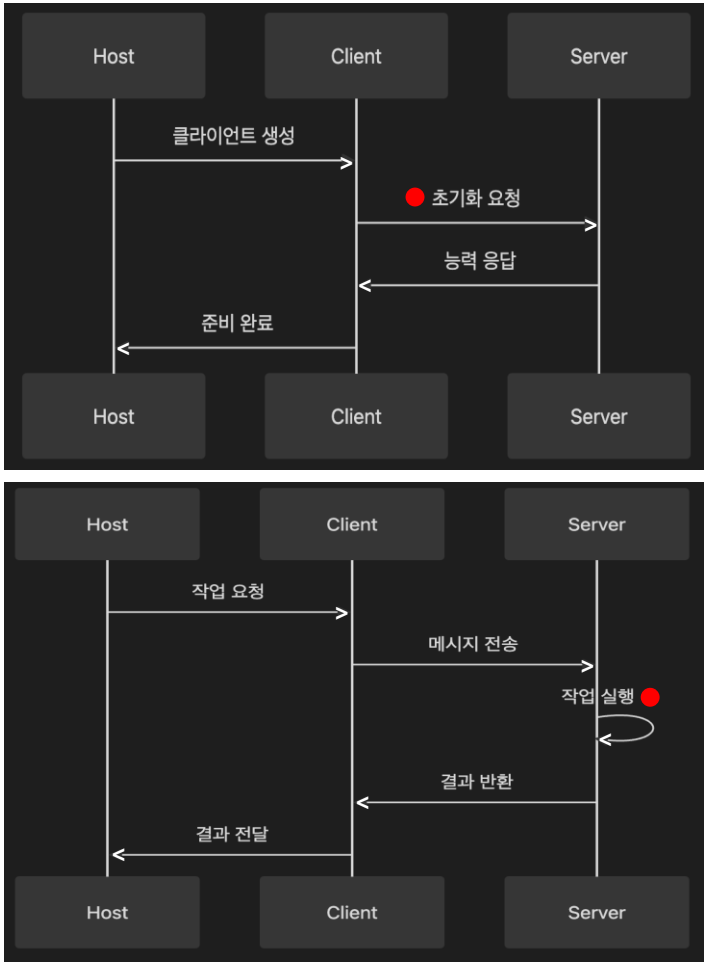

The **Model Context Protocol (MCP)** is a standardized protocol that connects AI agents to various external tools and data sources

- Just as USB-C simplifies how you connect different devices to your computer, MCP simplifies how AI models interact with your data, tools, and services.
- Marketplace, Smithery (https://smithery.ai)

*Source : https://norahsakal.com/blog/mcp-vs-api-model-context-protocol-explained/*

# 1-(2). Host-Client-Server

명확한 역할과 책임을 가지며, 이를 통해 유연하고 확장 가능한 시스템 구현 가능




## MCP Host

- 회사의 팀장처럼, 여러 부서(서버)와 연결을 조율하는 중앙 통제실 역할
- 사용자와 직접 상호 작용하는 어플리케이션
- 주요 역할 :
  - LLM과의 통합 및 전체 데이터 흐름 조율
  - 다양한 서버의 응답을 취합 및 정제하여 AI모델에 전달

## MCP Client

- 팀장(Host)과 각 부서(Server) 사이에서 업무 요청/회신을 담당하는 전담 인력
- 호스트 내부에서 동작
- 각 MCP Server와 1:1로 연결되는 중개자 역할을 하는 컴포넌트
- 주요 역할 :
  - 서버와의 독립적인 연결 및 세션 관리
  - 프로토콜 협상, 기능 교환, 요청/응답 메시지 라우팅

## MCP Server

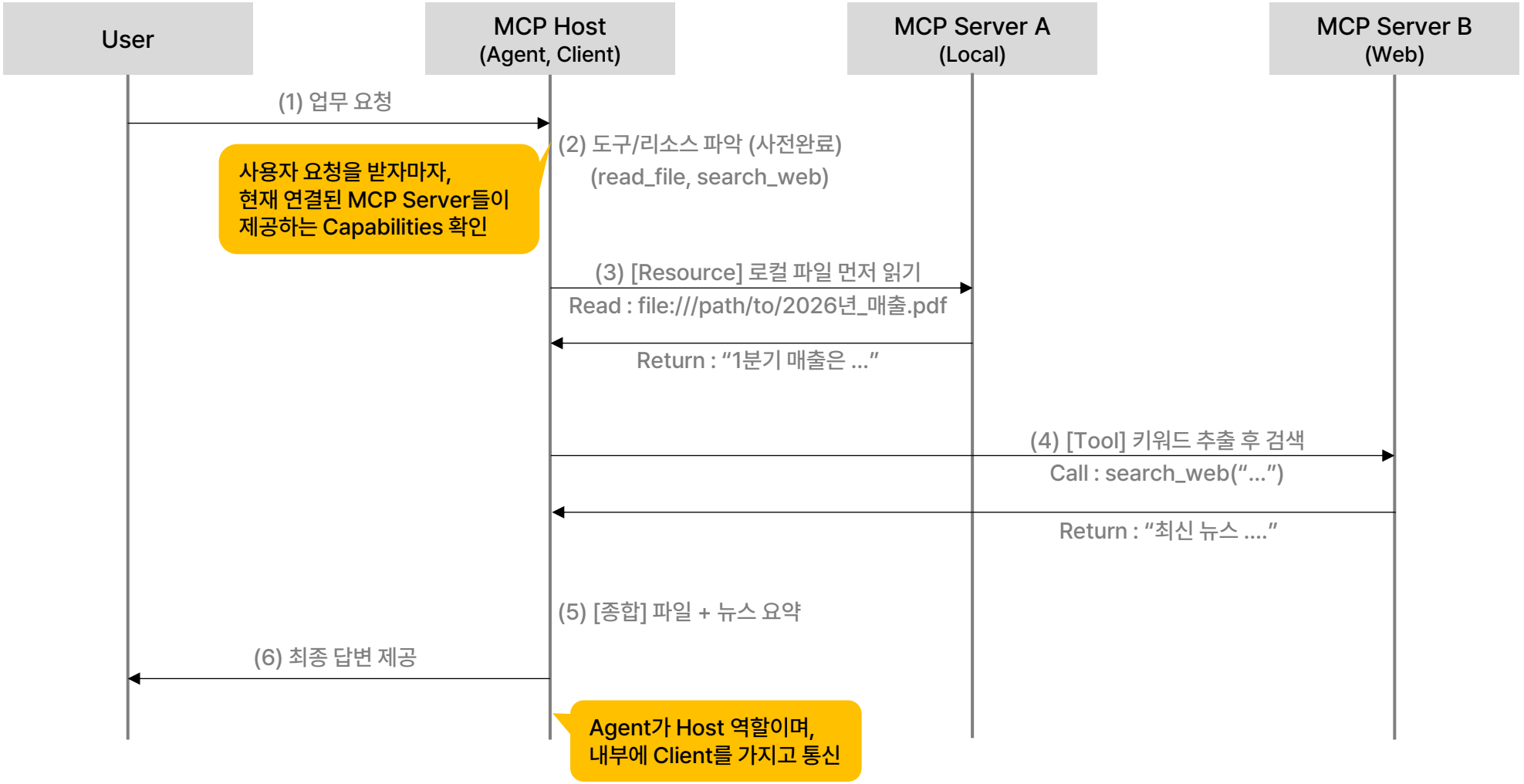
- 회사의 각 부서처럼, 특정 업무(데이터 제공, 도구 실행 등)를 담당
- AI 모델이 사용할 수 있는 3대 핵심 요소(Resources, Tools, Prompts) 제공
- 주요 역할 :
  - MCP 프로토콜 통해 클라이언트의 요청을 받아 정보 제공 또는 작업 실행
  - 로컬 데이터(파일, DB)나 원격 서비스(API, 클라우드 등)와 연결

※ Remark
- Resources : File, API, DB 등 모델이 읽을 수 있는 데이터 컨텍스트
- Tools : 모델이 외부 시스템에 개입하여 실행할 수 있는 액션 (함수 호출)
- Prompts : 사용자가 쉽게 작업을 시작할 수 있도록 정의된 프롬프트 템플릿

*Source : https://wikidocs.net/268794, https://modelcontextprotocol.io/specification/2025-06-18*

# [참고] Sequence Diagram

내 PC의 “2026년_매출.pdf” 파일을 읽고, 관련된 최신 뉴스 동향을 웹에서 검색하여 종합 요약해 주세요.



# 1-(3). MCP 필요성

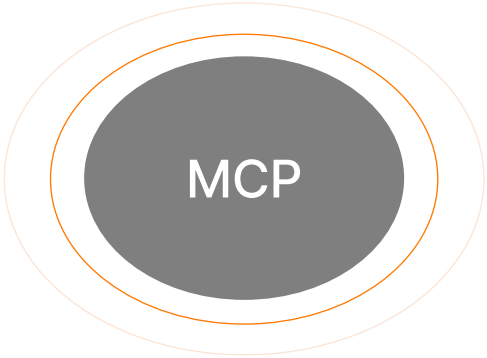
AI Application과 외부 데이터, 도구를 연결하는 개방형 표준 프로토콜

## 연결 파편화 해소

- Before : 모델(Host)와 서비스(DB, Git, Slack 등) 마다 개별 커넥터를 매번 개발
- MCP :
  - 1개의 표준 규격으로 개발
  - 모든 AI Host에서 즉시 Plug & Play

## 로직 분리

- Before : 에이전트 코드 내부에 API 호출 및 도구 로직을 하드코딩
- MCP :
  - 외부 데이터/도구를 독립 서버로 격리,
  - 에이전트 수정 없이 도구 추가/교체 가능




## 표준 규격 통합

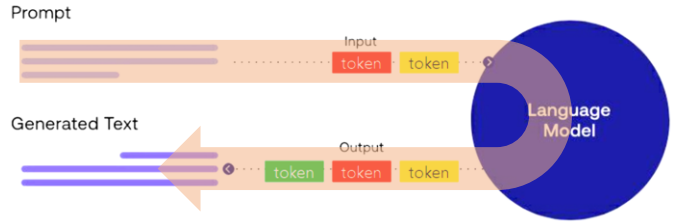
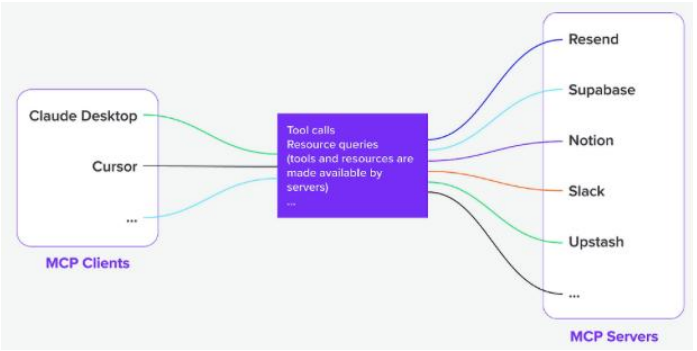
- 데이터와 도구를 단일 프로토콜로 규격화
  - Resources : File, DB 등 데이터 컨텍스트
  - Tools : API 호출, 코드 실행 등 외부 작업 수행
  - Prompts : 표준화된 작업 지시 템플릿

## 보안 및 권한 제어

- 모델에게 인프라 직접 권한을 주지 않고, MCP Server가 권한 통제
- 기업 보안, 접근 범위, 감사 추적을 결정론적 경계에서 관리 가능

# 1-(4). LLM vs MCP

기존 LLM 구조를 넘어서, AI시스템의 Context를 제대로 관리하는 새로운 표준

| 항목 | 기존 LLM 구조 | MCP 기반 구조 |
|---|---|---|
| 프로세스 |  |  |
| 기억 (메모리) | • **Stateless (메모리 없음)** | • **명시적 Context 객체로 상태 저장 및 전이 가능** |
| 입력 방식 | • 자연어 프롬프트 기반<br>• 비구조적, 단방향 | • 구조화된 JSON/YAML 기반<br>• Context Object |
| 작업 전이 | • 수동으로 복&붙 통해 전달, 사용자 정의 로직으로 처리 | • Context를 다른 에이전트나 모델에게 그대로 전달 |
| 다중 에이전트 입력 | • Memory 공유가 어렵고, 그에 따른 상태 추적 안됨 | • MCP context에 agent_id, task, memory 등 명시하여 처리 가능 |
| 문맥 흐름 추적 | • 거의 불가능<br>• 어떤 세션에서 무슨 일이 있었는지 알 수 없음 | • 각 단계의 context 기록<br>• 흐름 추적 및 디버깅 가능 |
| 상호 운용성 | • 모델/시스템 간 context 전달 어려움 | • 공통 포맷으로 모델 간 interoperability 확보 |
| 보안/제한 설정 | • 프롬프트 내 명시 불가능 | • Context에 역할/도구 제한 , 사용자 권한 명시도 가능 |

*Source : https://digitalbourgeois.tistory.com/911*

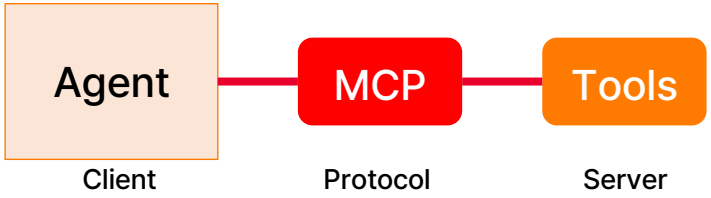
# 1-(5). Tool Calling vs MCP

서비스 목적과 요구사항에 맞춰 강한 결합(Native)과 느슨한 결합(MCP) 중 적절한 아키텍처를 선택해야 함

## Native Tool Calling



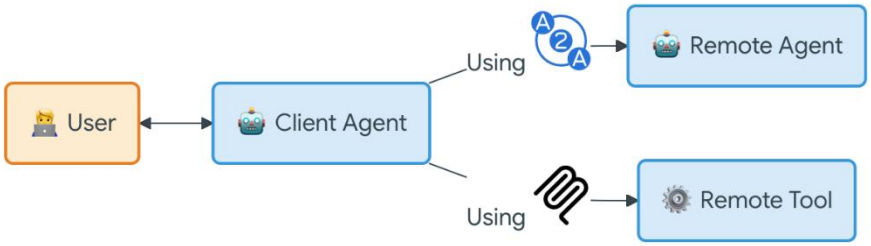

## MCP

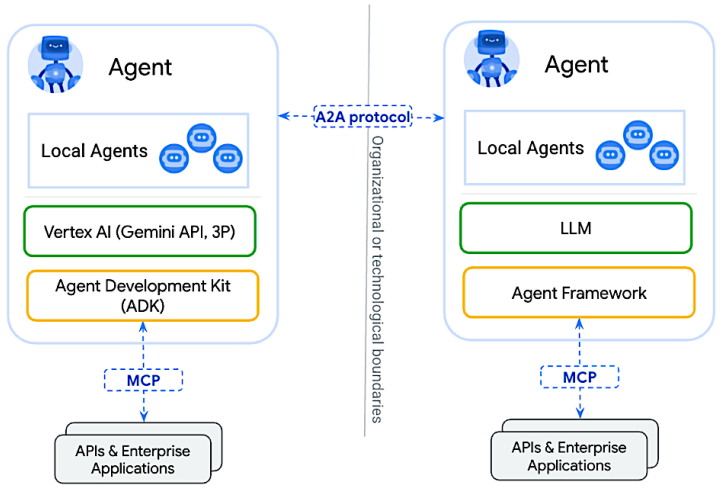



| | Native Tool Calling | MCP |
|---|---|---|
| 아키텍처 성격 | • 강한 결합 (Tightly Coupled)<br>• 에이전트 어플리케이션 내부에 도구(코드)를 직접 구현 | • 느슨한 결합 (Loosely Coupled)<br>• 클라이언트(Agent)와 서버(Tools)를 분리하여 프로토콜로 통신 |
| 도구 정의 및 관리 | • 개발자가 명시적으로 스키마를 정의<br>• LLM 프롬프트에 직접 반영 (+ 유지보수) | • MCP Server가 동적으로 도구 목록을 제공<br>• 에이전트는 도구 목록을 탐색 |
| 장점 | • 세밀한 제어 : 실행 과정과 에러 핸들링을 코드 레벨에서 완벽히 통제 가능<br>• Low Latency : 내부 함수를 직접 호출하므로 통신 오버헤드 최소화 | • 높은 확장성 : 코드 수정 없이 서버 연결만으로 수많은 외부 도구를 일괄적으로 확장 가능<br>• 재사용성 : 하나의 MCP Server를 여러 다른 에이전트가 공유 가능 |
| Best For | • 목적형 에이전트<br>• 사내 DB/API 연동 등 보안 로직 적용이 고려되어야 하는 경우 | • 범용 에이전트<br>• SaaS, Local File 등 다양한 외부 생태계와의 통합이 빈번한 경우 |

# 2. A2A Protocol

Enabling communication and collaboration between AI agents across different platforms and services




## 개요 :

- 에이전트들이 상호 운용할 수 있도록 돕는 오픈소스 프로토콜
  (현재 리눅스 재단 산하 프로젝트로 150개 이상 글로벌 기업 참여)
- MCP가 외부 도구와의 연결을 담당한다면,
  **A2A는 에이전트 간 협업과 통신을 전담**
- 에이전트들은 서로의 기능을 활용하여 복잡한 비즈니스 프로세스를 자동화하고, 개입을 최소화 할 수 있음

## What A2A is Not :

- Not an Agent Development Kit. 에이전트 간 통신 계층 역할 담당
- Not a Sub-agent or Tool-call Protocol
- Not a replacement for MCP (두 프로토콜은 상호 보완적 관계)
- Not an interactive messaging app
  : 사용자용 메시징 앱이 아니라,
  자율 에이전트 위한 Machine-to-Machine 프로토콜

*Source : https://a2a-protocol.org/latest/#how-a2a-works-with-mcp*

# 2-(1). 주요 기능 및 고려사항

## A2A는 Multi-Agent와 달리 프로세스, 네트워크, 조직, 벤더 경계를 넘음 → 대부분 "경계 넘기"에서 발생하는 비용

| 구분 | 주요 기능 | 고려사항 |
|---|---|---|
| **Capability Discovery & Intelligent Routing** | • 에이전트는 JSON 형식의 **Agent Card** 통해 자신의 역량과 인터페이스 광고<br>• 오케스트레이터가 적절한 에이전트를 식별하고 작업 할당 | • 발견 전략 : (1)Well-known URI, (2)Curated Registry, (3)직접 입력<br>• 스키마 공백 : Agent Card는 미디어 타입만 선언, 본문의 JSON 구조는 표준화 되지 않음 → 형식만 호환, 실제 상호운용은 보장 안됨 |
| **Task Management & Collaboration** | • **클라이언트가 작업을 요청**하면 **서버가 수행**하고 결과물 반환<br>• 오케스트레이터가 전체 워크플로우 조율, 에이전트 간 Context, 응답, 산출물, 지시는 메시지로 전달 | • Task Management :<br>▪ Task 상태 기반 실행 : Submitted, Working, Input_Required 등<br>▪ LLM 실행 시간 가변적 : 단순 풀링 대신 SSE 스트리밍, 웹훅 사용<br>• 운영 부담 : 지속 연결, 타임아웃 관리, 웹훅 인증 등<br>• 오류/복원력 : 프로토콜은 상태, 에러 객체만 정의. Retry, Circuit Breaker는 Client 담당 → 부분 실패, 네트워크 분할은 개발 필요<br>• 디버깅 난이도 : 전체 추적 위해 W3C Trace Context + OpenTelemetry |
| **UX Negotiation & Enterprise Security** | • 단순 데이터 전송 넘어, 입출력 모드 기반으로 **어떻게 표현할지 까지 조율** 가능하도록 설계<br>• gRPC 지원 및 Agent Card 서명으로 엔터프라이즈급 **고성능, 고보안 통신 지원** | • Security Scheme : Agent Card에 OAuth2, OIDC, API Key, mTLS 선언<br>• Agent Card 서명 : 카드 자체가 공격 가능하기 때문 서명 요구<br>• Delegation Downscoping : 오케스트레이터가 하위 에이전트에 큰 토큰을 그대로 넘기면 권한 확대 발생, 스펙 중심으로만 전달<br>• 토큰 탈취 대비 : Sender-constrained 토큰<br>• 버전 협상 : 조직 간 연동 시 버전 협상 이슈 발생 가능성 |

*※ Remark*

- A2A 1.0은 안정화된 규격. 상당수 문제점은 애플리케이션, 인프라 계층에 남아 있음
- W3C Trace Context + OpenTelemetry : 하나의 요청이 여러 에이전트를 거쳐 갈 때, "이 요청이 어디를 지나 어디서 느려지거나 실패했는지"를 추적하는 방법.
  - W3C Trace Context : 요청마다 공통 추적번호(traceparent)를 붙여 여러 에이전트를 지나도 같은 요청임을 구분
  - OpenTelemetry : 그 기록을 모아 전체 흐름을 볼 수 있게 해주는 도구
- OAuth2 : 비밀번호를 넘겨주지 않고, "무엇을 할 수 있는지"만 담은 임시 출입증(토큰)을 발급하여 다른 에이전트에 권한을 주는 방식 (e.g., 호텔 카드키 처럼, 정해진 방에 정해진 기간 동안만 열 수 있도록 함)
- OIDC, OpenID Connect : OAuth2에 얹어 너는 누구인가를 증명하는 신원 확인 방식 (OAuth2가 권한, OIDC는 신원)
- mTLS, Mutual TLS : HTTPS는 서버만 신원을 증명하지만, mTLS는 클라이언트와 서버 양쪽이 서로 인증서를 제시해 서로를 확인하는 방식 (양방향 신원 확인 + 암호화)

# 2-(2). Progress & Adoption

2025-08

---

인공지능

These protocols will help AI agents navigate our messy lives

## 구글·앤트로픽, '에이전트 시대' 위한 프로토콜 구축 나선다

구글과 앤트로픽은 디지털 환경에서 AI 에이전트를 더 효율적으로 활용하기 위한 프로토콜을 개발하고 있다. 하지만 보안·개방성·효율성 등 여전히 풀어야 할 과제도 많다.

Peter Hall 2025년 8월 13일

AS-IS :

- 실제 업무를 수행할 수 있는 'AI Agent'를 내세우는 기업들이 빠르게 늘고 있지만, 지금까지의 평가는 썩 긍정적이지 않다.
- 에이전트들이 디지털 환경에서 쓰이는 다양한 기술과 아직 원활하게 연동되지 않기 때문이다.

PROGRESS :

- 두 프로토콜은 상호 보완적인 성격을 띤다. 여러 설계적 장점을 지니고 있지만, 아직 초기 단계에 머물러 있고 새로운 한계와 문제도 드러내고 있다.
- 핵심 과제 :
  - 보안 : 유해한 접근/행동에 대한 사전 차단이 가능한 보안 체계 (e.g. indirect prompt injection attack)
  - 개방성 : 오픈소스이나 소유/운영 방식 차이 (MCP – Anthropic, A2A – Linux Foundation)
  - 효율성 : HUMAN↔AI 간에는 토큰이 유의미하지만, Agent↔Agent 간에는 굳이??!!

*Source : https://www.technologyreview.kr/구글·앤트로픽-에이전트-시대-위한-프로토콜-구축/*

# 2-(2). Progress & Adoption

2026-06

Blog / analysis

## A2A Protocol Adoption: Where Things Stand in Mid-2026

Google's Agent2Agent protocol hit 150 organizations and landed in every major cloud. But adoption tells a more complicated story than the press releases suggest. Here is what the numbers actually say.

analysis · ecosystem · 2026-06-27

a2a | agent-to-agent | protocol | adoption | mcp | multi-agent | interoperability | 2026

---

A year ago, Google launched the Agent2Agent protocol with more than 50 technology partners and a promise: AI agents from different vendors and frameworks would finally be able to talk to each other. Twelve months later, the numbers look strong on paper. But the real picture of A2A adoption is messier than the press releases suggest.

The result: developers who needed agent-to-tool connectivity chose MCP because it worked in minutes. Developers who needed agent-to-agent coordination often found they could approximate it with MCP's existing primitives rather than managing a second protocol.

*Source : https://agentndx.ai/blog/a2a-protocol-adoption-mid-2026/*

# 2-(3). A2A Protocol 필요성

에이전트 간 협업을 연결하고 확장하는 통신의 표준으로...

## Standardized Protocol

- 에이전트 간 통신 위한 메시지 구조를 개방형 표준으로 통일
  : 전송 계층은 HTTP(S) 위에서 3가지 바인딩(JSON-RPC 2.0, gRPC, HTTP+REST)과 스트리밍(SSE)로 구분
- Custom API나 복잡한 파라미터 맵핑의 부담을 줄여, 서로 다른 프레임워크로 만든 에이전트가 공통 규격으로 연동
- 표준 에러 객체와 상태 정의로 오류 처리의 공통 기반 제공
  (단, Retry, 복원 로직은 구현 계층의 몫)

## Scaled and Structured Collaboration

- 에이전트 간 협업을 표준화된 구조 안에서 엔터프라이즈급 확장 목표로 설계
- 멀티 에이전트 구조의 유연성 극대화 :
  - 오케스트레이터가 전체 Workflow와 Context를 조율하므로, 특정 Server 에이전트를 교체하거나 삭제하는 것이 매우 용이
  - 전체 시스템의 재구성 없이도 유연하게 기능 확장 가능
  - 수시간 이상 소요되는 장기 작업의 협업 흐름도 명확하게 추적 가능
    (전체 추적은 W3C Trace Context + OpenTelemetry 적용 전제)

## Cross-Platform Interoperability

- 서로 다른 플랫폼과 기술 환경에서도 원활한 에이전트 간 통신 보장
- 내부 프롬프트나 상태를 노출하지 않는 블랙박스 형태 연결로 데이터 보안 유지
- 예시 흐름 :
  - 오케스트레이터 : 사용자 입력 분석 및 하위 작업 위임
  - Server 에이전트 1 : 대규모 데이터 및 논문 검색 처리
  - Server 에이전트 2 : 검색 결과 바탕으로 전문가 수준의 응답 초안 작성
  - 오케스트레이터 : 각 결과를 통합/중재하여 최종 응답을 사용자에게 전달

## Modular, Extensible AI Ecosystem

- Linux Foundation의 벤더 중립적 오픈소스 거버넌스 통해 특정 단일 플랫폼에 종속되지 않음
- 다양한 조직과 개발자가 참여할 수 있는 개방형 생태계 기반 통신 표준 지향
- 이를 통해 개별 기업은 고유한 강점을 가진 에이전트를 개발하고,
  향후 에이전트 마켓플레이스, 레지스트리가 성숙하면 타서비스와 연결하여
  연합형 에이전트 에코 구축 가능 (현재 관련 인프라는 표준화 진행중)

# 2. AI Agent 핵심 요소

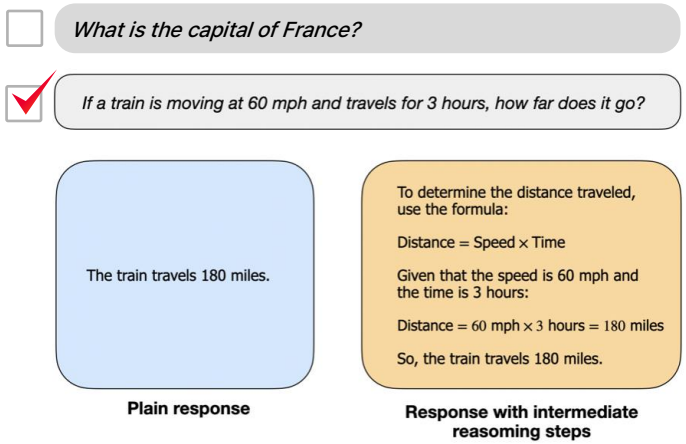
SUMMARY

SUMMARY

# 1. LLM Reasoning

단순 답변 생성이 아닌 목표 달성 위해 "판단하고 계획하는 **사고의 과정, Process of Planning**"으로 봐야 함

- Reasoning : 주어진 정보로부터 논리적으로 결론을 도출하거나, 다음 행동을 결정하는 사고 과정
- AI Agent가 "왜 이 행동을 해야 하는가"를 판단하게 하는 과정




A regular LLM may only provide a short answer (as shown on the left), whereas reasoning models typically include intermediate steps that reveal part of the thought process. (Note that many LLMs who have not been specifically developed for reasoning tasks can also provide intermediate reasoning steps in their answers.)

AI Agent의 Reasoning 역할 (현재 기준) :

| Agent 역할 | 내용 |
|---|---|
| 행동 판단 | • 도구를 쓸 것인지, 응답을 생성할 것인지를 결정 |
| 작업 계획 | • 단계를 나누고 어떤 순서로 처리할지 구상 |
| 결과 검토 및 수정 | • 행동 결과를 평가하고 다시 사고(재추론)하여 행동 수정 |
| 목표 일치 판단 | • 현재 정보가 사용자의 목표를 충족하는지 확인 |

➔ 단순한 응답 생성기를 넘어,
상황/행동을 판단하고 실행을 조정하는 기능 수행

※ Remark
- System 1 : 빠르고 본능적인 판단. 기존 LLM의 즉각적인 패턴 매칭 응답 (Fast Thinking)
- System 2 : 느리고 논리적인 연산. AI Agent가 수행하는 추론(Reasoning) 및 계획(Planning) 과정 (Slow Thinking)
- 과거(2023) LLM은 즉각적 패턴 매칭(System 1) 위주였으나, 현재는 강화학습 기반의 내부 사고 과정(System 2)이 통합된 추론 모델로 확장됨

*Source : https://www.linkedin.com/pulse/understanding-reasoning-llms-sebastian-raschka-phd-1tshc/*

# 2. 전략적 도구



## LLM Reasoning 위한 보완 방식

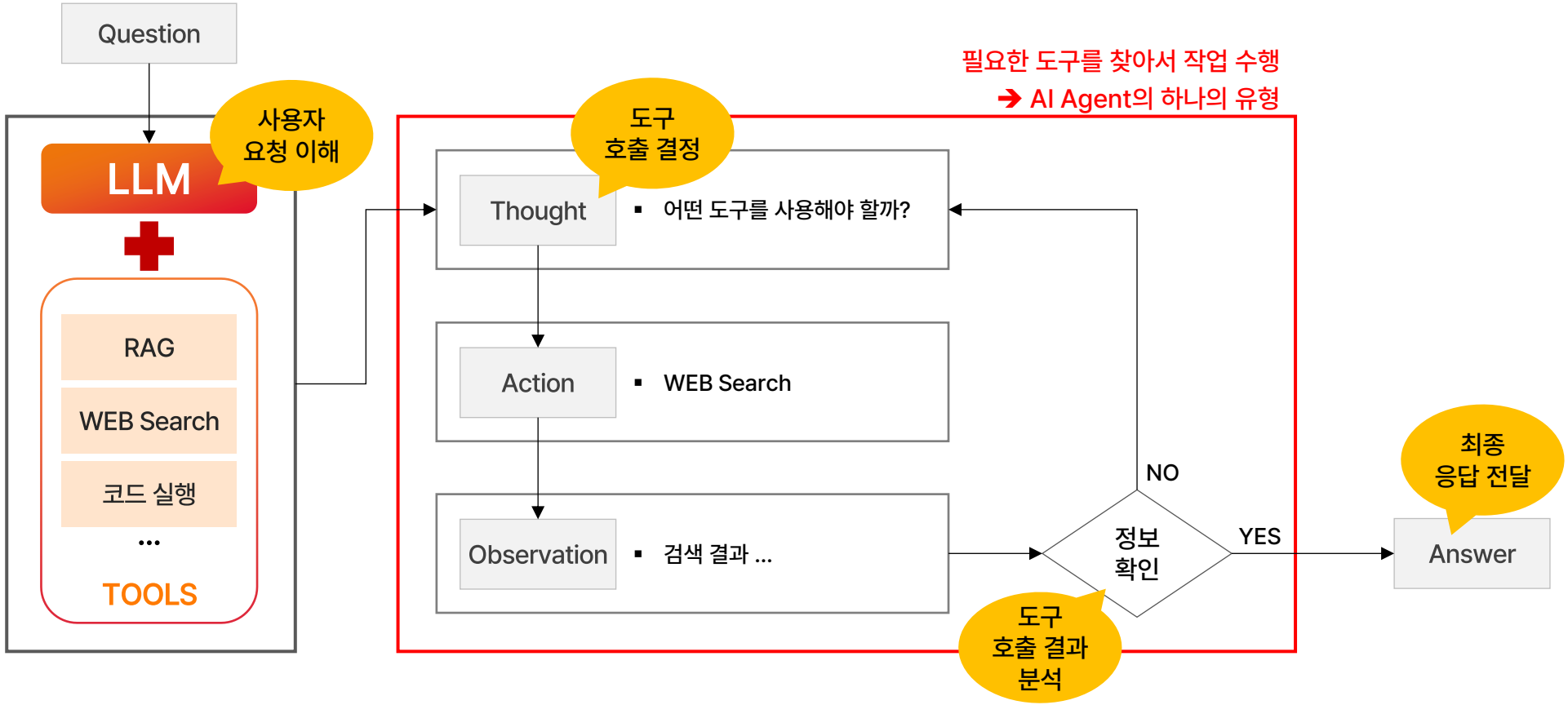
| 구분 | 보완 방식 | 핵심 개념 | 역할/목적 |
|---|---|---|---|
| Model | Inference-time Scaling | 추론 시간에 탐색 연산량을 동적으로 투입/조절 | 복잡한 문제 해결 위한 심층 추론 고려 |
| | Reasoning Tuning | 전문가 추론 궤적 데이터(SFT)와 보상 모델(RL) 기반 튜닝 | 모델 자체의 논리적 사고 및 역추적 내재화 |
| | Reasoning Distillation | 대형 추론 모델의 사고 토큰/경로를 소형 모델 형태로 증류 | 경량 모델의 추론 효율성 및 비용 최적화 |
| | Scratchpad | 중간 계산 및 추론 메모를 컨텍스트 내에 기록하도록 유도 | 다단계 추론의 투명성과 디버깅 확보 |
| Context | Chain-of-Thought | 중간 사고 과정을 단계별로 서술하도록 유도하는 프롬프트 | 단순 응답을 넘어 추론 단계 명시화 |
| | Self-Consistency Decoding | 여러 추론 경로를 병렬 실행 후 다수결 집계 | 추론의 일관성 및 정답 신뢰도 향상 |
| Tool | Tool Augmentation | 계산기, 웹검색, 코드 실행기 등의 외부 도구와 연동 | 지식 최신화 및 결정론적 연산 한계 극복 |
| | Agentic RAG | 검색 쿼리 재작성, 검색 결과 평가 및 반복 검색 결합 | 정보 왜곡 방지 및 고품질 근거 기반 추론 |
| Harness | Feedback Loop | 실행기의 피드백을 받아 계획 재수립 | 실제 환경 기반의 자가 수정(Self-Correction) |
| | Cross-Validation | 생성자와 평가자의 교차 확인 | 단일 모델 편향 및 환각 제어 |

SUMMARY

# 1. Tool Calling

모델이 적절한 도구를 반복적으로 호출하고 결과를 받아 문제 해결

- 도구 호출을 하면 모델이 하나 이상의 도구가 호출되어야 하는 시기를 감지하고 해당 내용을 전달할 수 있어야 함
- 목적에 맞는 유효하고 유용한 도구 호출(Tool Calling) 결정 및 실행

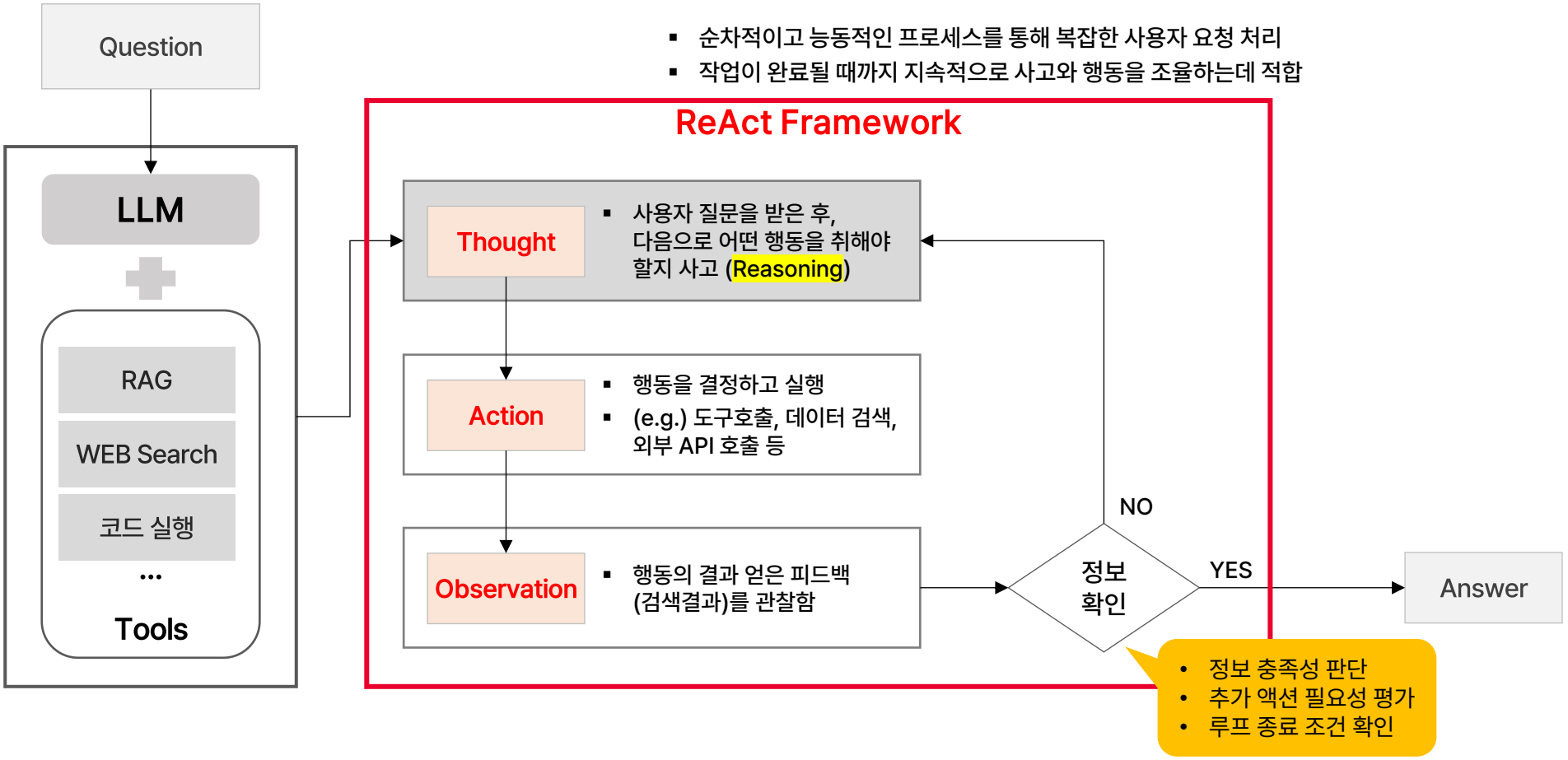



*Reference : https://python.langchain.com/v0.1/docs/modules/agents/agent_types/tool_calling/*

SUMMARY

# 2. ReAct Framework

Workflow : ReAct = **Reasoning** + Acting

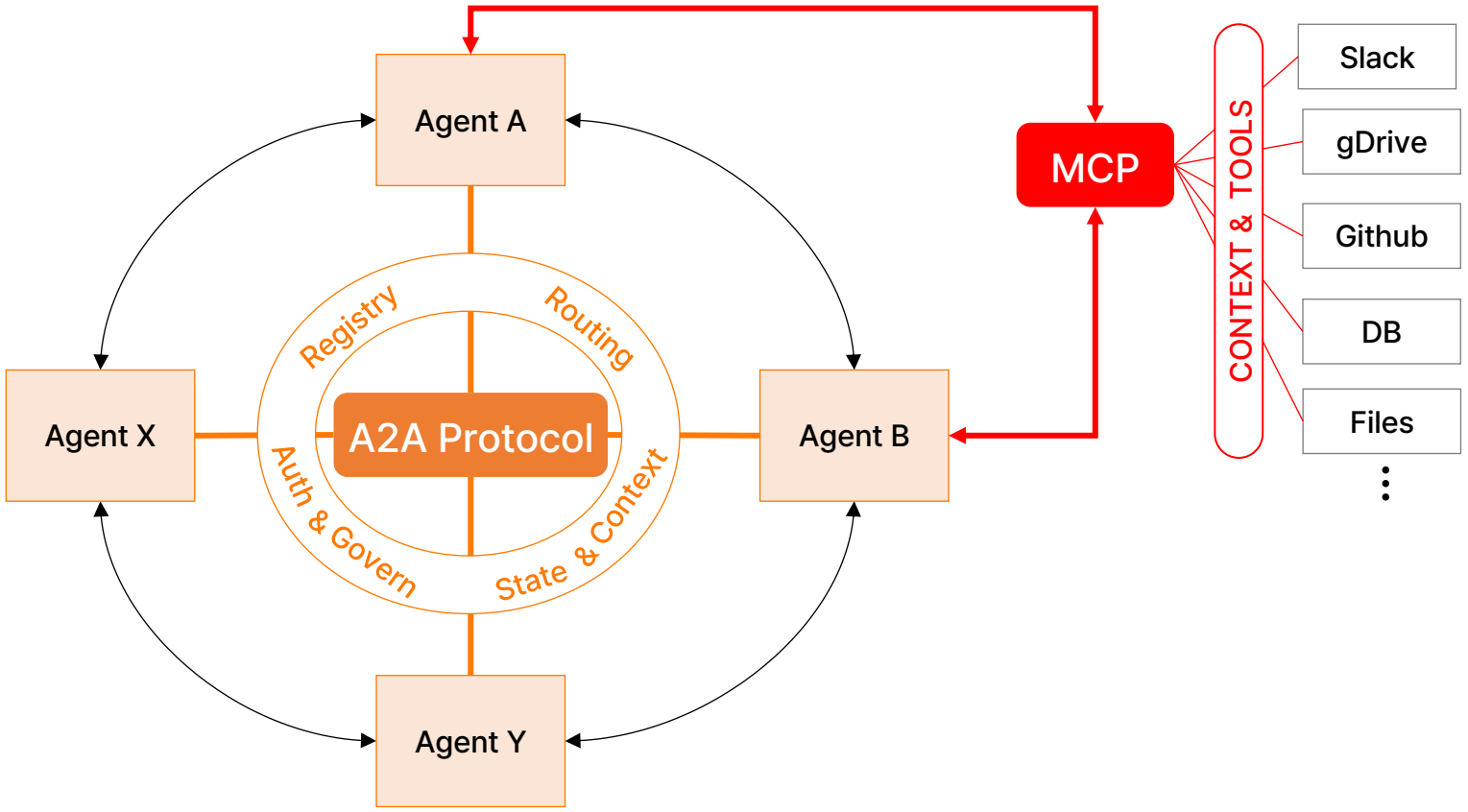



- 순차적이고 능동적인 프로세스를 통해 복잡한 사용자 요청 처리
- 작업이 완료될 때까지 지속적으로 사고와 행동을 조율하는데 적합

SUMMARY

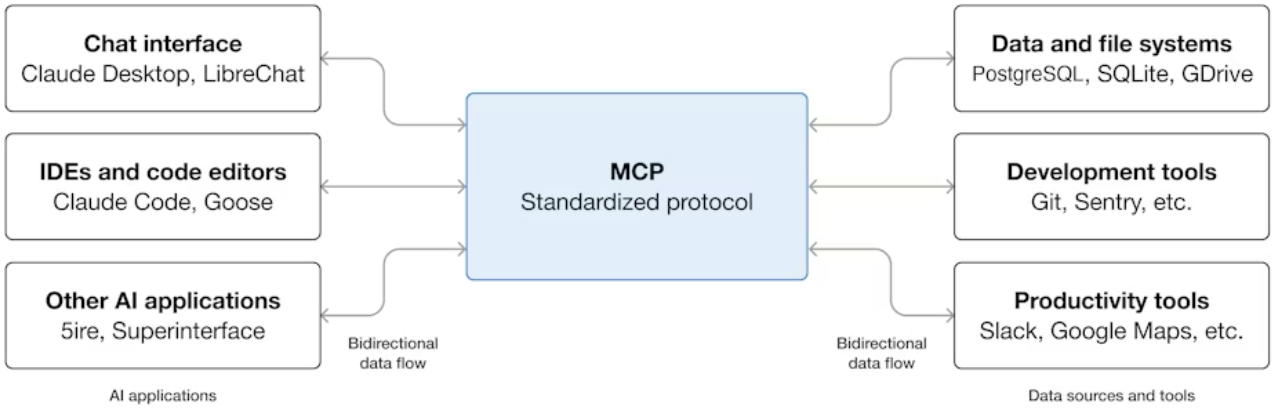
# 3. Agent Protocol

2026.H1



# 1. MCP

SUMMARY




## 목적 :

- 에이전트와 다양한 데이터베이스를 쉽게 연결 할 수 있는 오픈소스 프로토콜
- 이를 통해 AI 시스템이 외부 정보에 직접 접근하여 성능과 활용도를 높일 수 있음

## 기능 및 특징 :

- 표준화된 인터페이스 : 다양한 AI시스템과 데이터 소스 간의 상호 운용성을 높이기 위해 공통의 규칙과 형식 정의
- 통합의 용이성 : 각 데이터 소스에 맞춰 별도의 인터페이스 코드를 작성할 필요 없이, 통합된 프로토콜로 여러 데이터 소스와 연결 할 수 있음

Key Milestones :

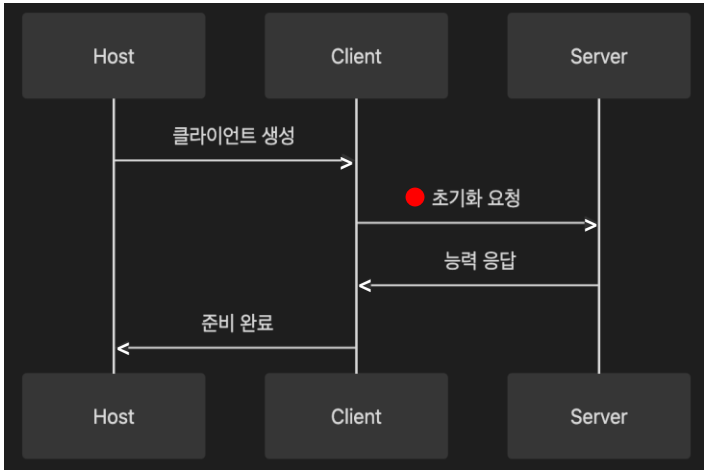
- Phrase 1. 초기 런칭 (2024.11)
  : Spec 공개, Python/TypeScript SDK 배포
- Phrase 2. 생태계 확산 (2025.1H)
  : Java, Kotlin, Go 등 주요 언어 SDK 지원 확대 및 Community Server 활성화
- Phrase 3. 표준 정착 (2025.2H)
  : 주요 IDE(Cursor, VS Code) 및 Agent Framework (LangChain, LlamaIndex) 네이티브 통합 완료

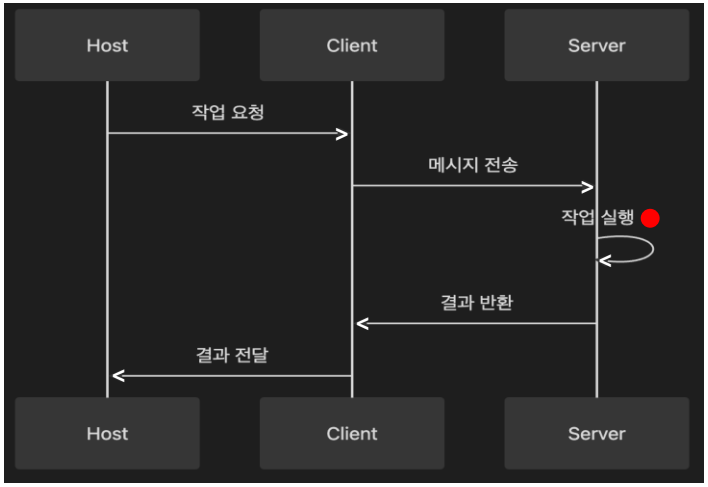
*Source : https://modelcontextprotocol.io/docs/getting-started/intro, https://github.com/modelcontextprotocol*

SUMMARY

# 1-(2). Host-Client-Server

명확한 역할과 책임을 가지며, 이를 통해 유연하고 확장 가능한 시스템 구현 가능




## MCP Host

- 회사의 팀장처럼, 여러 부서(서버)와 연결을 조율하는 중앙 통제실 역할
- 사용자와 직접 상호 작용하는 어플리케이션
- 주요 역할 :
  - LLM과의 통합 및 전체 데이터 흐름 조율
  - 다양한 서버의 응답을 취합 및 정제하여 AI모델에 전달

## MCP Client

- 팀장(Host)과 각 부서(Server) 사이에서 업무 요청/회신을 담당하는 전담 인력
- 호스트 내부에서 동작
- 각 MCP Server와 1:1로 연결되는 중개자 역할을 하는 컴포넌트
- 주요 역할 :
  - 서버와의 독립적인 연결 및 세션 관리
  - 프로토콜 협상, 기능 교환, 요청/응답 메시지 라우팅

## MCP Server

- 회사의 각 부서처럼, 특정 업무(데이터 제공, 도구 실행 등)를 담당
- AI 모델이 사용할 수 있는 3대 핵심 요소(Resources, Tools, Prompts) 제공
- 주요 역할 :
  - MCP 프로토콜 통해 클라이언트의 요청을 받아 정보 제공 또는 작업 실행
  - 로컬 데이터(파일, DB)나 원격 서비스(API, 클라우드 등)와 연결

※ Remark
- Resources : File, API, DB 등 모델이 읽을 수 있는 데이터 컨텍스트
- Tools : 모델이 외부 시스템에 개입하여 실행할 수 있는 액션 (함수 호출)
- Prompts : 사용자가 쉽게 작업을 시작할 수 있도록 정의된 프롬프트 템플릿

*Source : https://wikidocs.net/268794, https://modelcontextprotocol.io/specification/2025-06-18*

SUMMARY

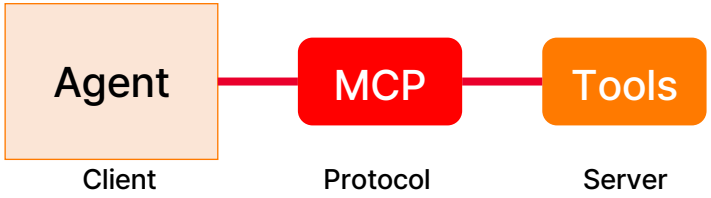
# 1-(5). Tool Calling vs MCP

서비스 목적과 요구사항에 맞춰 강한 결합(Native)과 느슨한 결합(MCP) 중 적절한 아키텍처를 선택해야 함

## Native Tool Calling




## MCP

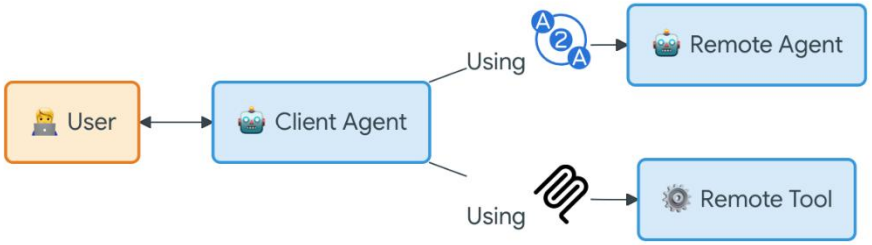


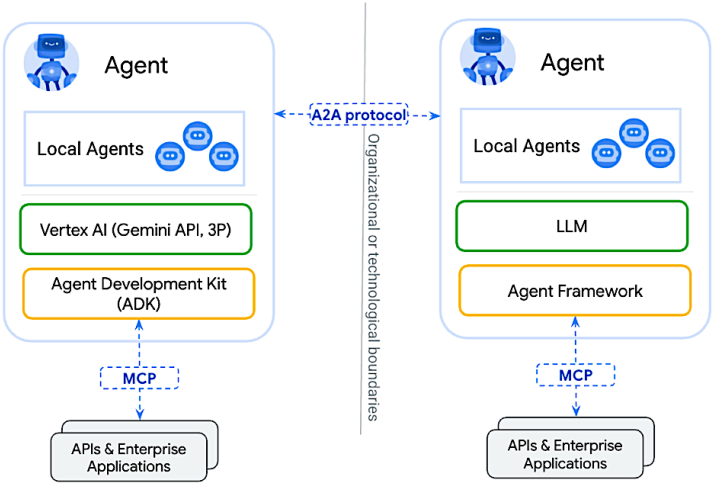

| | Native Tool Calling | MCP |
|---|---|---|
| 아키텍처 성격 | • 강한 결합 (Tightly Coupled)<br>• 에이전트 어플리케이션 내부에 도구(코드)를 직접 구현 | • 느슨한 결합 (Loosely Coupled)<br>• 클라이언트(Agent)와 서버(Tools)를 분리하여 프로토콜로 통신 |
| 도구 정의 및 관리 | • 개발자가 명시적으로 스키마를 정의<br>• LLM 프롬프트에 직접 반영 (+ 유지보수) | • MCP Server가 동적으로 도구 목록을 제공<br>• 에이전트는 도구 목록을 탐색 |
| 장점 | • 세밀한 제어 : 실행 과정과 에러 핸들링을 코드 레벨에서 완벽히 통제 가능<br>• Low Latency : 내부 함수를 직접 호출하므로 통신 오버헤드 최소화 | • 높은 확장성 : 코드 수정 없이 서버 연결만으로 수많은 외부 도구를 일괄적으로 확장 가능<br>• 재사용성 : 하나의 MCP Server를 여러 다른 에이전트가 공유 가능 |
| Best For | • 목적형 에이전트<br>• 사내 DB/API 연동 등 보안 로직 적용이 고려되어야 하는 경우 | • 범용 에이전트<br>• SaaS, Local File 등 다양한 외부 생태계와의 통합이 빈번한 경우 |

# 2. A2A Protocol



Enabling communication and collaboration between AI agents across different platforms and services




## 개요 :

- 에이전트들이 상호 운용할 수 있도록 돕는 오픈소스 프로토콜
  (현재 리눅스 재단 산하 프로젝트로 150개 이상 글로벌 기업 참여)
- MCP가 외부 도구와의 연결을 담당한다면,
  **A2A는 에이전트 간 협업과 통신을 전담**
- 에이전트들은 서로의 기능을 활용하여 복잡한 비즈니스 프로세스를 자동화하고, 개입을 최소화 할 수 있음

## What A2A is Not :

- Not an Agent Development Kit. 에이전트 간 통신 계층 역할 담당
- Not a Sub-agent or Tool-call Protocol
- Not a replacement for MCP (두 프로토콜은 상호 보완적 관계)
- Not an interactive messaging app
  : 사용자용 메시징 앱이 아니라,
  자율 에이전트 위한 Machine-to-Machine 프로토콜

*Source : https://a2a-protocol.org/latest/#how-a2a-works-with-mcp*

AI Campus 생성형AI

# 3. AI Agentic Workflow

- AI Agentic Workflow란?
- Design Framework : Goal → Plan → Execute → Reflect
- 설계 시 유의사항
- Applied Agentic Workflows – RAG, Analytics, Coding

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

# Workflow

## Traditional Workflow

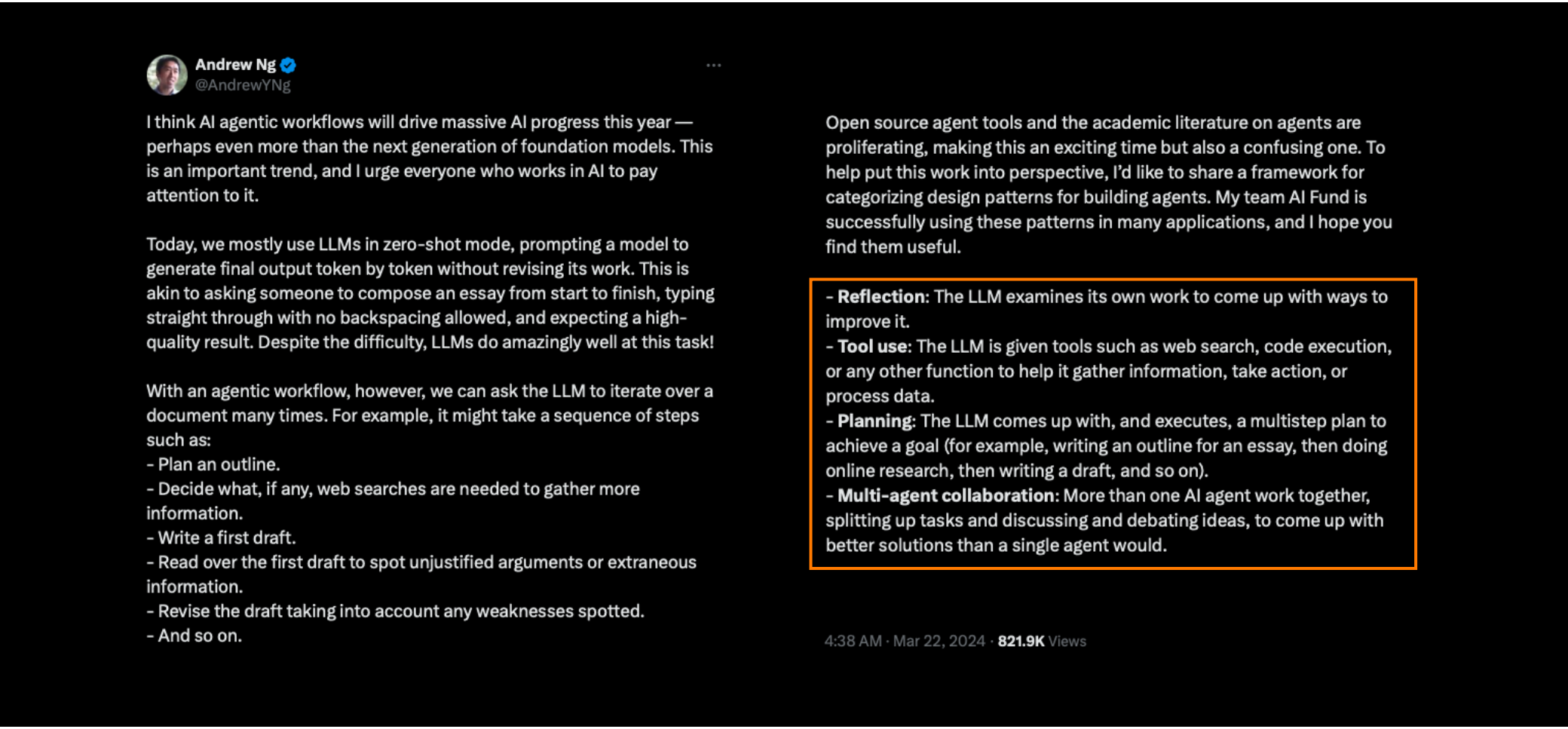
- A predefined sequence of processes (Linear Pipeline)
- Input → Process A → Process B → ... → Output
- 정해진 길을 따라가는 단방향 흐름 (Zero-shot)

## AI Agentic Workflow

- An iterative process of self-correction
- Goal ⟳ (Plan ⇌ Execute ⇌ Reflect) → Output
- **결과를 평가하고 스스로 개선하는** 순환형 흐름 (Iterative Loop)

# 1. AI Agentic Workflow

2024



Andrew Ng
@AndrewYNg

I think AI agentic workflows will drive massive AI progress this year — perhaps even more than the next generation of foundation models. This is an important trend, and I urge everyone who works in AI to pay attention to it.

Today, we mostly use LLMs in zero-shot mode, prompting a model to generate final output token by token without revising its work. This is akin to asking someone to compose an essay from start to finish, typing straight through with no backspacing allowed, and expecting a high-quality result. Despite the difficulty, LLMs do amazingly well at this task!

With an agentic workflow, however, we can ask the LLM to iterate over a document many times. For example, it might take a sequence of steps such as:
- Plan an outline.
- Decide what, if any, web searches are needed to gather more information.
- Write a first draft.
- Read over the first draft to spot unjustified arguments or extraneous information.
- Revise the draft taking into account any weaknesses spotted.
- And so on.

Open source agent tools and the academic literature on agents are proliferating, making this an exciting time but also a confusing one. To help put this work into perspective, I'd like to share a framework for categorizing design patterns for building agents. My team AI Fund is successfully using these patterns in many applications, and I hope you find them useful.

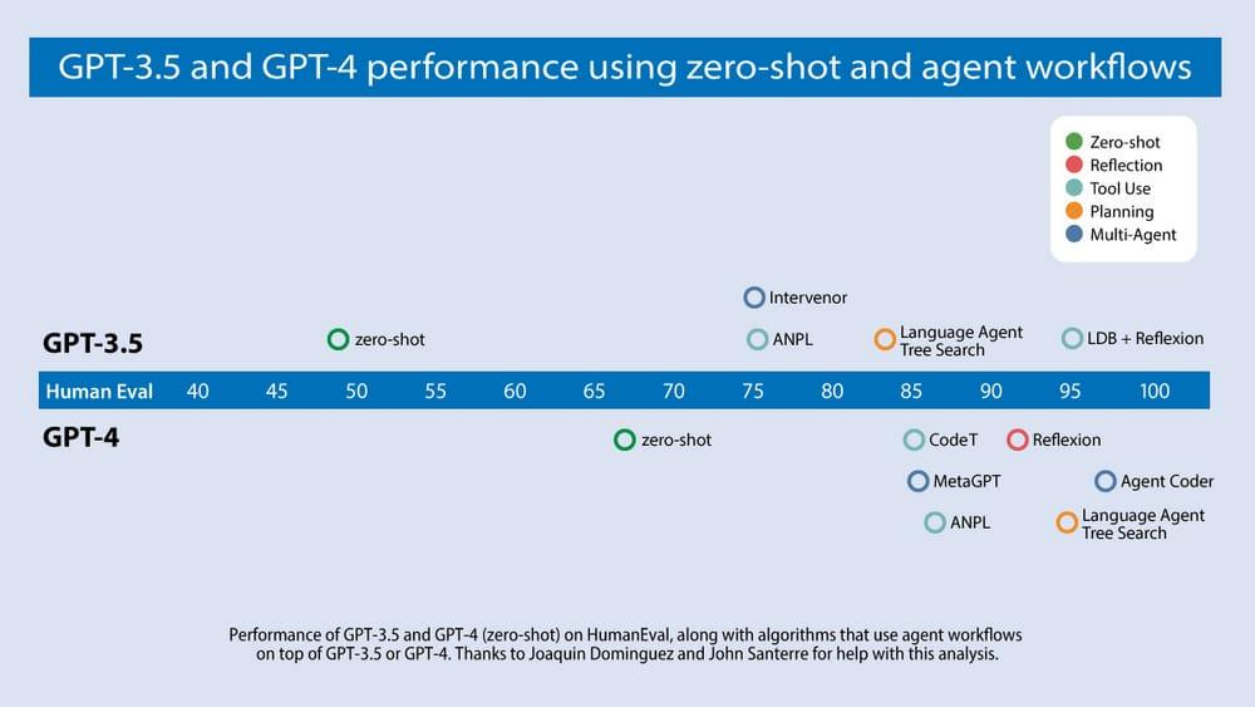
- **Reflection**: The LLM examines its own work to come up with ways to improve it.
- **Tool use**: The LLM is given tools such as web search, code execution, or any other function to help it gather information, take action, or process data.
- **Planning**: The LLM comes up with, and executes, a multistep plan to achieve a goal (for example, writing an outline for an essay, then doing online research, then writing a draft, and so on).
- **Multi-agent collaboration**: More than one AI agent work together, splitting up tasks and discussing and debating ideas, to come up with better solutions than a single agent would.

4:38 AM · Mar 22, 2024 · **821.9K** Views

*Source: https://x.com/AndrewYNg/status/1770897666702233815?lang=en*

# 1. AI Agentic Workflow

2024




Performance of GPT-3.5 and GPT-4 (zero-shot) on HumanEval, along with algorithms that use agent workflows on top of GPT-3.5 or GPT-4. Thanks to Joaquin Dominguez and John Santerre for help with this analysis.

GPT-3.5 (zero shot) was 48.1% correct.
GPT-4 (zero shot) does better at 67.0%.

However, the improvement from GPT-3.5 to GPT-4 is dwarfed by incorporating an iterative agent workflow. Indeed, wrapped in an agent loop, GPT-3.5 achieves up to 95.1%.

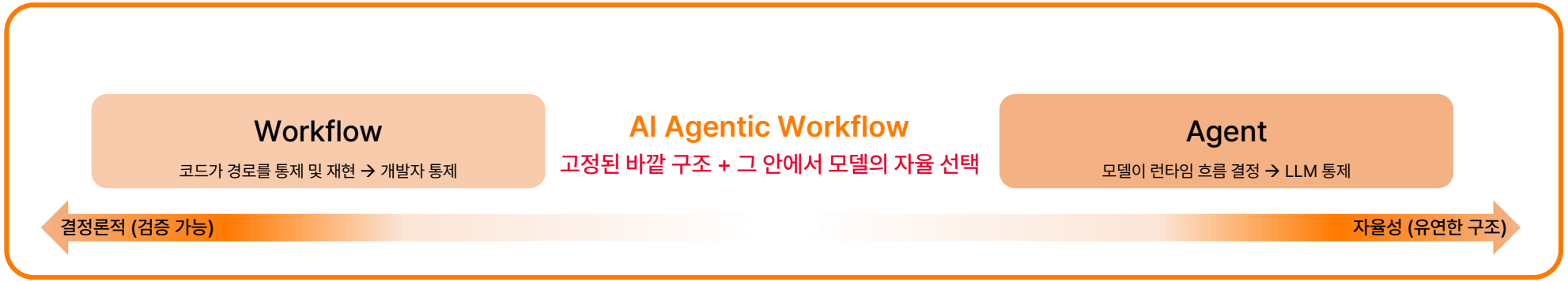
**강력한 모델 + 탄탄한 전략 ➔ 우수한 성능**

계획 수립 전략: Planning
⊕ 자기 피드백 전략: Reflection
⊕ 다중 에이전트 협업: Collaboration

*Source: https://www.deeplearning.ai/the-batch/issue-241/*

# 1. AI Agentic Workflow

2026 – 흐름을 누가 통제하는가?



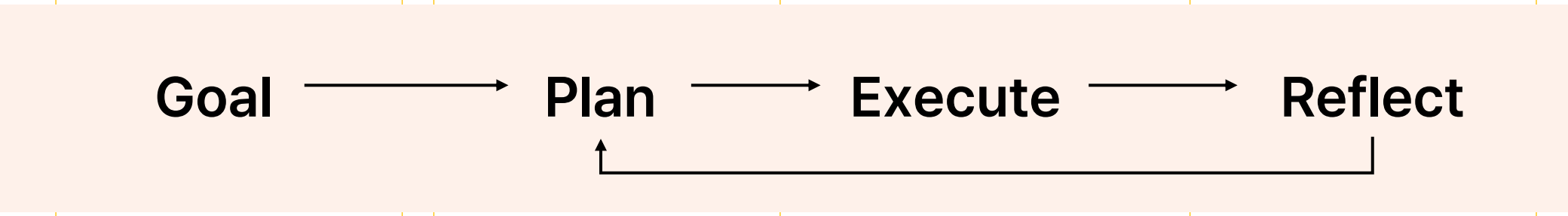

- Agentic은 정도의 문제 : Agent냐 아니냐 보다 "얼마나 자율적"인가의 관점으로 해석
- 가장 단순한 것부터 : 높은 신뢰성을 요구하는 Production에서는 Agent 보다 Workflow 기반으로 개발 비중 높음 ← 검증 가능, 비용 관리
- Agent ≠ Loop : Self-Correction Loop이라 하더라도, 구조가 코드로 고정되어 있으면 Workflow임

# 2. Design Framework

Key Point

Agent는 사고 구조를 설계하는 것에서 시작합니다 !!!

**Strategic Layer** | **Operational Layer**




- 무슨 목적인가?
- 무엇을 목적 달성으로 볼 것인가? (Success Criteria)

- Task Decomposition
- State Schema Definition
- Control Strategy : Linear, Loop, Branch, Retry
- Collaboration Structure : Centralized, Distributed, Hybrid

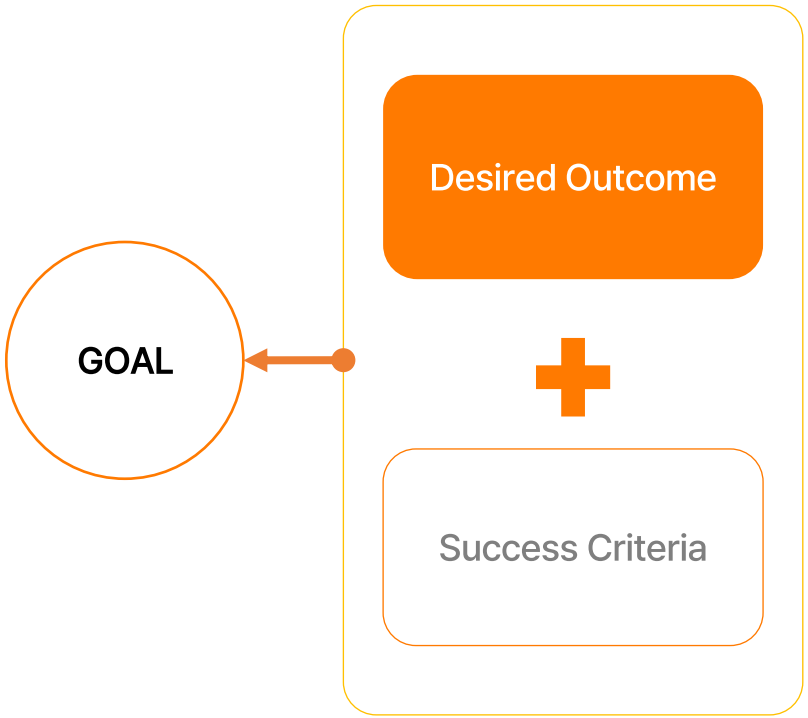
- Tool Invocation
- State Update
- Action Execution
- Memory

- Evaluation Mechanism : Reflection, Tool-based Validation, External Judge, etc.

# 1. Goal – Desired Outcome

Goal은 무엇을 할지가 아니라, **'무엇이 달성되면 끝인지'를 업무 관점에서 정의**하는 단계임

- Desired Outcome : 산출물(Output)이 아니라 업무 결과(Outcome) 중심으로 정의
- Success Criteria : 좋은 답변 같은 정성 표현이 아니라, 판정 가능한 기준으로 정의

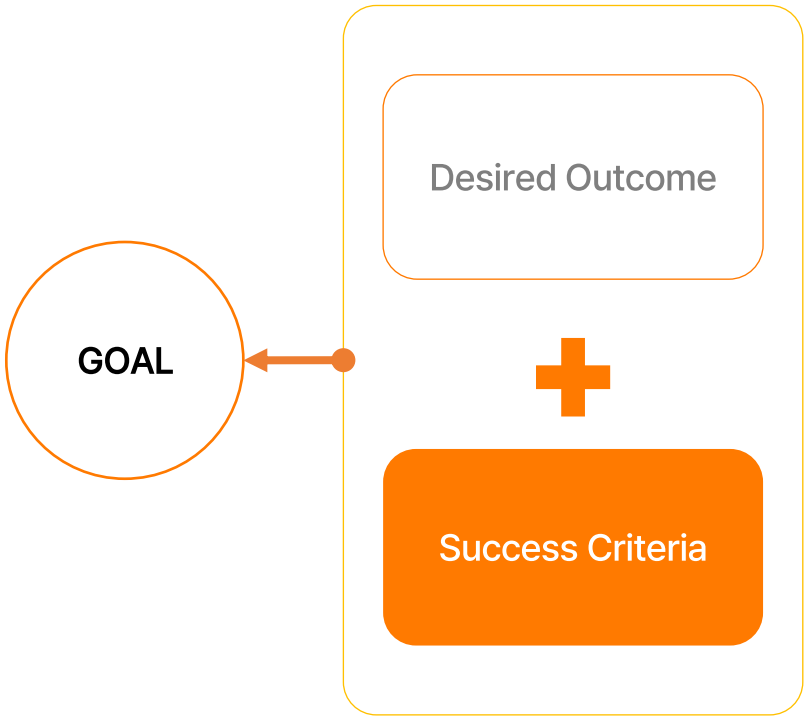



- 대상 : 무엇에 대한 결과인가?
- 변화 : 무엇이 달라져야 하는가?
- 결정/행동 : 결과가 어떤 의사결정/행동에 사용되는가?
- 범위 : 어디까지 포함/제외 하는가?

# 1. Goal – Success Criteria

Goal은 무엇을 할지가 아니라, **'무엇이 달성되면 끝인지'를 업무 관점에서 정의**하는 단계임

- Desired Outcome : 산출물(Output)이 아니라 업무 결과(Outcome) 중심으로 정의
- **Success Criteria : 좋은 답변 같은 정성 표현이 아니라, 판정 가능한 기준으로 정의**




결과 품질 기준 (Quality) :
- 근거성 : 주장마다 근거가 있는가? (출처)
- 충실성 : 그 출처가 실제로 그 주장을 뒷받침하는가?
- 완결성 : 필수 항목을 빠짐없이 포함했는가?
- 일관성 : 내부 모순이 없는가? – 내부 일관성

업무적 제약 조건 (Constraints) :
- Target Actor : 톤, 매너, 답변 길이 등
- Compliance : 내부정보 노출 금지, 법적 제한 등
- Scope Boundaries : 포함, 제외 범위
- Formatting : JSON, 템플릿, 표 등

서비스 기준 (Operational) :
- Cost : 호출 횟수, Latency 등
- Error Handling : 근거 없을 때 응답 규칙 등
- Consistency : 같은/유사 입력에 일관된 품질 수준 유지 – 재현 일관성
- Stability : 반복 실행 시 품질 변동 허용 범위

# [참고] Goal 명세

**Goal – 공개 약관/정책에 근거하여, 고객이 추가 문의 없이 스스로 다음 행동을 결정할 수 있는 답변을 제공한다.**

Desired Outcome :

- 대상 : 고객의 요금제, 약관, 해지/변경 관련 문의
- 변화 : 1회 답변으로 판단/실행 가능 → 동일/유사 건의 재문의 감소, 상담사 이관 감소
- 결정 : 고객이 유지/변경/해지 등 다음 행동 결정, 시스템은 상담사 이관 필요 여부 판정
- 범위 : 공개 약관/정책 기반 일반 문의 (단, 정산/분쟁/법적 판단 등 제외 : 상담사 이관 필요)

Success Criteria :

- 결과 품질 :
  - 근거성 : 정책 관련 주장마다 해당 약관 조항/출처 명시 – 목표 : 출처 명시율 100%
  - 충실성 : 인용 조항이 실제로 그 주장을 뒷받침 – 목표 : Faithfulness 판정 통과 ≥ 95%
  - 완결성 : 문의 유형별 필수 안내 누락 없음 – 목표 : 필수 항목 전달율 100%
  - 일관성 : 답변 내 상충 진술 0건
- 업무 제약 :
  - Target Actor : 일반 고객 대상, 정중한 존댓말, 답변 400자 이내, 전문용어 최소화
  - Compliance : 비공개 내부정책 노출 금지, 확정적 법적 판단 표현 금지, 타 고객정보 노출 금지,
  - Boundaries : 범위 외 요청에 대하여 이관 안내
  - Formatting : [요약 → 근거(조항 인용) → 다음 절차]의 고정 3단 구성
- 서비스 기준 :
  - Cost : 답변 당 LLM 호출 ≤ 3회, P95 ≤ 3초 **(※ P95 : 질문을 던지면 95%의 상황에서 10초 이내에 답변을 마친다)**
  - Error : 근거 미검색 시 임의 생성 금지 → "확인 후 안내" 또는 상담사 이관
  - Consistency : 동일/유사 문의에 대한 재현 품질 편차 허용치 명시 (e.g., 판정 점수 ± 0.5 이내)
  - Stability : 동일 입력 10회 반복 시 필수 항목 누락, 상충 0 유지

# 2. Plan – Task Decomposition

일을 나누는 단계가 아니라, Goal을 달성하기 위한 실행 단위를 설계하는 단계임

- 분해된 Subtask는 실행 단위를 설계할 때 작업 중심이 아니라
- Outcome 중심으로 분해 가능해야 하고, Success Criteria를 실행 가능한 단위로 연결되어야 함

### Task Decomposition 고려사항

Outcome Alignment :
- 이 단계가 Outcome에 직접 기여하는가?
- 이 단계가 없다면 Goal 달성이 어려운가?

Dependency Structure : Control Strategy
- 병렬 가능한가?
- 반드시 순차적이어야 하는가?
- 실패 시 어느 단계로 돌아가는가?

Granularity : LOOP 전략
- 각 Subtask가 단일 책임을 가지는 수준
- 너무 크면, Reflection이 의미 없음
- 너무 잘게 나뉘면, 구조 복잡성 증가

[예시]

- Goal : 공개 약관/정책에 근거하여 고객이 추가 문의 없이 다음 행동 결정

- Task Decomposition :
  1. 질문 유형 분류 — Boundaries
  2. 관련 정책 검색 — 근거성
  3. 정책-질문 매핑 검증 — 충실성
  4. 근거 포함 답변 생성 — 완결성, Formatting, Target Actor
  5. 내부 정책 위반 여부 체크 — Compliance

**Task Decomposition은 Plan의 뼈대를 구성하는 단계임**

# 2. Plan – State Schema

Key Point

Subtask 간 데이터를 전달하고, Goal 달성 여부를 추적하는 Single Source of Truth 정의 단계

- 이전 단계의 Output이 다음 단계의 Input으로 이어지도록 설계
- Eval & Control : Reflect 및 Loop 제어에 필요한 판정 기준 데이터 반영

**State Schema 설계 고려사항**

Data Lifecycle :
- 특정 단계에서만 사용되는 임시 데이터인가?
- 전체 파이프라인이 공유하는 전역 상태인가?

Traceability :
- Subtask 실패 시 이전 시점(checkpointer)로 롤백 할 수 있는가?
- 원인을 파악해야 하는가? 할 수 있는가?

Granularity :
- 원본 데이터 전체를 전달하는가?
- 필요한 Key로 Scoping 가능한가?

Control Flags :
- Loop 제어 위한 Flag가 필요한가? (retry_count, is_valid, etc.)

[Vanilla Definition]

- Input : User Query, 대화 이력
- Working : Sub-Agent별 input/output, Control Flag
- Eval : Rubric별 평가 점수 (+커멘트)
- Loop : 루프 카운터, 이관/종료 트리거

# [참고] State Schema (1/2)

Single Source Truth 기반 정의

```
{
  /* 1. Input Context */
  "user_query": "문자열 (고객 인입 질문)",
  "user_profile": "Dict (회원 등급, 가입 상품 등)",

  /* 2. Intermediate State (Task Decomposition 결과물) */
  "query_category": "Enum (요금제 | 약관 | 해지/변경 | 범위외)",          // 1. 질문 유형 분류
  "retrieved_docs": [                                                  // 2. 관련 정책 검색
      {
          "doc_title": "String (약관/정책명)",
          "section": "String (조항 번호/위치)",
          "content": "String (검색된 청크 본문)",
          "effective_date": "YYYY-MM-DD (시행일자/최신성 검증용)",
          "is_latest": "Boolean (최신 버전 여부 플래그)"
      }
  ],
  "mapping_validation": {                                              // 3. 매핑 검증
          "is_faithful": "Boolean",
          "faithfulness_score": "Float (>= 0.95)"
  },
  "draft_response": "문자열 (3단 구성 초안 답변)",                      // 4. 답변 생성

  /* 3. Evaluation & Control State */
  "compliance_check": {                                                // 5. 정책 위반 체크
          "passed": "Boolean",
          "violation_reason": "문자열 (비공개 정책/법적 판단 포함 시)"
  },
  "retry_count": "Integer (Max: 2회)",                                 // 서비스 기준 (Cost 통제)
  "is_escalate": "Boolean (상담사 이관 여부)"                          // Error Handling
}
```

# [참고] State Schema (2/2)

Agent Scoping 기반 – 에이전트 역할을 분명히 나누어 컨텍스트 오염 사전 방지

```
{
  /* 1. Global Context (모든 Sub-agent가 참조 가능한 공통 상태) */
  "user_query": "문자열 (고객 인입 질문)",
  "user_profile": "Dict (회원 등급, 가입 상품 등)",

  /* 2. Sub-agent Private / Working Scopes (각 Agent 담당 영역) */
  "router_agent": {"category": "Enum (요금제 | 약관 | 해지/변경 | 범위외)"},      // 1. 질문 유형 분류 Agent
  "retriever_agent": {                                                          // 2. 관련 정책 검색 Agent
        "retrieved_docs": [{
              "doc_title": "String (약관/정책명)",
              "section": "String (조항 번호/위치)",
              "content": "String (검색된 청크 본문)",
              "effective_date": "YYYY-MM-DD (시행일자)",
              "is_latest": "Boolean (최신 버전 여부)"
     }]
  },
  "writer_agent": {                                                             // 3. 답변 생성 Agent
        "draft_response": "문자열 (3단 구성 초안 답변)"
  },
  "evaluator_agent": {                                                          // 4. 검증 및 평가 Agent (Reflector)
        "faithfulness": { "score": 0.95, "passed": true },                      // 매핑 충실성 검증
        "compliance": { "passed": true, "reason": null }                        // 정책 위반 체크
  },

  /* 3. Orchestrator Control State (워크플로우 제어) */
  "retry_count": "Integer (Max: 2회)",                                          // 비용/호출 횟수 제어
  "is_escalate": "Boolean (상담사 이관 여부)"                                   // Error Handling / 라우팅 플래그
}
```

```
# (e.g.) writer_agent를 호출할 때 필요한 데이터만 주입
writer_input = {
   "query": state["user_query"],
   "docs": state["retriever_agent"]["retrieved_docs"]
}
# 실행 후 writer_agent의 산출물만 해당 스코프에 기록
state["writer_agent"]["draft_response"] = writer_agent.run(writer_input)
```

# 2. Plan – Control Strategy

몇 번 반복할 것인가를 정하는 단계가 아니라 언제 진행하고, 언제 되돌아가고, 언제 멈출지를 설계하는 단계임

### Linear Flow :

- 가장 간단한 구조
- Subtask A → B → C → Output

- 의존성이 명확할 때 적합
- Reflection 의미 약함

### Conditional Branch :

- 모델의 판단이 아니라 설계자의 정책
- Branch :
  - Rule : 재현성이 중요하고, 조건을 명확하게 열거할 수 있는 경우
  - Judge : 열린 Task 이지만 검증이 필요한 경우

### Loop Strategy (Iterative Control) :

- 부분 개선을 가능하게 만드는 구조
- Plan → Execute → Reflect → (조건부 재진입)

- 어떤 조건에서 재진입하는가?
- 어떤 단계로 돌아가는가?
- 무엇이 개선 대상인가?

### Retry & Termination Policy :

- Retry는 품질 향상이 예상될 때에만 허용되어야 함
- 종료 조건이 없으면 Agent는 멈추지 않음

반드시 정의해야 하는 것 :
- Max iteration
- 실패 시 fallback 전략
- 근거 부족 시 응답 규칙
- 비용 초과 시 종료 정책

# [참고] Plan – Task & Control

**Goal – 공개 약관/정책에 근거하여, 고객이 추가 문의 없이 스스로 다음 행동을 결정할 수 있는 답변을 제공한다.**

Task Decomposition :

| # | Task | 처리 방식 | 실패 처리 | Success Criteria |
|---|---|---|---|---|
| 1 | 질문 유형 분류 | Rule | • 범위 외 질문 → 상담사 이관 (Rule-based) | Boundaries |
| 2 | 관련 약관/정책 조항 검색 | 검색 Tool | • 결과 부족 여부 판단 : (결과수 < N) or (score < threshold)<br>• 부족 시 max 2회 재검색<br>• 그래도 부족하면 임의 생성 금지 → "확인 후 안내" 메시지 출력 & 상담사 이관 | 근거성<br>Error |
| 3 | 조항 – 질문 매핑 검증 | LLM Judge | • 충실성 미달 시 맵핑 단계만 재실행 (max 1회) | 충실성 |
| 4 | 근거 포함 답변 생성 | LLM 생성 | • 3단 구조, 400자 위반 시 재생성 (max 1회) | 완결성 Formatting Target Actor |
| 5 | 내부 정책 위반 여부 체크 | Rubric | • 위반 시 답변 재생성 → 반복 실패 시 상담사 이관 | Compliance |

Cost :

- 정상 : LLM 생성 1회 (#4) + LLM 검증 1회 (#3) + 검색 1회 (#2) → 예산 이내
- 최대 : 생성 ≤ 2회, 검증 ≤ 2회, 검색 ≤ 3회 → 초과 시 중단 & 서비스 이관

**Task가 구조라면, Control Strategy는 의사결정 로직 (esp., 실패를 어떻게 처리할 것인가)**

# 2. Plan – Collaboration Structure

**책임과 의사결정 권한**을 설계하는 단계

## Chaining

- 고정 순차 : 미리 정한 순서대로 연결. A → B → C
- 사람이 미리 고정. 런타임 결정 없음
- 앞 단계의 산출물이 다음 단계로 넘어감.
- 복귀 없음 (일방향)

Pattern :

- Pipeline
- 순차 워크플로우

## Routing, Delegation

- 중앙에서 작업 배분/위임
- 관리자의 지시 + 맥락이 다음 단계로 전달
- 일이 끝나면 제어가 중앙으로 복귀

Pattern :

- Supervisor : 매 스텝 순차 라우팅 (조건부 엣지로 라우팅)
- Orchestrator : 실행 전 계획수립/분해 + 병렬
- Agent-as-Tool : LLM의 tool-calling으로 서브에이전트를 함수처럼 호출

## Hand-off

- 담당 에이전트가 스스로 다른 에이전트에게 직접 넘김
- 현재 담당 에이전트가 런타임에 다음 순서 결정
- 제어권 + 맥락이 다음 단계로 전달
- 제어권이 넘어가면, 넘긴 쪽은 빠짐 (돌아오는 구조 없음)

Pattern :

- Swarm

## Feedback Loop

- 답변 생성(Generator) 결과를 평가(Evaluator)가 Rubric으로 검증
- Evaluator가 품질 게이트 역할
- 미달 시 결과물과 피드백을 Generator에게 전달하여 재생성
- 무한루프 방지 위해 최대 반복 횟수, 종료 조건 필요

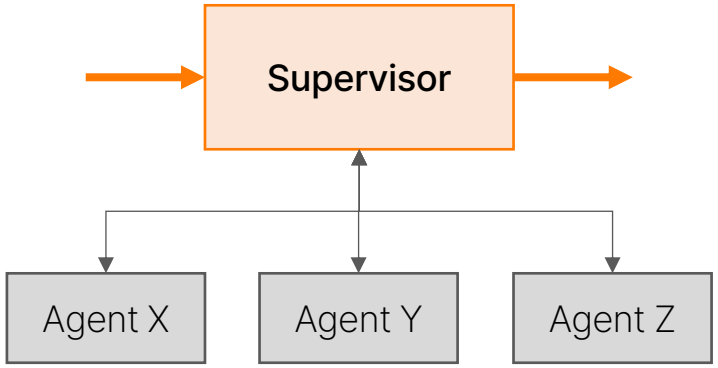
Pattern :

- Evaluator-Optimizer

# 2-(1). Supervisor

Key Point

Supervisor가 작업 분배, 품질 판단, 재시도, 종료 결정을 모두 통제하는 구조




구조적 특징 :

- 명확한 상위 의사결정 노드 – Supervisor (Decision Authority)
- Sub-Agent는 실행 단위 역할 담당
- Control Strategy와 강하게 결합
- State는 Global 공유되나, 에이전트별 Scoping 고려

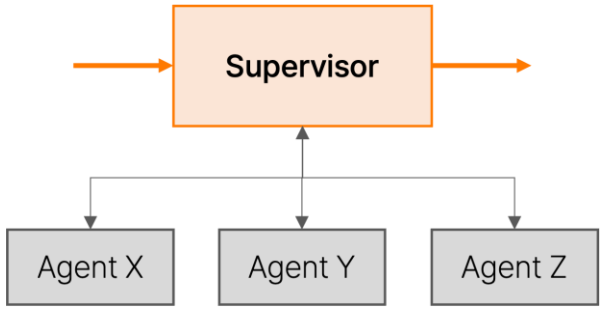
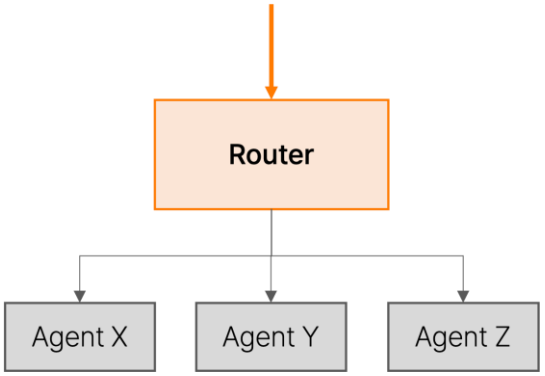
적합한 케이스 :

- Workflow가 복잡하고 Branch가 많은 경우
- 품질 검증이 핵심인 도메인
- 운영 정책 (종료 조건, fallback 등)을 통제해야 하는 서비스

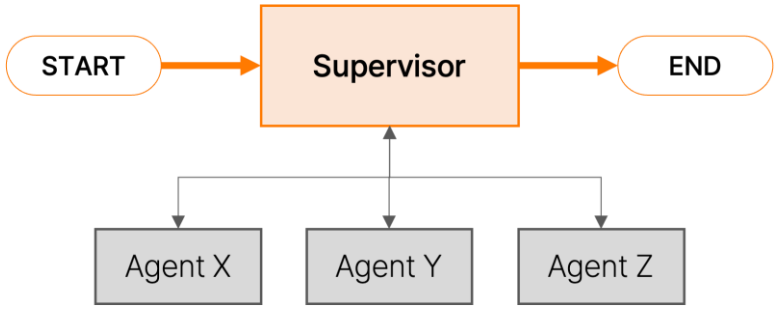
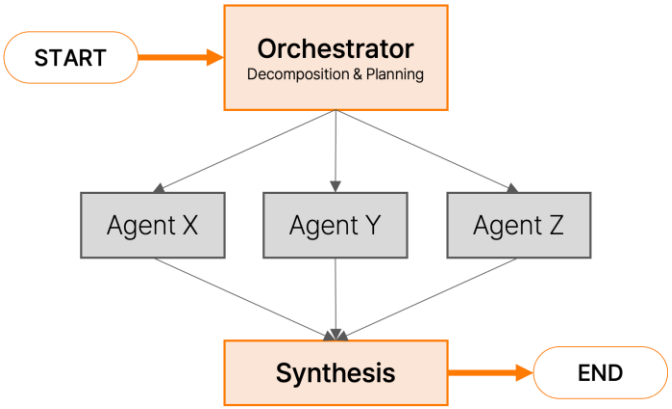
고려사항 :

- Success Criteria와 직접 연결 가능
- Supervisor 중심 : 제어 설계, 병목 현상 등
- 비용 : 매 스텝 Supervisor(LLM) 호출 누적

# [참고] Router

| 구분 | | Supervisor | Router |
|---|---|---|---|
| Pattern | |  |  |
| 역할 | | • 매 결과를 받아 다음에 누구를 부를지 반복 판단<br>• 종료 시점을 스스로 결정하고, 결과를 종합하는 **중앙 제어자** | • 입력을 분류해 적절한 한 곳으로 분기<br>• **분기하면 끝**. 되돌아오지 않고, 작업 완료를 **책임지지 않음** |
| 주요 특징 | 반복성 | • N회 (Loop) | • 1회 결정 |
| | 제어권 | • 실행 후 Supervisor로 복귀 | • 분기처리만 담당. 제어권 미보유 |
| | 종료 판단 | • "충분한가"를 스스로 판단 | • 없음 (분기 처리만 담당) |
| | State | • 작업, 대화 맥락 유지 | • 판단은 Stateless. 선택된 분기는 필요 시 State 반영 |
| | 최종 답변 | • 하위 에이전트 결과 취합 및 종합하여 전달 | • 없음 |

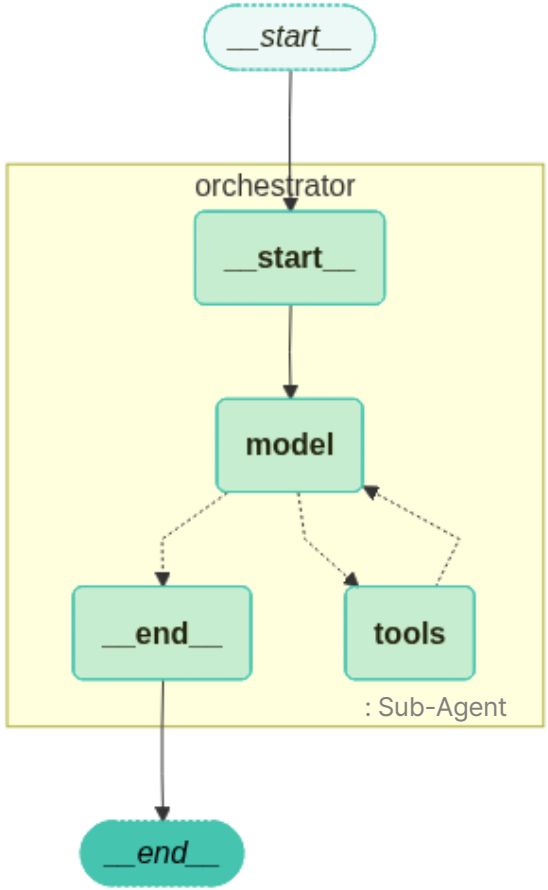
# 2-(2). Orchestrator

| 구분 | | Supervisor | Orchestrator-Workers |
|---|---|---|---|
| Pattern | |  |  |
| 역할 | | • 매 결과를 받아 다음에 누구를 부를지 반복 판단<br>• 종료 시점을 스스로 결정하고, 결과를 종합하는 **중앙 제어자** | • 실행 전에 전체 작업을 하위 작업 N개로 분해<br>• 병렬 실행 결과를 하나로 취합 하여 완료 |
| 주요 특징 | 계획 수립 시점 | • **매 스텝마다** | • **실행 전 일괄** |
| | 하위 에이전트 | • State 기준으로 상황에 따라 1개씩 **순차 호출** | • **병렬 처리 (Fan-out)** |
| | Loop | • 필수적<br>• 매 스텝마다 Supervisor로 복귀 | • 선택적 – Replan(*), Re-decomposition 시에만 반복<br>• 하위 에이전트 처리 결과는 Synthesis에서 수렴 |

(※) Replan이 잦아질수록 Orchestrator는 Supervisor로 수렴
: 재분해를 매 스텝 하게 되면, Supervisor 역할과 유사

# 2-(3). Agent-as-Tool

Orchestrator가 다른 전문화된 에이전트들을 도구처럼 호출하는 구조




구조적 특징 :

- Sub-Agent는 실행 단위 역할 담당하나, 외부에서는 도구(Tool)로 보임
  (※) Tool과 차이점 : 호출되는 도구 내부에서 자체적인 판단/추론 루프를 보유하여 복잡한 다단계 작업을 자율 처리함
- Sub-Agent의 내부 복잡성을 숨기고 입출력 인터페이스만 노출
- State Isolation : Sub-Agent가 일하는데 필요한 데이터만 전달

적합한 케이스 :

- 복잡한 라우팅 로직 없이 Orchestrator 노드 하나로 구성 가능
- 전문화된 Sub-Agent의 재사용성과 유연한 조합이 중요

고려사항 :

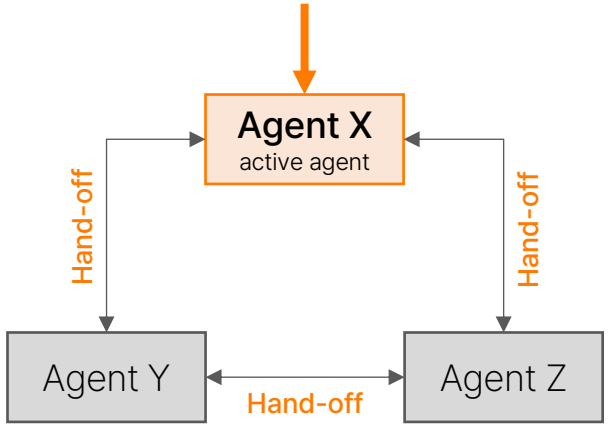
- Orchestrator 의존성 : 상위 에이전트 판단 오류 시 전체 영향
- Latency & Cost : Sub-Agent 루프 실행 따른 비용 누적
- Context Loss : 격리된 프롬프트로 인해 Sub-Agent 판단 정보 부족할 위험 잔존

※ 캡슐화 하는 이유

- Scoping : 무슨 데이터를 넘길까 – 입력 필터링
- 캡슐화 : 상위 입장에서는 입력/출력 스키마만 알면 됨. 여러 상위 시스템에서 재사용 가능
  - 로직 변경 : 내부 로직이 변경되더라도 상위에는 영향 없음
  - 오염 방지 : 내부에서 작업 중 컨텍스트는 상위 입장에서는 노이즈가 될 수 있음
  - 블랙박스 : 상위가 하위의 단계를 신경 쓰지 않아도 됨
- Trade-off : 관측성 저하(디버깅 난이도), Context Loss 위험

# 2-(4). Swarm

Supervisor 없이 에이전트들이 서로에게 직접 제어권을 넘기며 협업하는 분산 구조




구조적 특징 :

- 중앙 제어 노드 없음 – 에이전트 간 직접 Hand-off (Decentralized)
- 각 에이전트가 처리/이관을 스스로 결정 (Peer Autonomy)
- 제어권이 대화 맥락에 따라 에이전트 간 이동
- State에 제어권을 하나의 값으로 관리 (active agent)

적합한 케이스 :

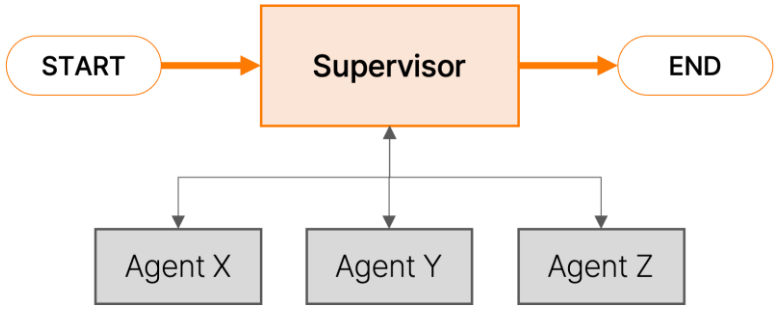
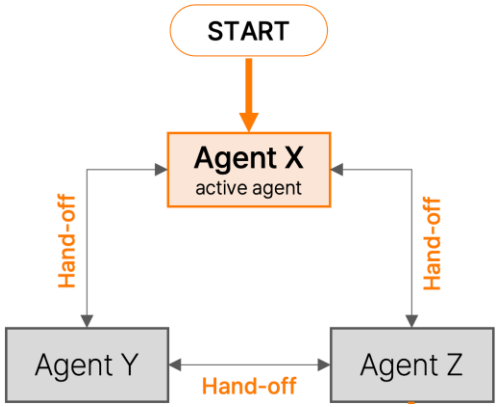
- 대화형 상호작용에서 주제/역할이 자주 전환되는 경우
- 전문 역할이 명확하고 순차 협업(이어달리기)이 잦은 경우
- 중앙 병목 없이 유연한 이관이 필요한 서비스

고려사항 :

- 전역 통제 부재 → 종료/품질 기준 보장 위한 탄탄한 설계 필요
- Hand-off 따른 핑퐁, 무한루프 위험 → 이관 규칙, 상한 필요
- active agent 유지 위한 checkpointer 관리

# 2-(4). Swarm

Supervisor 없이 에이전트들이 서로에게 직접 제어권을 넘기며 협업하는 분산 구조

| 구분 | | Supervisor | Swarm |
|---|---|---|---|
| Pattern | |  |  |
| 역할 | | • 매 결과를 받아 다음에 누구를 부를지 반복 판단<br>• 종료 시점을 스스로 결정하고, 결과를 종합하는 **중앙 제어자** | • 각 에이전트가 처리/이관을 스스로 결정하는 동료<br>• **중앙 제어자** 없음 |
| 주요 특징 | Routing/ Hand-off | • Supervisor 경유<br>• 에이전트는 Supervisor로 복귀 후 재배정 | • 에이전트 간 **직접 Hand-off** (Peer-to-Peer) |
| | State | • Supervisor 중심<br>• Global State 공유 | • State에 제어권을 하나의 값으로 관리<br>• 작업 유지 위해 checkpointer 필요 |
| | Loop 제어 | • 필수적<br>• 매 스텝마다 Supervisor로 복귀 | • 넘겨 받은 에이전트가 이어서 진행<br>• 종료는 각 에이전트가 담당 |

# [참고] Key blocks

## Supervisor

```python
supervisor_with_delegation = create_agent(
    model="gpt-4.1",
    tools=[
        assign_research_with_description,
        assign_math_with_description,
    ],
    system_prompt=(
        "You are an advanced supervisor that delegates tasks with explicit instructions.\n\n"
        "DELEGATION STRATEGY:\n"
        "When assigning tasks to agents, provide:\n"
        "1. Clear, specific instructions about what to find or calculate\n"
        "2. All necessary context and parameters\n"
        "3. Expected output format or requirements\n"
        "4. Any constraints or special considerations\n\n"
        "AGENT CAPABILITIES:\n"
        "- Research Agent: Web searches, data gathering, fact-checking\n"
        "- Math Agent: All mathematical operations and analysis\n\n"
        "WORKFLOW RULES:\n"
        "- Create detailed task descriptions for each delegation\n"
        "- Process complex requests step-by-step\n"
        "- Ensure each agent has all information needed to succeed\n"
        "- Synthesize results into comprehensive final answers"
    ),
    name="supervisor",
)
```

- 프롬프트 기준으로 매 스텝 "다음 실행 에이전트" 선택
- 라우팅 규칙 명시 → 제어 로직이 프롬프트에 집중

```python
assign_research_with_description = create_task_description_handoff_tool(
    agent_name="research_agent",
    description="Assign research task with specific instructions.",
)
```

```python
def create_task_description_handoff_tool(
    *, agent_name: str, description: str | None = None
):
    """Create a handoff tool that includes explicit task description"""
    name = f"transfer_to_{agent_name}"
    description = (
        description or f"Transfer task with detailed instructions to {agent_name}."
    )

    @tool(name, description=description)
    def handoff_tool(
        # Supervisor가 작성한 작업 설명
        task_description: Annotated[
            str,
            "Detailed description of what the agent should do, including all context
        ],
        # 상태 주입 (LLM은 무시)
        state: Annotated[MessagesState, InjectedState],
    ) -> Command:
        """Execute handoff with explicit task description"""
        # 작업 설명을 사용자 메시지로 변환
        task_description_message = {"role": "user", "content": task_description}
        # 에이전트에게 전달할 새로운 상태 생성
        agent_input = {**state, "messages": [task_description_message]}

        return Command(
            # Send를 사용하여 특정 상태를 에이전트에게 전달
            goto=[Send(agent_name, agent_input)],
            graph=Command.PARENT,
        )

    return handoff_tool
```

- 대상 에이전트에게 넘김
- 에이전트는 작업 후 Supervisor 복귀

# [참고] Key blocks

## Orchestrator–Workers

```python
planner_llm = init_chat_model("gpt-4.1").with_structured_output(Plan)


def orchestrator(state: OrchestratorState):
    prompt = (
        "당신은 리서치 작업을 조율하는 Orchestrator입니다.\n"
        "주어진 주제를 서로 '독립적으로' 병렬 조사할 수 있는 3~5개의 하위 주제로 분해하세요.\n"
        "각 하위 주제는 다른 하위 주제의 결과에 의존하지 않아야 합니다.\n\n"
        f"주제: {state['topic']}"
    )
    plan = planner_llm.invoke(prompt)
    print(f"🧩 분해된 하위 주제 {len(plan.subtasks)}개:")
    for i, s in enumerate(plan.subtasks, 1):
        print(f"   {i}. {s}")
    return {"subtasks": plan.subtasks}
```

- state[`topic`]을 하위작업 N개로 실행 전 동적 분해
- 분해된 작업은 state[`subtasks`] 저장

```python
def assign_workers(state: OrchestratorState):
    # subtasks 개수만큼 워커를 병렬로 fan-out
    return [Send("research_worker", {"subtask": s}) for s in state["subtasks"]]
```

- Subtasks 개수만큼 워커를 Fan-out

```python
def synthesizer(state: OrchestratorState):
    joined = "\n\n".join(
        f"### 하위 주제: {r['subtask']}\n{r['findings']}" for r in state["results"]
    )
    prompt = (
        "다음은 여러 워커가 '병렬로' 조사한 결과입니다. "
        "중복을 정리하고 하나의 일관된 종합 브리핑으로 작성하세요.\n\n"
        f"원 주제: {state['topic']}\n\n{joined}"
    )
    report = synth_llm.invoke(prompt)
    return {"final_report": report.content}
```

```python
class OrchestratorState(TypedDict):
    topic: str                                    # 사용자 입력(전체 주제)
    subtasks: List[str]                           # Orchestrator가 분해한 하위 주제들
    results: Annotated[List[dict], operator.add]  # 워커들의 병렬 결과 (리듀서로 병합)
    final_report: str                             # Synthesis 결과
```

- Fan-in : Hand-off 없이 취합

# [참고] Key blocks

## Swarm

```python
research_agent = create_agent(
    model="gpt-4.1",
    tools=[web_search, to_math],
    system_prompt=(
        "You are a research specialist in a DECENTRALIZED swarm (no supervisor).\n"
        "- Do web research and report facts.\n"
        "- If the task needs calculation, HAND OFF to math_agent via the handoff tool
        "- You yourself decide when to hand off."
    ),
    name="research_agent",
)

math_agent = create_agent(
    model="gpt-4.1",
    tools=[*math_tools, to_research],
    system_prompt=(
        "You are a math specialist in a DECENTRALIZED swarm (no supervisor).\n"
        "- Perform calculations only.\n"
        "- If you need more facts to compute, HAND OFF to research_agent via the hand
        "- You yourself decide when to hand off."
    ),
    name="math_agent",
)
```

- 각 에이전트가 Handoff 도구를 자기 도구로 보유
  → 감독자 없이 peer 간 이관

```python
# 계산이 필요하면 research_agent -> math_agent 로 직접 이관
to_math = create_handoff_tool(
    agent_name="math_agent",
    description="계산·수치 분석이 필요하면 math_agent에게 제어를 넘긴다.",
)
# 추가 사실 조사가 필요하면 math_agent -> research_agent 로 직접 이관
to_research = create_handoff_tool(
    agent_name="research_agent",
    description="추가 웹 조사·사실 확인이 필요하면 research_agent에게 제어를 넘긴다.",
)
```

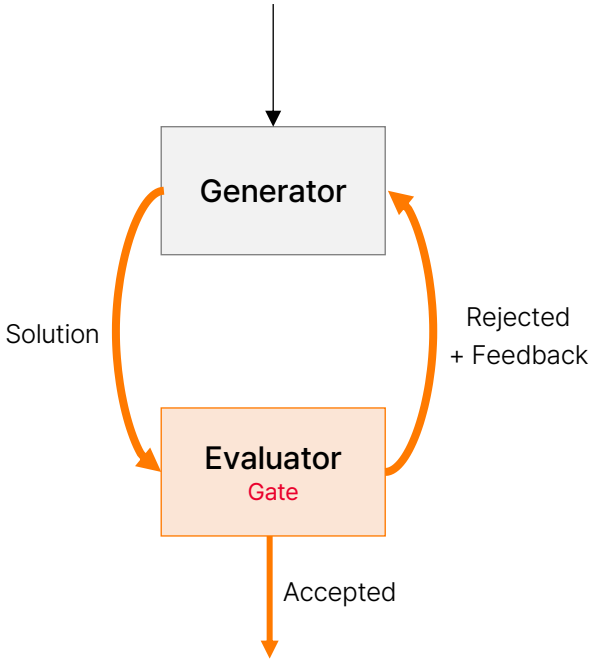
- 에이전트가 다른 에이전트에게 직접 제어권을 넘김
- 내부적으로 다음 내용으로 라우팅과 동시에 active agent 갱신
  Command(goto=대상, update={"active_agent": 대상})

```python
workflow = create_swarm(
    [research_agent, math_agent],
    default_active_agent="research_agent",  # 시작 활성 에이전트
)
app = workflow.compile(checkpointer=InMemorySaver())
```

- 감독자 노드 없이 그래프 구성, 시작 활성 에이전트 지정 필요
- active_agent를 대화 간 유지 시키기 위해 checkpointer 함께 적용

# 2-(5). Evaluator-Optimizer

생성 에이전트(Optimizer)의 결과물을 평가 에이전트(Evaluator)가 검증하고 피드백 하여 개선하는 워크플로우




구조적 특징 :

- Generator와 Evaluator의 역할 및 책임을 명확히 분리
- Evaluator가 명확한 평가 기준(Rubric) 따라 피드백을 제공하고 피드백 루프 수행
- 피드백 바탕으로 품질 개선 가능

적합한 케이스 :

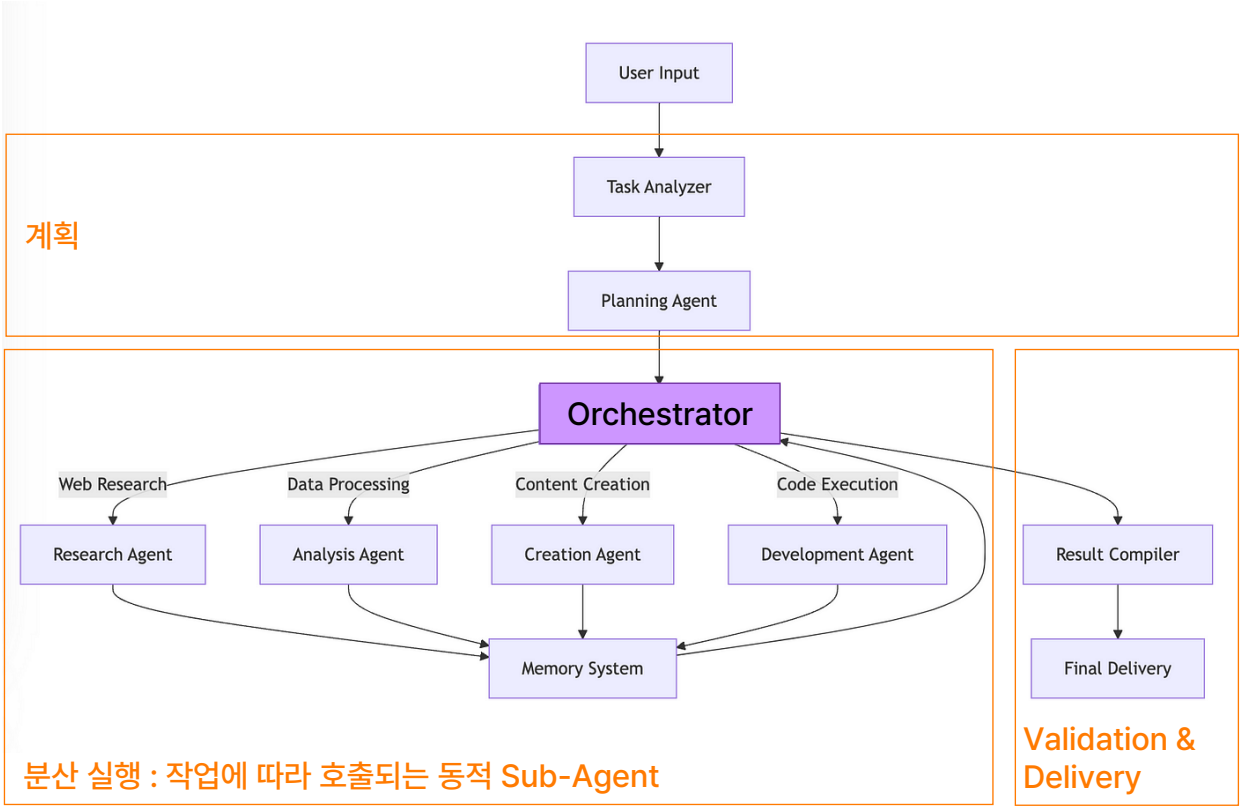
- 코드 생성, 번역, 기술 문서 등 명확한 검증 기준이 존재하는 작업
- 단일 실행 만으로 원하는 품질의 결과물을 얻기 어려운 Task

고려사항 :

- 무한 루프 방지 위한 최대 반복 횟수 및 종료 조건 설정 필수
- Evaluator 평가 기준이 모호하면 피드백 신뢰도 저하 및 토큰/비용 낭비

# 2-(6). Hybrid Structure

의사결정 권한을 계층적, 상황별로 배분하여 설계한 혼합 구조




적합한 케이스 :

- 대규모 시스템
- 다단계 Workflow
- Plan → Execute → Observe → Update → Verify → Refine/Deliver
- 품질 + 비용 균형 필요

고려사항 :

- 설계 난이도가 가장 높음
- Context/State/Memory 관리 필요
  - 현재 대화 문맥
  - 실행 trace와 관찰 결과
  - Todo 및 계획 상태
  - 파일 시스템과 프로젝트 산출물
  - 오류/실패 로그
  - 도구/환경 상태 : 세션, 실행 프로세스, 생성파일, 코드 실행 환경 등

*Source : https://levelup.gitconnected.com/manus-im-the-ai-agent-that-works-while-you-rest-d5505697b7de (modified)*

# TODAY QUIZ

1. 아래 상황에 가장 적합한 패턴은 무엇인가요?

   사내 데이터로 분석을 도와주는 Analytics Agent를 설계하고자 합니다.
   메인 에이전트가 사용자와 대화를 주도하다가,
   사내 데이터 조회가 필요할 때만 DW 쿼리 전문 에이전트를 호출해야 합니다.
   보안 정책상 전체 대화 맥락을 넘기지 않고 필요한 파라미터만 전달하며,
   결과를 받아 메인 에이전트가 계속 후속 작업을 진행합니다.

   A. Pipeline
   B. Supervisor
   C. Agent-as-Tool
   D. Orchestrator-Workers
   E. Swarm

# TODAY QUIZ

2. 아래 상황에 가장 적합한 패턴은 무엇인가요?

고객서비스 지원 위한 에이전트를 설계/구축 하고자 합니다.
메인 에이전트가 문의를 받아 매 단계마다 어느 팀에게 맡길지를 판단하여 위임해야 합니다.
각 전문 에이전트 – 요금팀, 기술팀, 해지방어팀 – 이 처리를 끝내면 다시 메인 에이전트로 제어가 돌아와야 합니다.
메인 에이전트는 충분히 해결되었는지를 판단하여 종료 시점을 정하고 최종 답변을 종합합니다.
품질, 종료 통제를 중앙에서 컨트롤 하는 것이 중요합니다.

A. Pipeline
B. Supervisor
C. Agent-as-Tool
D. Orchestrator-Workers
E. Swarm

# TODAY QUIZ

3. 아래 상황에 가장 적합한 패턴은 무엇인가요?

시장조사 리포트를 생성하는 시스템을 설계/구축 하고자 합니다.
요청이 들어오면 실행 전에 전체 작업을 경쟁사 5곳에 대한 조사처럼 독립적인 하위작업 N개로 한 번에 분해하고,
여러 하위 에이전트에게 병렬로 동시에 실행시킨 후 마지막에 하나의 리포트로 취합 합니다.
SLA 위해 목표 시간에 맞춰 여러 개 리포트가 생성되는 것이 중요합니다.

A. Pipeline
B. Supervisor
C. Agent-as-Tool
D. Orchestrator-Workers
E. Swarm

# TODAY QUIZ

4. 아래 상황에 가장 적합한 패턴은 무엇인가요?

옴니채널 고객 상담 시스템을 설계/구축 하고자 합니다.
상담을 맡은 에이전트가 주제가 바뀌면 (e.g., 요금 문의 → 기술 장애 → 환불)
자신이 직접 해당 전문 에이전트에게 대화를 넘기고 빠지면 됩니다.
넘겨 받은 다음 에이전트는 대화를 이어가면서 목적에 맞게 대응합니다.

A. Pipeline
B. Supervisor
C. Agent-as-Tool
D. Orchestrator-Workers
E. Swarm

# TODAY QUIZ

5. 아래 상황에 가장 적합한 패턴은 무엇인가요?

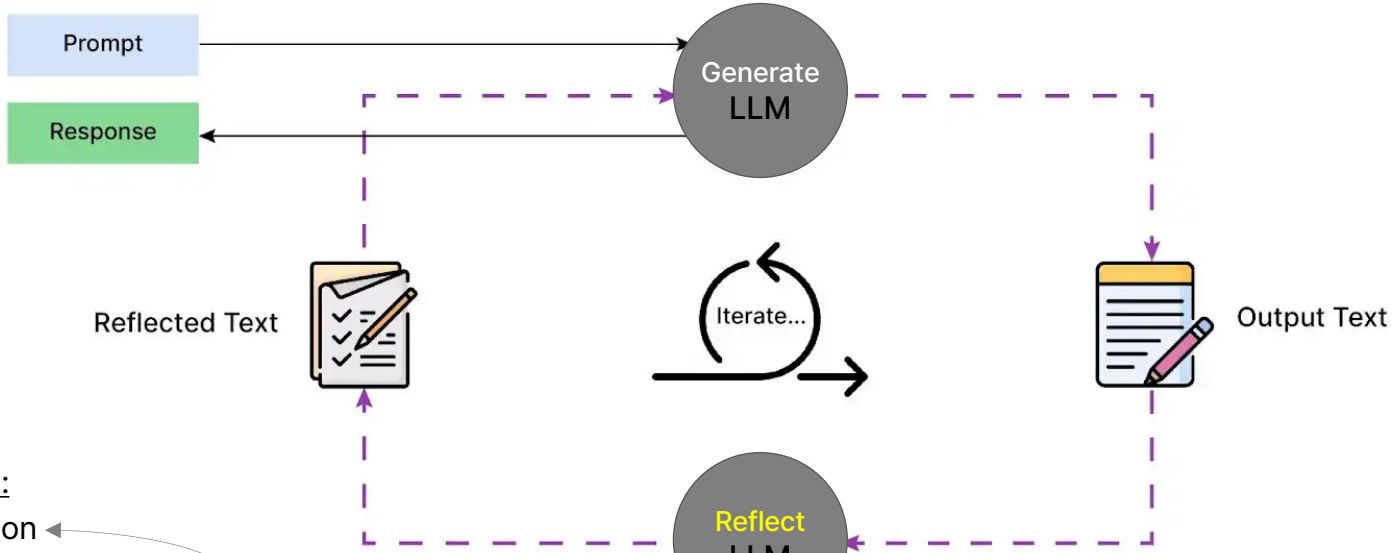
규제 대응 위한 계약서 처리 에이전트를 설계/구축 하고자 합니다.
매일 들어오는 계약서는 사내 업무 프로세스에 따라 처리되면 됩니다.
업무 프로세스는 컴플라이언스 규정으로 거의 정해져 있고, 단계마다 앞 단계 산출물이 다음 단계로 전달되면 됩니다.

A. Pipeline
B. Supervisor
C. Agent-as-Tool
D. Orchestrator-Workers
E. Swarm

# 3. Reflection

품질 통제 메커니즘

- AI가 자기 응답을 되돌아보고 더 나은 답을 만드는 과정
- 루프 자체도 자동화 하여 사람의 반복 개입 없이도 품질 향상 가능
- Human : Reflection 구조 설계, 피드백 방향 설계, 최종 결과 검토 및 선택




Reflection 유형 :

- Self-Reflection
- Tool-based Reflection
- External Judge Reflection

- Based on Success Criteria
- Based on Quality, Compliance, Operational

*Source : https://www.analyticsvidhya.com/blog/2024/10/agentic-ai-reflection-pattern/*

# 4. 설계 유의사항

Multi-Agent System, 3대 실패 범주

1. 도입 판단부터
   - 단일 에이전트로 충분한지 먼저 검토
   - 에이전트를 추가했을 때 따라오는 설계 난이도 검토 – Cost, Trace, Fallback, Latency, etc.

2. 역할과 책임 분배 – 명세/설계 실패 (약 40%)
   - 누가 무엇을 할 것인지 명확 : 모호한 역할 정의 = 최대 실패 원인
   - 각 에이전트는 독립적, 중복되지 않도록 설계
   - 종료 조건 명시

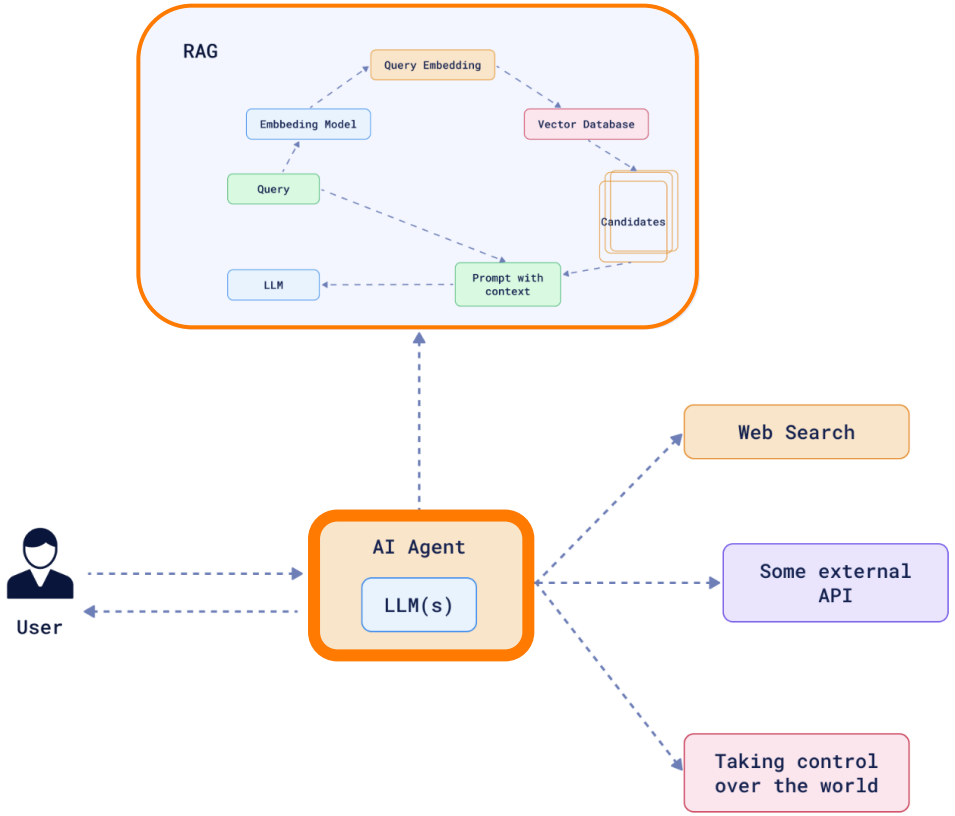
3. Agent Interface & Context – 에이전트 간 정렬 실패 (약 35%)
   - 데이터 교환 방식 통일 → State (형식, 정보 단위 규정)
   - Context Engineering : 선택, 압축, 격리 등을 통해 Context 저하 방지
   - Global State 관리 : 오염된 쓰기 하나가 전역 전파 → Agent Scoping

4. Loop & Verification – 검증/종료 실패 (약 20%)
   - 끝장 토론이 아니라, 끝을 내주는 조건 또는 에이전트 추가 – 최대 반복 횟수, 의사결정 에이전트, Voting/Scoring, etc.
   - 검증 단계 필수 : A의 오류가 B의 사실로 전파되지 않도록 차단 필요
   - Observability : 실행 그래프 추적은 선택 아님

*Source : https://arxiv.org/pdf/2503.13657*

# 1. Agentic RAG

- AI가 정보를 검색하고 생성하는 방식에서 Agent 처럼 동작하도록 설계된 개념
- 단순히 정보를 검색하고 답변을 생성하는 것을 넘어, 반복적인 검색과 생성 과정을 통해 유의미한 결과를 도출할 수 있음

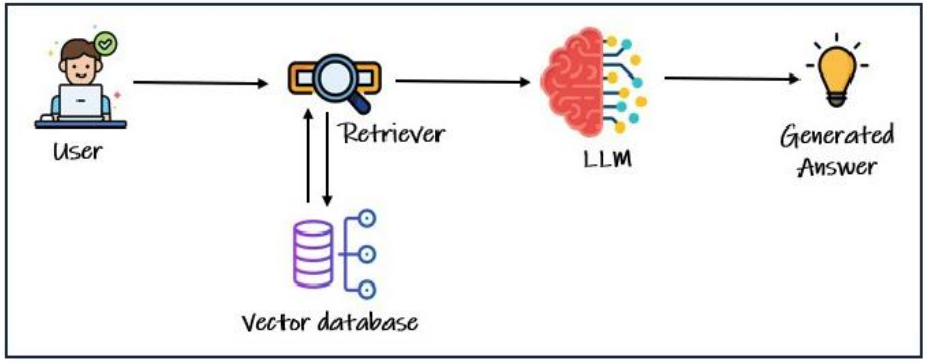



*Source : https://qdrant.tech/articles/agentic-rag/*

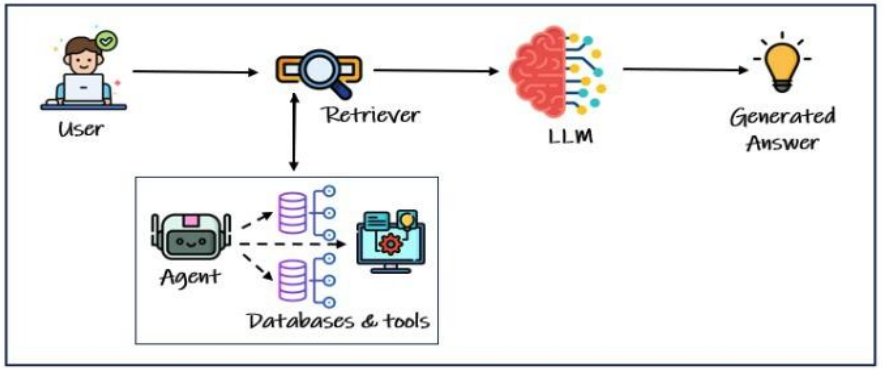
# 1. Agentic RAG

**단순 검색 도구가 아니라, 목적 지향적 사고를 수행하는 AI Agent에게 필요한 도구**

**RAG Architecture**




**Agentic RAG Architecture**



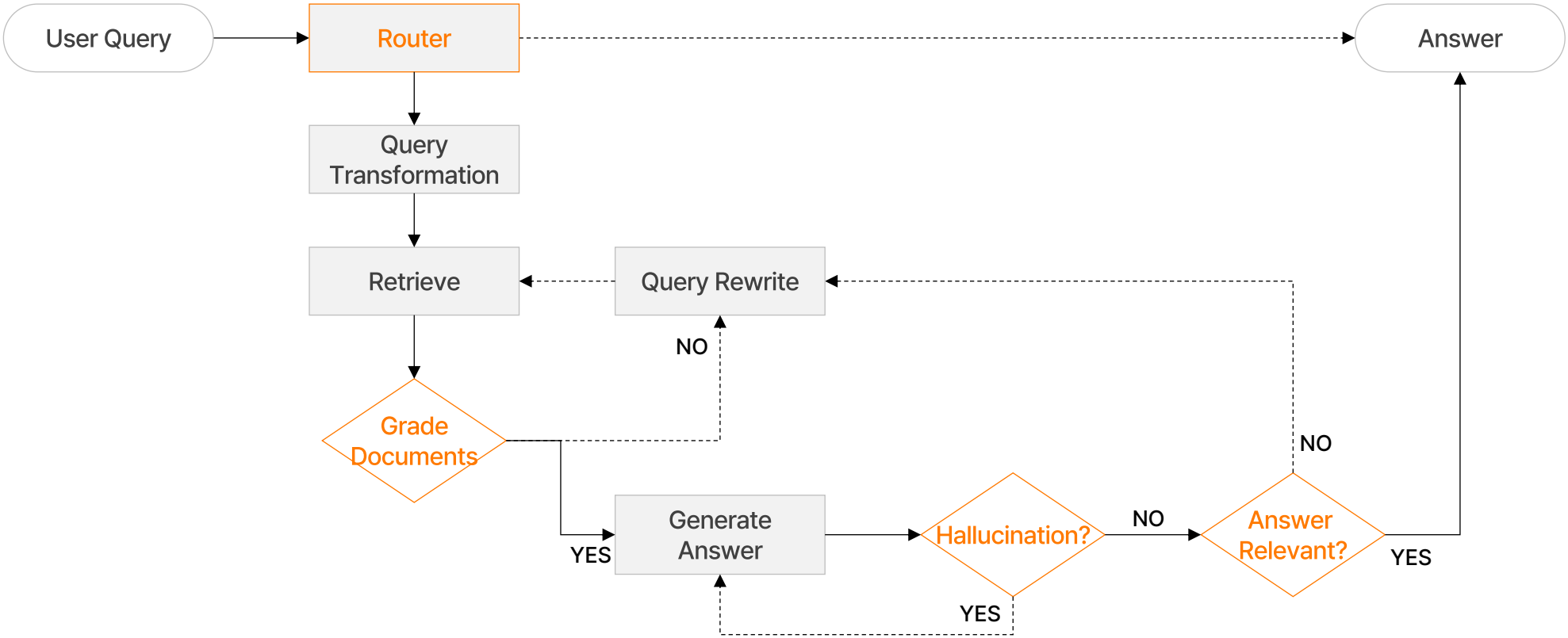

**Agentic RAG = [Planning + Reflection + Collaboration] + RAG**

| Strategy | | |
|---|---|---|
| Planning | • 검색 → 답변 생성 | • AI가 목적에 맞게 검색 전략을 스스로 설계함<br>• 어떤 문서를 찾을지, 어떤 순서로 검색 · 요약 · 생성할지 결정 |
| Reflection | • 검색된 정보 그대로 사용 (별도 평가 없음) | • AI가 생성된 응답에 대해 스스로 검토함<br>• 검색 내용이 불충분하면 다시 검색→수정 루프를 수행 |
| Collaboration | • Agent 아닌 정보 검색 프로세스 (협업 없음) | • 여러 역할을 가진 Agent들이 협업<br>(e.g., 검색 에이전트, 요약 에이전트, 응답 검토 에이전트 등 역할 분담) |

*Source : https://www.deepchecks.com/glossary/agentic-rag/*

# 2. Workflow

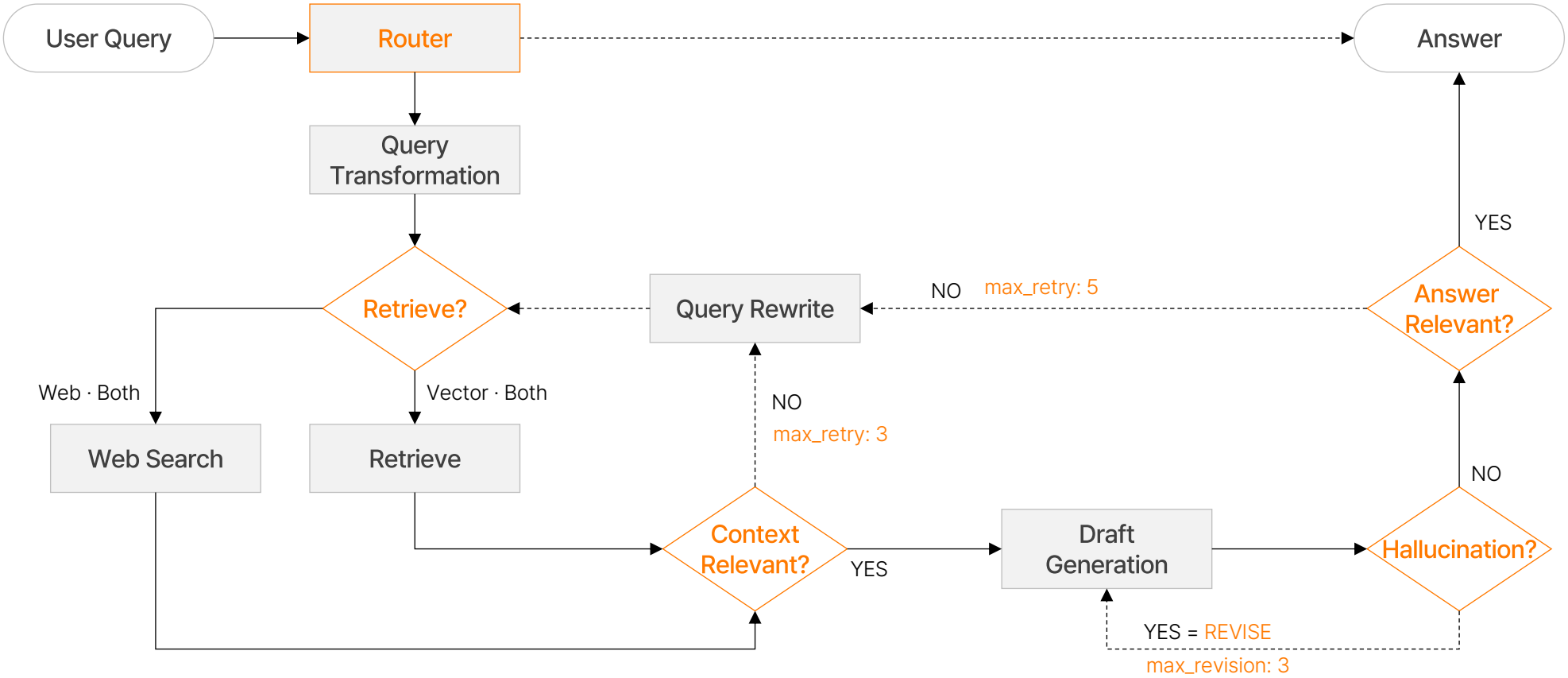
## Agentic RAG



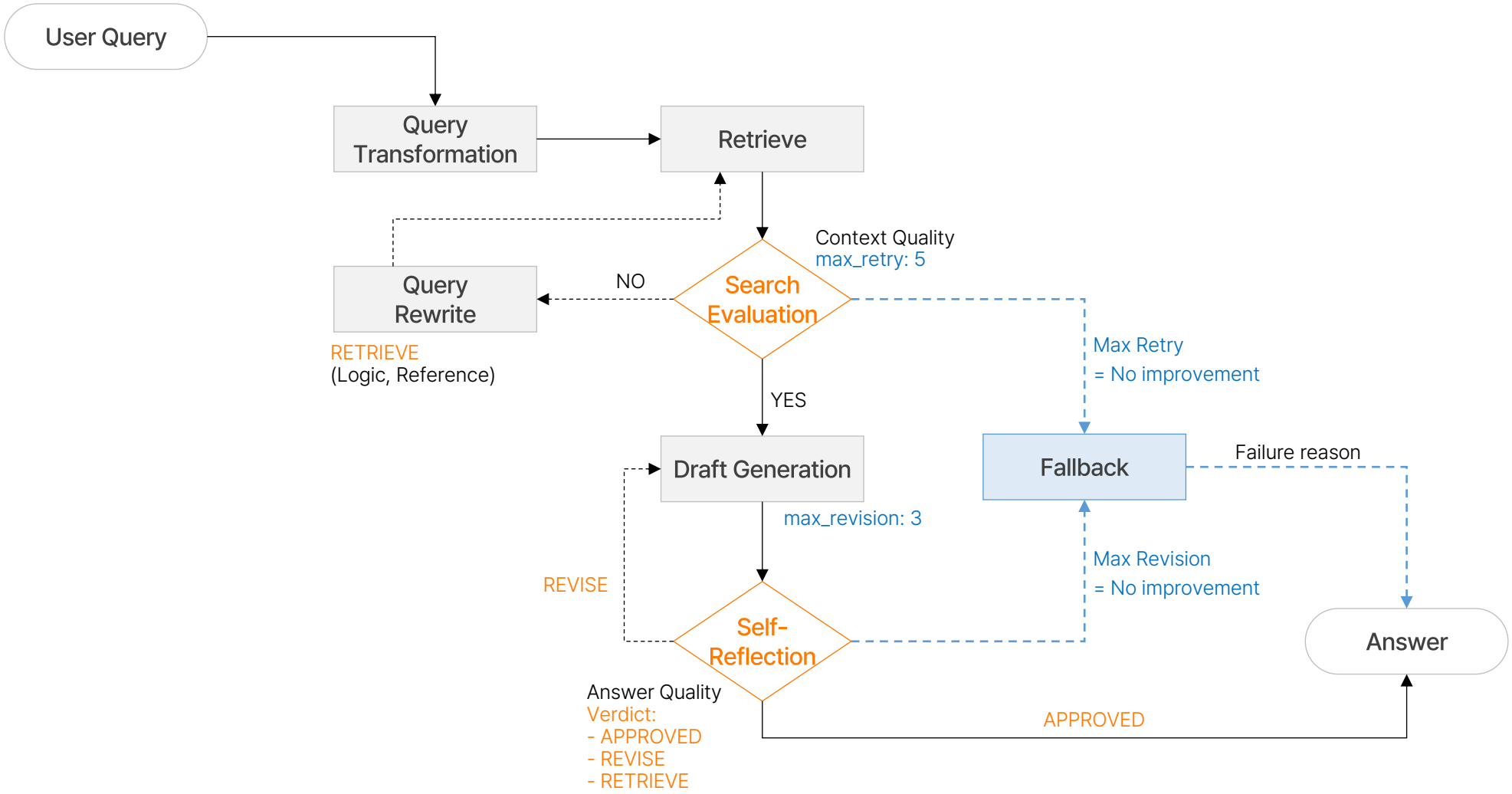

※ Remark

- Router : 질문 유형에 따라 경로 분기하는 역할로 확장

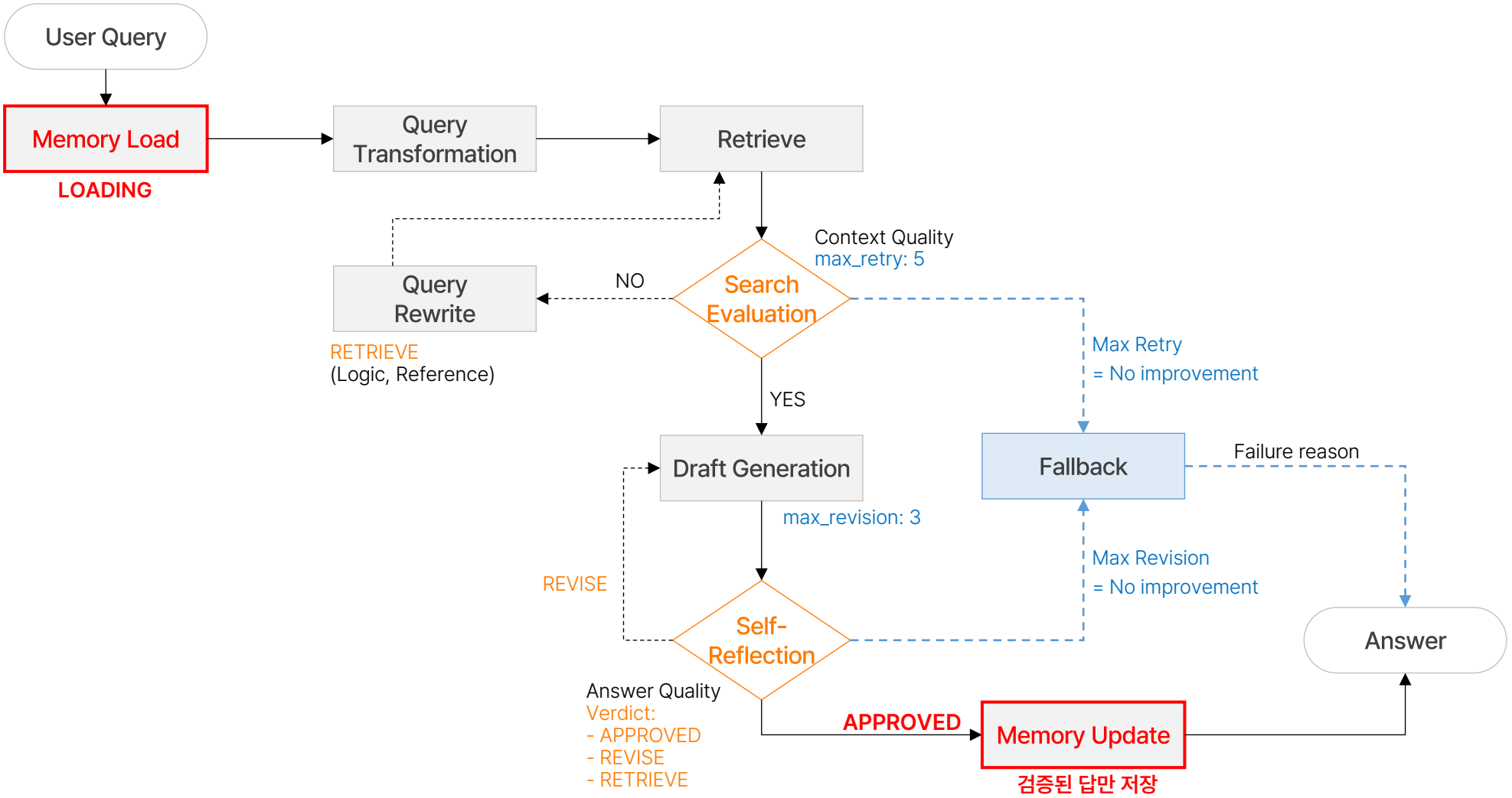
# 1. (+) Control Strategy



# 2. (+) Loop


User Query
Query Transformation
Retrieve
Context Quality
max_retry: 5
Search Evaluation
NO
Query Rewrite
RETRIEVE
(Logic, Reference)
YES
Max Retry
= No improvement
Draft Generation
Fallback
Failure reason
max_revision: 3
REVISE
Max Revision
= No improvement
Self-Reflection
Answer
Answer Quality
Verdict:
- APPROVED
- REVISE
- RETRIEVE
APPROVED

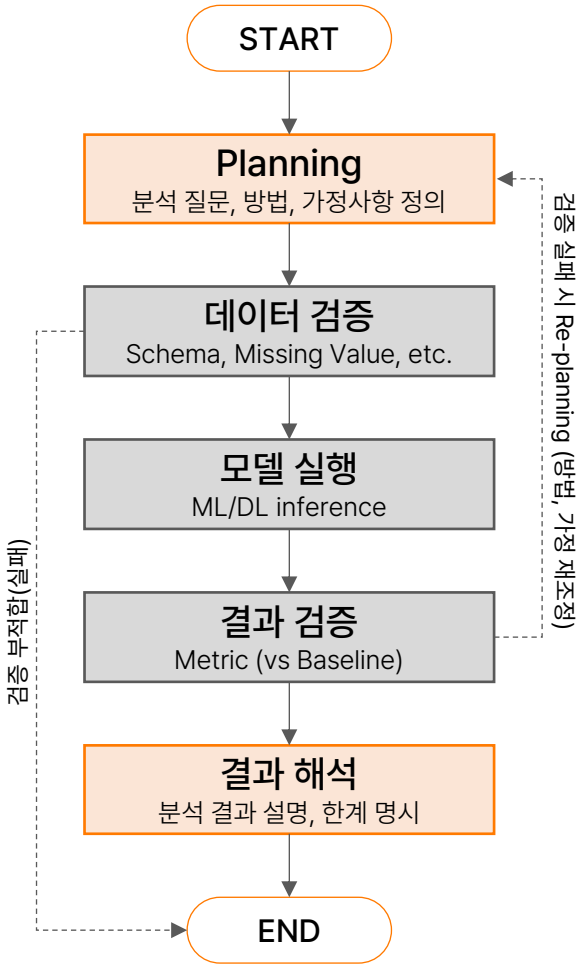
# 3. (+) Memory


User Query
Memory Load
LOADING
Query Transformation
Retrieve
Context Quality
max_retry: 5
Search Evaluation
NO
Query Rewrite
RETRIEVE
(Logic, Reference)
YES
Max Retry
= No improvement
Draft Generation
Fallback
Failure reason
max_revision: 3
Max Revision
= No improvement
REVISE
Self-Reflection
Answer
Answer Quality
Verdict:
- APPROVED
- REVISE
- RETRIEVE
APPROVED
Memory Update
검증된 답만 저장

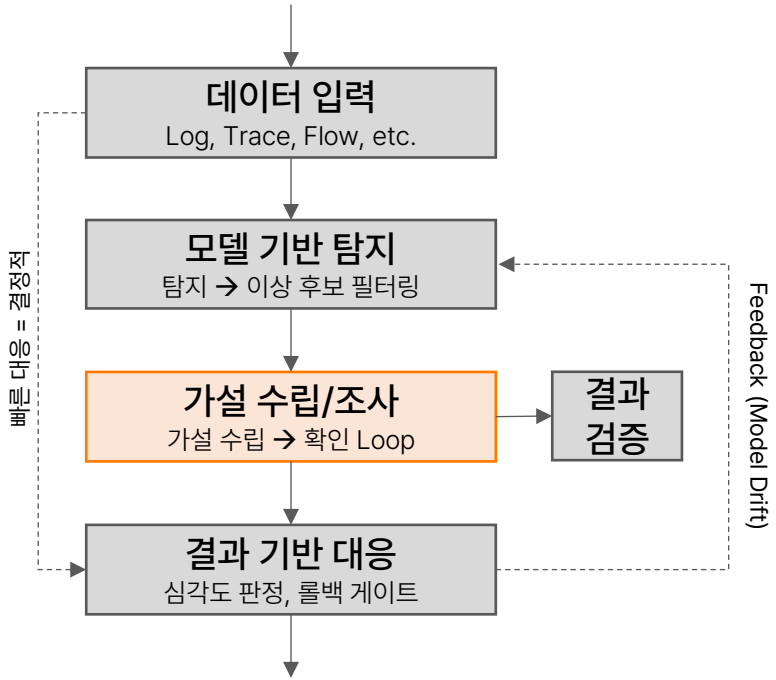
# 3. Agentic Analytics

데이터 분석과 LLM 역할을 명확하게 구분하여 에이전트 설계하는 것이 필요 – LLM vs Non-LLM 계층

LLM Non-LLM

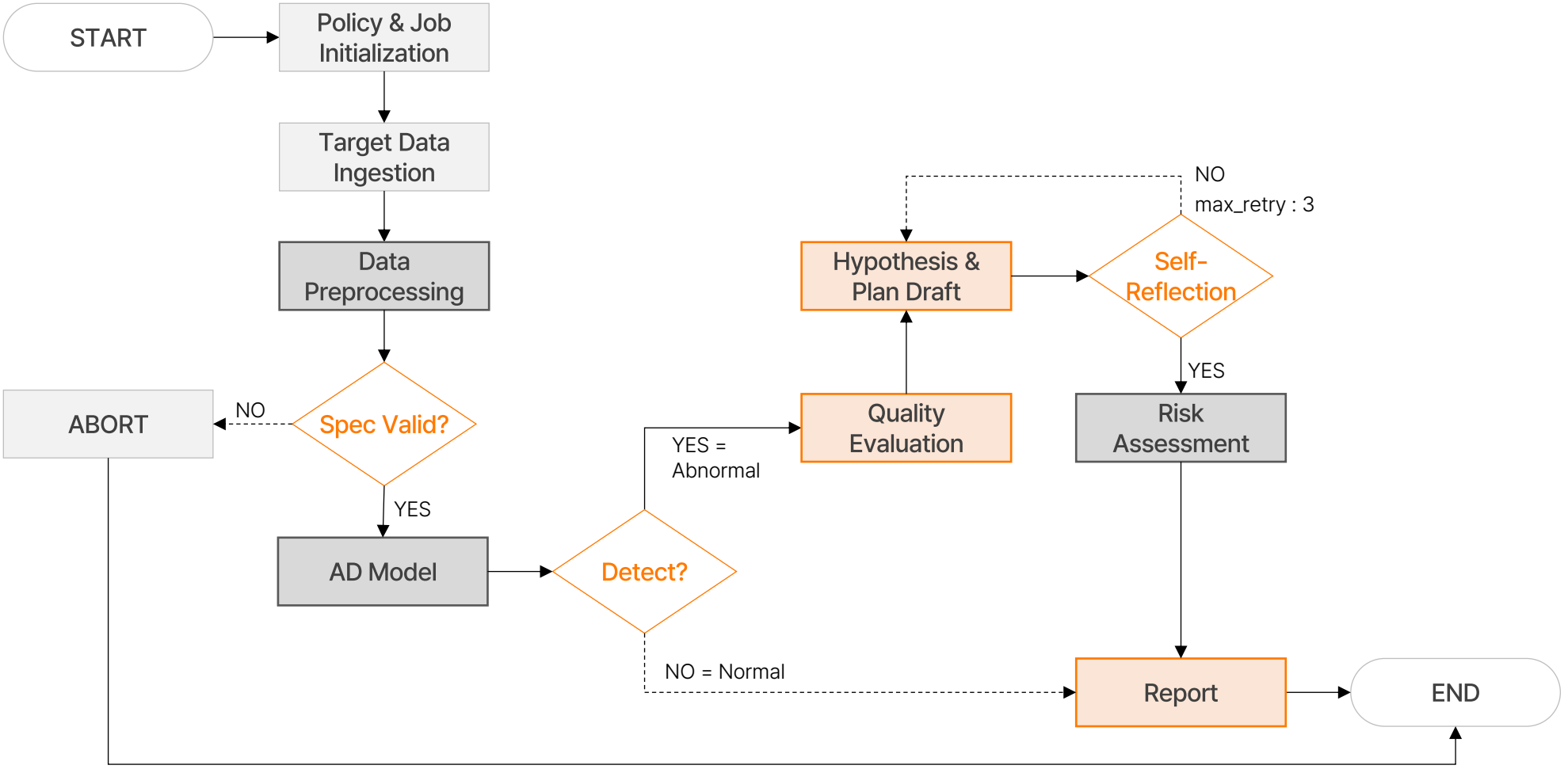



(e.g.) 로그 기반 이상탐지



# 3. Agentic Analytics

Workflow – [Case] 로그 기반 이상탐지 LLM Non-LLM



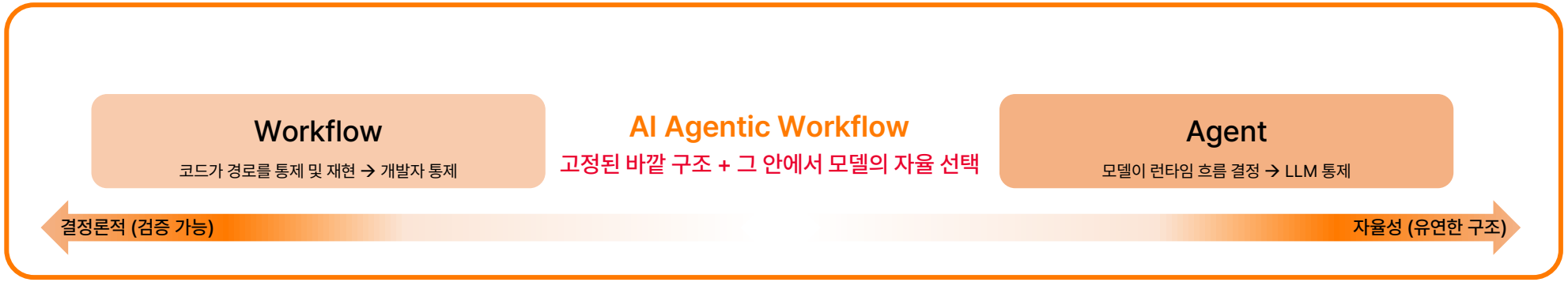
# 3. AI Agentic Workflow

SUMMARY

SUMMARY

# 1. AI Agentic Workflow

2026 – 흐름을 누가 통제하는가?



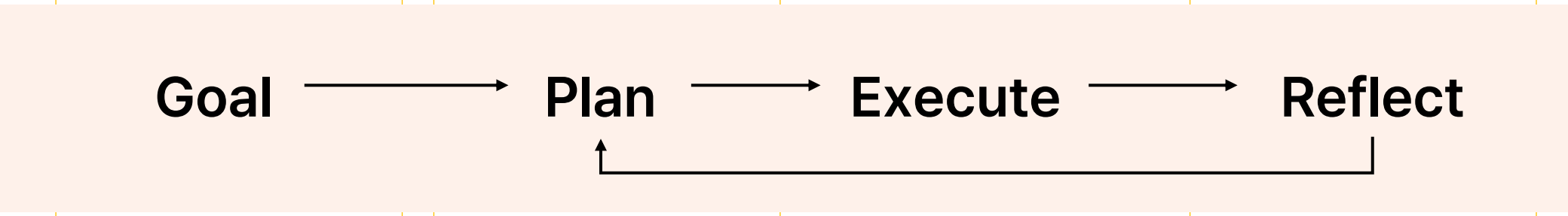

- Agentic은 정도의 문제 : Agent냐 아니냐 보다 "얼마나 자율적"인가의 관점으로 해석
- 가장 단순한 것부터 : 높은 신뢰성을 요구하는 Production에서는 Agent 보다 Workflow 기반으로 개발 비중 높음 ← 검증 가능, 비용 관리
- Agent ≠ Loop : Self-Correction Loop이라 하더라도, 구조가 코드로 고정되어 있으면 Workflow임

SUMMARY

# 2. Design Framework

Agent는 사고 구조를 설계하는 것에서 시작합니다 !!!

Strategic Layer | Operational Layer




- 무슨 목적인가?
- 무엇을 목적 달성으로 볼 것인가? (Success Criteria)

- Task Decomposition
- Control Strategy : Linear, Loop, Branch, Retry
- Collaboration Structure : Centralized, Distributed, Hybrid

- Tool Invocation
- State Update
- Action Execution
- Memory

- Evaluation Mechanism : Reflection, Tool-based Validation, External Judge, etc.

SUMMARY

# 2. Collaboration Structure

**책임과 의사결정 권한**을 설계하는 단계

## Chaining

- 고정 순차 : 미리 정한 순서대로 연결. A → B → C
- 사람이 미리 고정. 런타임 결정 없음
- 앞 단계의 산출물이 다음 단계로 넘어감.
- 복귀 없음 (일방향)

Pattern :

- Pipeline
- 순차 워크플로우

## Routing, Delegation

- 중앙에서 작업 배분/위임
- 관리자의 지시 + 맥락이 다음 단계로 전달
- 일이 끝나면 제어가 중앙으로 복귀

Pattern :

- Supervisor : 매 스텝 순차 라우팅 (조건부 엣지로 라우팅)
- Orchestrator : 실행 전 계획수립/분해 + 병렬
- Agent-as-Tool : LLM의 tool-calling으로 서브에이전트를 함수처럼 호출

## Hand-off

- 담당 에이전트가 스스로 다른 에이전트에게 직접 넘김
- 현재 담당 에이전트가 런타임에 다음 순서 결정
- 제어권 + 맥락이 다음 단계로 전달
- 제어권이 넘어가면, 넘긴 쪽은 빠짐 (돌아오는 구조 없음)

Pattern :

- Swarm

## Feedback Loop

- 답변 생성(Generator) 결과를 평가(Evaluator)가 Rubric으로 검증
- Evaluator가 품질 게이트 역할
- 미달 시 결과물과 피드백을 Generator에게 전달하여 재생성
- 무한루프 방지 위해 최대 반복 횟수, 종료 조건 필요

Pattern :

- Evaluator-Optimizer

# TIP WebSearch, 효과적으로 사용하자

## Context 보강을 위해 WebSearch 노드를 추가"만" 하지 말고~

원인 진단 :

- **검색 자체의 특징**
  - 웹은 원래 텍스트, 이미지, 비디오에 걸쳐 이질적이고 노이즈가 많고 불완전하며 소스 간 충돌하는 정보로 구성되기도 함
  - 검색 결과의 신뢰도 편차가 큰 편
- **검색을 수행하는 에이전트와 Loop**
  - 소스를 잘못 고르거나, 좋은 결과를 무시하거나, over-exploration으로 산만해짐
  - Loop에서 잘못된 정보가 섞이면 전체 품질에 영향
- **Context Window**
  - 윈도우가 차면 LLM이 earlier context 무시/환각 하여 정확도 저하
  - Window 40% → 80% 되면, 답변 정확도는 94% → 61%

(Ref) *What Makes a Good LLM Agent for Real-world Penetration Testing?* (2026.02)

***Context Engineering :***

"검색하면 좋아지는 노드가 아니라, 관리가 필요한 서브 시스템"

**검색 전 – 전략**

- 모든 쿼리가 웹검색 수행하지 않음 → 필요한 질의만 검색 경로로
- Query Rewrite/Decomposition
- 범용적인 검색 보다 목적성을 가지는 Agent-native search API 선택
- 에이전트 오판 시 최신 정보 누락

**아키텍처 – 노이즈 격리**

- Searcher (sub) → Reasoner : 검색 raw 결과 처리를 sub-agent의 컨텍스트 내에서 수행 → 정제된 요약을 상위 에이전트 (Reasoner)에게 전달
- Raw 검색 결과는 하위에서 소비/폐기
- 메인에는 정제된 요약만 누적
- Latency, 복잡도 증가

**검색 후 ~ 주입 전 – 정제**

- 주입 전 : 검색 raw 결과를 그대로 넣지 않고 정제 후 주입 (rerank, compress, filter 등)
- 압축 = 출처를 보존하는 요약
- Relevance 노드 통해 소스 신뢰도, 관련성 검증 후 주입
- 과압축 시 신호 손실

**Loop – Context 관리**

- Sliding-window : 최근 K개만 유지
- Periodic summarization
- Observation masking : 오래된 tool output을 placeholder로 치환
- Proactive pruning : context editing
- Task 목적/성격에 맞는 전략 필요

AI Campus 생성형AI

# 4. AI Agent 구현

- LangChain 1.0
- LangGraph 구성요소 : State, Node, Edge
- Parallel Execution : Fan-out/Fan-in

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

# [참고] LangChain (0.x)




AS-IS :

- Prompt : 체인 내부에 로직이 섞여 있음
- Chain : Callback은 가능하지만 관찰 중심
- Chain : 실행 흐름과 실행 통제가 명확히 분리되지 않음

➔ "사고 구조"와 "운영 정책"이 뒤섞여 있음

한계점 :

- 운영 통제 : 비용/토큰 제한, Latency 제어
- 정책 통제 : 정책에 맞지 않는 Tool 호출 방지, Command 제어 등
- 모니터링 : Tracing, Audit, Logging

서비스를 위해 필요한 것들이 Chain 내부에 포함되면..
설계가 복잡해지고, 재사용성이 떨어짐

# 1. LangChain 1.0

2025.10.22 정식 릴리즈 (2.0 전까지는 breaking change 없음 약속)

0.x → GA, Generally Available

**에이전트 중심 재편**

- 새 표준 상위 API **`create_agent`** 도입 ← 기존 `AgentExecutor` 대체
- `create_agent`는 **LangGraph 런타임 위에서 실행**

**미들웨어 시스템 도입**

- 에이전트 루프의 **각 단계에 개입하는 세밀한 제어** 제공 (before/after model, wrap tool call, etc.)
- HITL, Summarization, PII 마스킹 등의 미들웨어 기본 탑재

**구조화 출력 개선**

- 메인 에이전트 루프에 통합
- 별도 LLM 호출 제거하여 Latency, Cost 절감

**Surface area 축소**

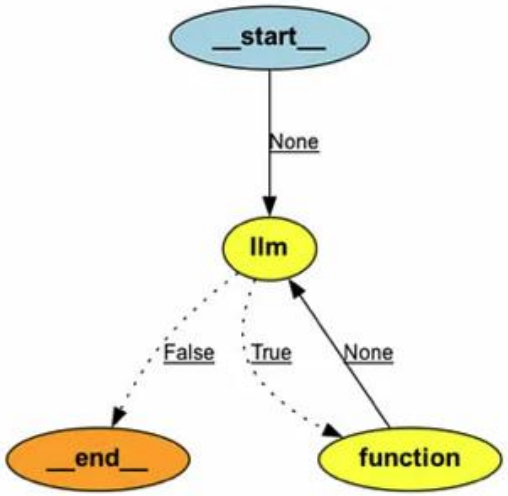
- 예전 기능 정리 : 레거시 기능은 `langchain-classic` 패키지로 이동
- 데이터 형식 통일 : 모델마다 상이한 반환 단위를 `content_blocks` 공통 형식으로 표준화

*Source : https://docs.langchain.com/langsmith/changelog, https://docs.langchain.com/langsmith/changelog*

# [참고] LangGraph

에이전트의 실행 흐름을 그래프로 설계하고 실행하는 런타임

Runtime : 실행되는(Run) 동안(time)
설계를 실제로 실행하며, 지금 상태를 기억하고 다음 단계를 챙기는 "실행 담당"




### 왜 "Chain"이 아니라 "Graph인가" :

- Chain : [Prompt → LLM → Output] 한 방향으로 한 번 통과하고 끝
- 복잡한 문제는 결과 확인, 검토, 재검색 등의 반복(Loop) 필요
- 되돌아오는 흐름, 조건 분기를 반영하려면 Chain 보다 Graph 필요

### Graph 요소 :

- State : 모든 단계가 함께 보고 갱신하는 작업 메모리
- Node : 실제로 일을 하는 한 단계
- Edge : 한 노드 다음에 어디로 갈지 연결. 조건부 엣지는 State를 보고 갈림길 선택

### 무엇이 가능한가 :

- 반복과 분기 : 에이전트가 스스로 판단하여 작업 반복/수정 → Agentic Workflow
- State 저장 : 중간에 끊겨도 이어서 실행, 긴 작업 맥락 유지
- HITL : 흐름 일시정지 → 사람이 승인/수정 후 재개

*Source : https://medium.com/data-science/from-basics-to-advanced-exploring-langgraph-e8c1cf4db787*

# [참고] LangChain? LangGraph?

## 필요한 추상화 수준 선택

### LangChain

`create_agent`

- 가장 흔한 에이전트 패턴을 이미 조립해 둔 상위 추상화
- 패턴 : 모델 호출 → 도구 실행 → (반복)
- 추상화 : 에이전트 내부를 덜 들여다봐도 되는 편의
  vs 무슨 일이 일어나는지 덜 보이는 불투명함

(대부분) 표준 패턴으로 에이전트 개발

- `create_agent`로 빠르게 만들고
- 그 안에서 커스터마이즈가 필요한 부분만 미들웨어로 처리

### LangGraph

State, Node, Edge

- 직접 조립하는 저수준 도구
- 그래프를 그리는 것은 매번 새롭게 에이전트 개발

표준 패턴 보다는

- 복잡한 제어 흐름 필요
- 비선형 분기, 다중 에이전트, 세밀한 State 설계 등

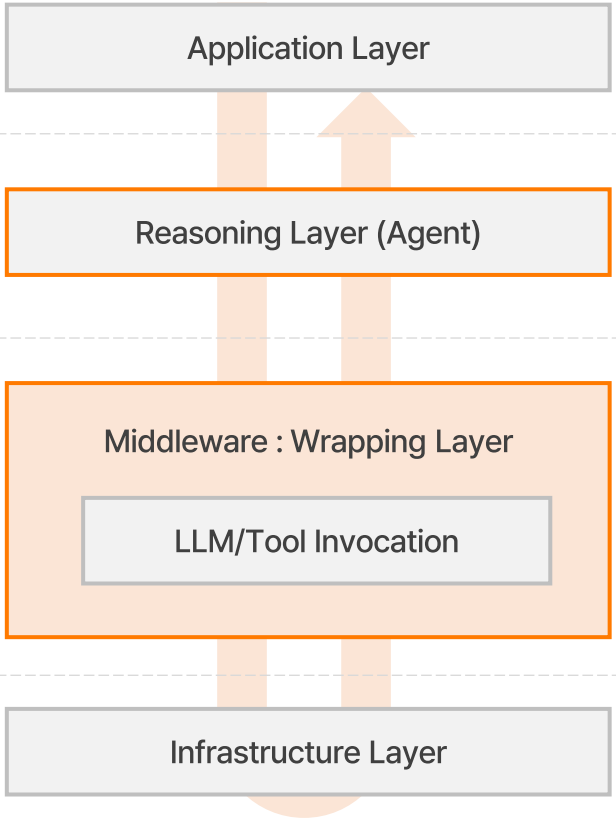
# 2. Architecture

Key Point

Agent(실행)와 Runtime Governance(통제)를 계층으로 분리한 아키텍처

[Strategy] Goal, Plan, Graph Design

Goal/Plan




**Agent를 사용하는 서비스 계층 :**

API Endpoint, Service Orchestration, User session 관리

**Agent**

- `create_agent` 표준 에이전트 루프
- 필요 시 Node+Edge, Tool Call as Node, Explicit State Object로 직접 조립

Execute

Reflect

**Runtime Governance Layer : Agent 실행 경계에서 정책과 컨텍스트를 주입하는 통제 계층**

- Governance : 정책 검증, PII *Personally Identifiable Information* 필터링, tool access control
- Observability : tracing, logging, metrics, audit
- Operational : retry, timeout, rate limiting, cost tracking
- Transformation : schema validation, prompt 수정, response 후처리

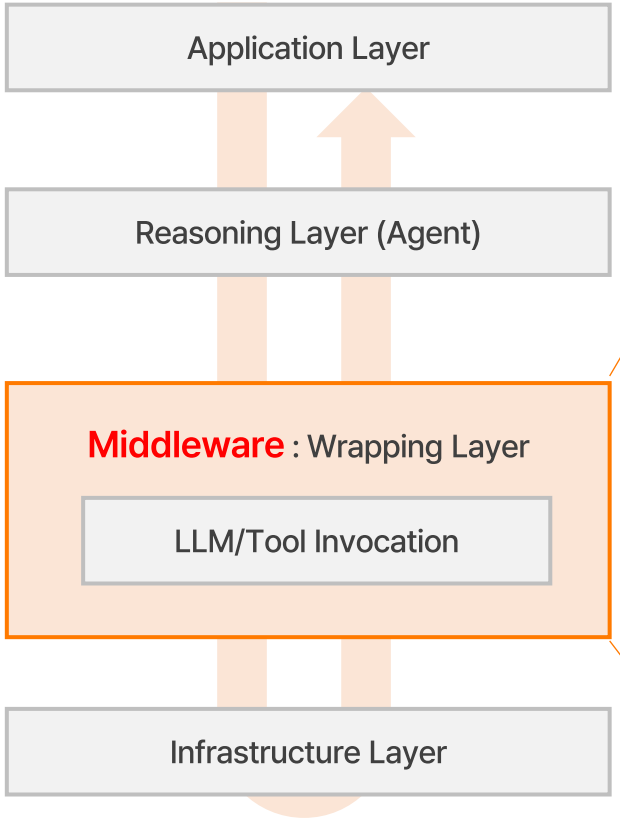
Execute
Runtime Control

- **LLM Provider**
- **Tool Runtime**

# 3. Middleware

Key Point

`create_agent` 실행 경계에 끼워, 에이전트 루프의 각 단계 전후에 정책/컨텍스트를 주입하는 Hook




❑ Node-Style : 정해진 지점에서 State를 보고 갱신. 단 호출 자체를 건드리지 못함

| | | |
|---|---|---|
| @before_agent | 에이전트 시작 전 1회 | 메모리 로드, 초기 입력 검증 |
| @before_model | 매 모델 호출 직전 | PII 마스킹, 로깅 등 |
| @after_model | 매 모델 응답 직후, 도구 실행 전 | 응답 검증, 가드레일, 출력 변환 |
| @after_agent | 에이전트 완료 후 1회 | 결과 저장, 알림, 정리 |
| @dynamic_prompt | 사용자/State 따라 프롬프트 동적 생성 | (@before_model 단축형) |

❑ Wrap-Style : 실제 호출을 감싸 실행 자체 통제

| | | |
|---|---|---|
| @wrap_model_call | 모델 호출 전체를 감쌈 | 캐싱, 재시도, 폴백, 도구 변경 등 |
| @wrap_tool_call | 도구 실행 감쌈 | 컨텍스트 주입, 특정 도구 차단 |

# @before_agent

```python
@before_agent(can_jump_to=["end"])
def content_filter(state: AgentState, runtime) -> dict[str, Any] | None:
    """결정론적 가드레일: 금지된 키워드가 포함된 요청 차단

    에이전트 실행 전에 호출되어 부적절한 콘텐츠를 사전에 필터링합니다.
    """
    # 금지된 키워드 목록
    banned_keywords = ["hack", "exploit", "malware", "해킹", "악성코드"]

    # 첫 번째 사용자 메시지 가져오기
    if not state["messages"]:
        return None

    first_message = state["messages"][0]
    # human 타입 메시지만 검사
    if first_message.type != "human":
        return None

    # 소문자로 변환하여 대소문자 구분 없이 검사
    content = first_message.content.lower()

    # 금지된 키워드 확인
    for keyword in banned_keywords:
        if keyword in content:
            # 처리 전에 실행 차단 - jump_to="end"로 에이전트 종료
            return {
                "messages": [{
                    "role": "assistant",
                    "content": "부적절한 콘텐츠가 포함된 요청은 처리할 수 없습니다. 요청을 다시 작성해주세요."
                }],
                "jump_to": "end"
            }

    # None 반환 시 정상 진행
    return None
```

```python
agent = create_agent(
    model=model,
    tools=[search_tool],
    middleware=[content_filter],
)
```

```python
# 테스트
# "hack" 키워드로 인해 처리 전에 차단
invoke_graph(
    agent,
    {"messages": [{
        "role": "user",
        "content": "How do I hack into a database?"
    }]
})
```

# 3-(1). Pseudo Code

## LangChain 0.x

Reasoning → LLM → (Policy / Logging / Retry / Cost)
Prompt

```
def handle_request(user_input):

    # 1. Reasoning (Prompt)
    prompt = build_prompt(user_input)

    # 2. LLM 호출
    response = llm.invoke(prompt)

    # 3. 운영 로직 (실행 후 처리)
    if violates_policy(response):
        response = regenerate(response)

    log_trace(response)
    track_cost(response)

    return output_parser(response)
```

## LangChain GA

Reasoning → [Middleware] → LLM
Agent

```
# Reasoning 구조 정의
agent = create_agent(
    model=model,
    tools=[retrieve_tool, python_tool, web_search_tool],

    # 미들웨어
    middleware=[
        # Prebuilt Middleware
        # 모델 호출 제한 (thread 단위, 실행 단위로 모델 호출 횟수 제한
        ModelCallLimitMiddleware(
            thread_limit = 5,
            run_limit = 2,
            exit_behavior="end",
        )
    ]
)

# 호출 흐름 동일
answer = agent.invoke(user_input)
```

# 3-(2). Prebuilt Middleware

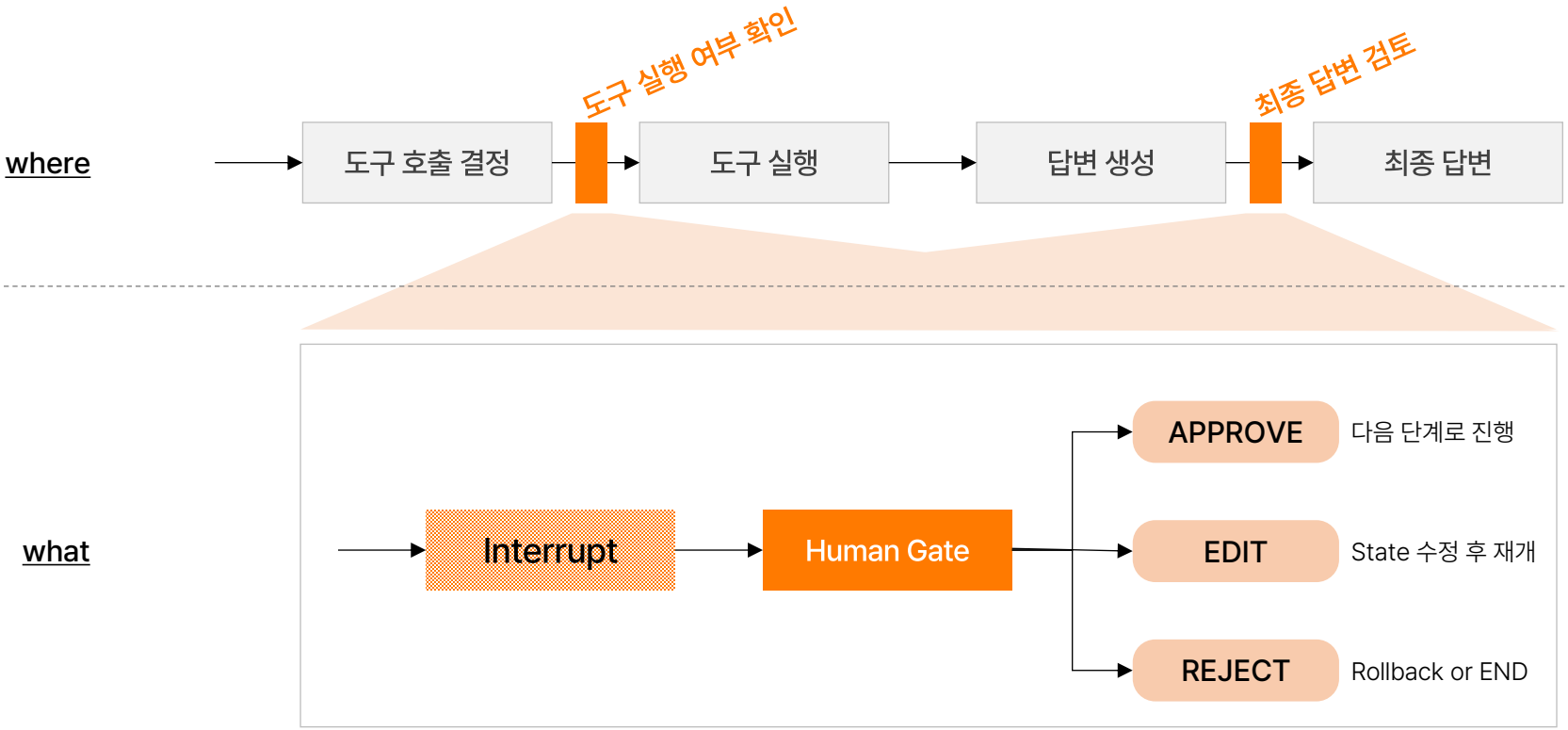
`create_agent` 실행 경계에 끼워, 에이전트 루프의 각 단계 전후에 정책/컨텍스트를 주입하는 Hook

| 구분 | Prebuilt Middleware | 내용 |
|---|---|---|
| 컨텍스트 | SummarizationMiddleware | • 히스토리가 임계치를 넘으면 오래된 메시지를 자동 요약<br>• Context overflow 방지<br>• @before_model hook |
| 보안 | PIIMiddleware | • 민감정보 – 이메일, 전화, 카드번호 등 – 를 탐지<br>• Block(오류), Redact(치환), Mask(부분가림), Hash 처리<br>• 커스텀 정규식 사용 가능 |
| 사람 개입 | HumanInTheLoopMiddleware | • 위험한 도구, 리스크 있는 Task 호출 전<br>• 실행을 멈추고 사람에게 승인, 인자 수정, 거부를 받음<br>• checkpointer + thread_id 필요 |
| 비용 | ModelCallLimitMiddleware | • 모델 호출 횟수 상한 |
| | ToolCallLimitMiddleware | • 도구 실행 횟수 제한 |
| Loop | ModelFallbackMiddleware<br>ModelRetryMiddleware | • 모델 API 호출을 재시도 핸들러로 감싸거나 대체 모델로 풀백<br>• wrap_model_call hook |
| | ToolRetryMiddleware | • 도구 실패 재시도 |

*Source : https://docs.langchain.com/oss/python/langchain/middleware/built-in*

# 3-(3). HumanInTheLoopMiddleware

Human을 State Machine의 Transition 조건으로 내재화 하는 구조



# 3-(3). HumanInTheLoopMiddleware

```python
@tool
def search_tool(query: str) -> str:
    """Search for information."""
    # 정보 검색 (안전한 작업)
    return f"Search results for: {query}"


@tool
def send_email_tool(recipient: str, subject: str, body: str) -> str:
    """Send an email. This is a sensitive operation."""
    # 이메일 전송 (민감한 작업 - 승인 필요)
    return f"Email sent to {recipient}"


@tool
def delete_database_tool(database_name: str) -> str:
    """Delete a database. This is a critical operation."""
    # 데이터베이스 삭제 (위험한 작업 - 승인 필요)
    return f"Database {database_name} deleted"
```

```python
agent = create_agent(
    model=model,
    tools=[search_tool, send_email_tool, delete_database_tool],
    middleware=[
        HumanInTheLoopMiddleware(
            interrupt_on={
                # 민감한 작업에 대해 승인 필요
                "send_email_tool": True,
                "delete_database_tool": True,
                # 안전한 작업은 자동 승인
                "search_tool": False,
            }
        ),
    ],
    checkpointer=InMemorySaver(),
)
```

- Pause는 State 저장 없이는 불가능함
- Pause/Resume = Checkpoint 기반

→ Checkpoint는 단순 대화 메모리가 아니라
실행 상태 (State Snapshot) 저장 장치

# 3-(3). HumanInTheLoopMiddleware

```python
# 테스트
# thread_id 필요 (대화 상태 추적용)
config = {"configurable": {"thread_id": "some_id"}}

# 이메일 전송 전 일시 중지됨
invoke_graph(
    agent,
    {"messages": [{
        "role": "user",
        "content": "Send an email to team@company.com with subject 'Weekly Update' and body 'Please review the latest changes."
        }]
        },
    config=config
)
```

```
================================================
🔄 Node: model 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================== Ai Message ==================================
Tool Calls:
  send_email_tool (call_8LFwT3LRaNYeMI2rOnGdk3fD)
 Call ID: call_8LFwT3LRaNYeMI2rOnGdk3fD
  Args:
    recipient: team@company.com
    subject: Weekly Update
    body: Please review the latest changes.
================================================

================================================
🔄 Node: __interrupt__ 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
Interrupt(value={'action_requests': [{'name': 'send_email_tool', 'args': {'recipient': 'team@company.com', 'subject': 'Weekly Update', 'body': 'Please review the
latest changes.'}, 'description': "Tool execution requires approval\n\nTool: send_email_tool\nArgs: {'recipient': 'team@company.com', 'subject': 'Weekly Update',
'body': 'Please review the latest changes.'}"}], 'review_configs': [{'action_name': 'send_email_tool', 'allowed_decisions': ['approve', 'edit', 'reject']}]},
id='93154c2f9f3515cd39bef6fa693e0fbd')
================================================
```

# 3-(3). HumanInTheLoopMiddleware

```python
# 승인 후 재개 - Command를 사용하여 승인 결정 전달
invoke_graph(
    agent,
    Command(resume={"decisions": [{"type": "approve"}]}),
    config=config  # 동일한 thread_id로 재개
)
```

```
================================================
🔄 Node: HumanInTheLoopMiddleware.after_model 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================ Ai Message ================================
Tool Calls:
  send_email_tool (call_8LFwT3LRaNYeMI2rOnGdk3fD)
 Call ID: call_8LFwT3LRaNYeMI2rOnGdk3fD
  Args:
    recipient: team@company.com
    subject: Weekly Update
    body: Please review the latest changes.
================================================

================================================
🔄 Node: tools 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================ Tool Message ================================
Name: send_email_tool

Email sent to team@company.com
================================================

================================================
🔄 Node: model 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================ Ai Message ================================

The email with the subject "Weekly Update" has been successfully sent to team@company.com with the message "Please review the latest changes."
================================================

================================================
🔄 Node: HumanInTheLoopMiddleware.after_model 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================================
```

# 3-(4). Custom Middleware

데코레이터 기반 : 단일 후크 미들웨어에 빠르고 간단하게 적용, 여러 후크를 적용하고자 한다면 클래스 기반으로 정의

```python
# 에이전트 시작 전
@before_agent
def log_before_agent(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """에이전트 시작 전에 메시지 수 출력"""
    print(f"에이전트를 시작하기 전에 메시지 {len(state['messages'])}개가 있습니다")
    return None


# 모델 호출 전
@before_model
def log_before_model(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """모델 호출 전에 메시지 수 출력"""
    print(f"모델을 호출하기 전에 메시지 {len(state['messages'])}개가 있습니다")
    return None


# 모델 호출 후
@after_model
def validate_output(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """모델 응답에 BLOCKED가 포함되면 차단 메시지 반환"""
    last_message = state["messages"][-1]
    if "BLOCKED" in last_message.content:
        return {
            "messages": [AIMessage("I cannot respond to that request.")],
        }
    return None


# 에이전트 종료 후
@after_agent
def log_after_agent(state: AgentState, runtime: Runtime) -> dict[str, Any] | None:
    """에이전트 종료 후 총 메시지 수 출력"""
    print(f"에이전트가 종료되었습니다. 총 메시지 수: {len(state['messages'])}개")

    return None
```

```python
# wrap_model_call 재시도 로직
@wrap_model_call
def retry_model(
    request: ModelRequest,
    handler: Callable[[ModelRequest], ModelResponse],
) -> ModelResponse:
    """모델 호출 실패 시 최대 3회 재시도"""
    for attempt in range(3):
        try:
            return handler(request)
        except Exception as e:
            if attempt == 2:
                raise
            print(f"오류 발생으로 {attempt + 1}/3 번째 재시도합니다: {e}")


# 동적 프롬프트
@dynamic_prompt
def personalized_prompt(request: ModelRequest) -> str:
    """사용자 ID에 따라 개인화된 프롬프트 생성"""
    user_id = request.runtime.context.get("user_id", "guest")
    return f"You are a helpful assistant for user {user_id}. Greeting with use
```

```python
agent = create_agent(
    model=model,
    middleware=[
        log_before_model,
        validate_output,
        retry_model,
        personalized_prompt,
        log_before_agent,
        log_after_agent,
    ],
    tools=[get_weather],
)
```

# 3-(4). Custom Middleware

```python
# 테스트
invoke_graph(
    agent,
    inputs={"messages": [{"role": "user", "content": "안녕하세요. SKALA 김영희 입니다."}]},
    context={"user_id": "김영희"},
    config=RunnableConfig(),
)
```

```
================================================
🔄 Node: log_before_agent.before_agent 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================================
모델을 호출하기 전에 메시지 1개가 있습니다

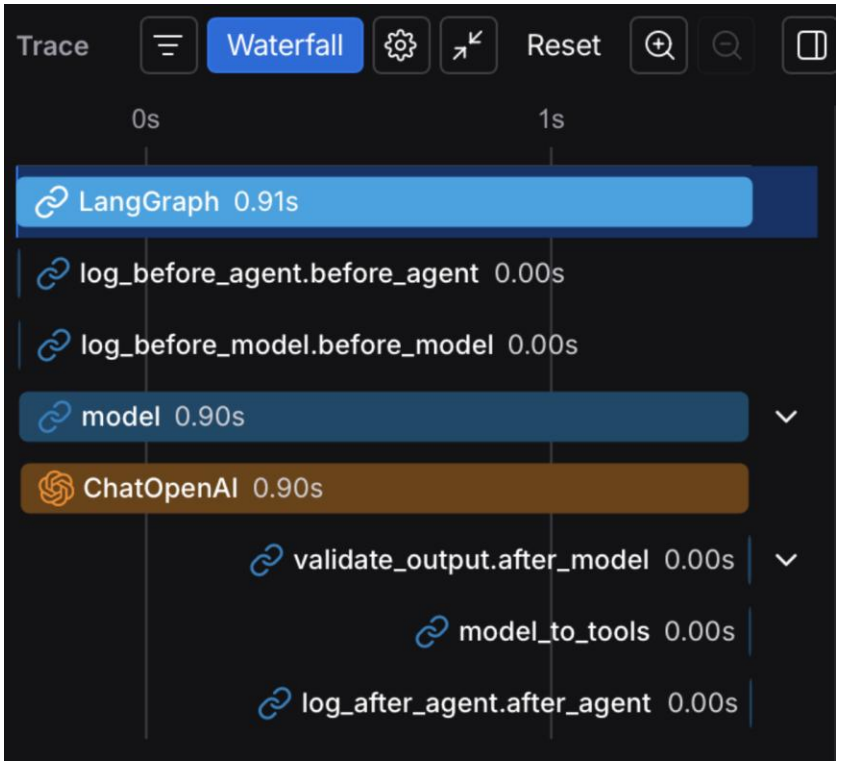
================================================
🔄 Node: log_before_model.before_model 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================================

================================================
🔄 Node: model 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================== Ai Message ==================================

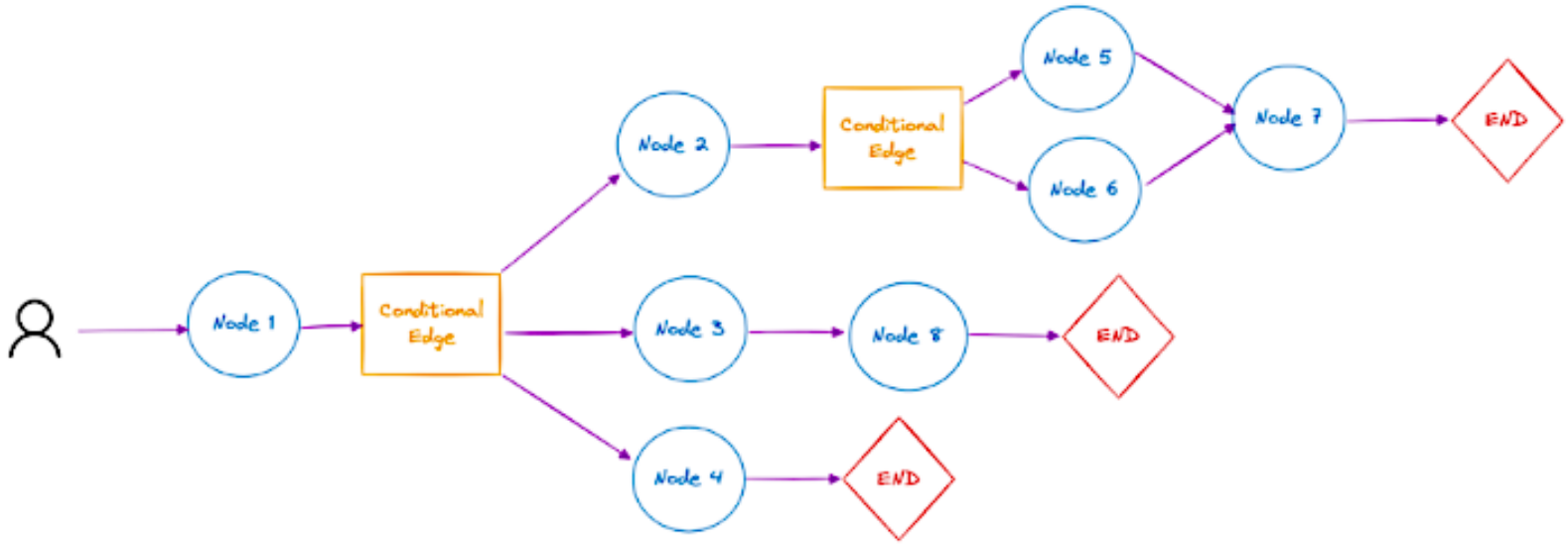
안녕하세요, 김영희님! 어떻게 도와드릴까요?
================================================

================================================
🔄 Node: validate_output.after_model 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================================
에이전트가 종료되었습니다. 총 메시지 수: 2개

================================================
🔄 Node: log_after_agent.after_agent 🔄
- - - - - - - - - - - - - - - - - - - - - - - -
================================================
```



# 2. LangGraph 구성 요소




| 구성 요소 | 목적 | 역할 |
|---|---|---|
| Node | 어떤 작업(task)을 수행할지 정의 | • 특정 로직 수행 또는 상태 업데이트(입력으로 현재 상태를 받고, 업데이트된 상태 반환) |
| Edge | 다음으로 실행할 동작 정의 | • 워크플로우의 흐름 제어<br>• 조건부 라우팅, 다음에 실행할 노드 결정 |
| Conditional Edge | 조건에 따른 분기 처리 | |
| State | 현재의 상태값을 저장 및 전달 | • 전체 워크플로우의 컨텍스트 유지, 노드 간 정보 공유 |

*Source : https://www.ionio.ai/blog/a-comprehensive-guide-about-langgraph-code-included*

# 1. State

Key Point

LangGraph 전체 흐름의 기억 저장소이며, 노드 간 연결성과 판단을 위한 핵심 데이터 구조

- 노드와 노드 간에 정보를 전달할 때 상태(State) 객체에 담아 전달 → 데이터 공유
- 에이전트가 현재까지 수행한 작업의 결과를 담고, 이후 판단에 기억처럼 사용 → Context 유지, 흐름 유지

## LangGraph > State

What to Design:

- 어떤 값을 저장할지 정의 : 질문, 응답, 평가 등 단계별 필요한 항목
- TypedDict :
  - State 스키마를 정의하는 수단 → 구조 선언
  - Python dict에 Typehinting을 추가한 방식으로 명세
- Reducer : 자동으로 리스트에 메시지 추가
  - (활용) 여러 값을 하나로 합치는(fold) 연산
  - (코딩) 필드 업데이트를 어떻게 병합할지 정하는 함수 (fold 연산)
  - 이전 대화 내용에 새 메시지를 붙여서 하나의 대화 흐름으로 만들어 줌
- State 업데이트 방식
  - Reducer 미적용 (기본) : Overwrite
  - Reducer 적용 : 병합/누적
    (메시지: `add_messages`, 일반 리스트: `operator.add`)

Pitfalls :

- 노드 간 상태 공유를 위해서는 키 이름이 정확히 일치해야 함
- State에 너무 많은 내용을 저장하면 흐름이 복잡해지고 디버깅 어려움

```python
import operator
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages


# State 정의
class GraphState(TypedDict):
    messages: Annotated[list, add_messages]  # 메시지 (누적)
    question: Annotated[list, operator.add]      # 질문 (누적 관리 필요 시)
    context: str                                 # 문서 검색 결과
    answer: str                                  # 답변
    relevance: str                               # 답변 관련성 (Good/Bad)
```

※ messages: Annotated[list, add_messages]
- messages : LLM과 주고 받는 대화 내용
- (left) list : 기본 메시지 리스트
- (right) add_messages : 병합할 메시지 리스트 또는 단일 메시지

# 1-(1). Overwrite

Reducer 없는 필드는 노드를 거치며 덮어쓰기로 정보 갱신

```
class GraphState(TypedDict):
    time: int
    name: str
    model: str
```

| | KEY | VALUE |
|---|---|---|
| Node 1 초기화 | time | 1 |
| | name | |
| | model | GPT |

| | KEY | VALUE |
|---|---|---|
| Node 2 이름 입력 | time | 2 |
| | name | 철수 |
| | model | GPT |

| | KEY | VALUE |
|---|---|---|
| Node 3 이름 확인 | time | 3 |
| | name | 철수 |
| | model | GPT |

| | KEY | VALUE |
|---|---|---|
| Node 4 모델 변경 | time | 4 |
| | name | 철수 |
| | model | Claude |

- 각 노드에서는 특정 KEY의 값을 새로 설정하며, 이는 기존 값을 덮어쓰기 함 (선택적 덮어쓰기)
- 필요한 Key만 갱신되고 나머지는 유지
- 위와 같은 방식으로 State가 관리되기 때문에 이전 정보도 유지하면서 새로운 작업을 추가할 수 있음 (필요한 부분만 갱신 !!)

# 1-(2). Reducer

Reducer 적용 필드는 정보 병학/누적 하여 관리

```
class GraphState(TypedDict):
    steps     : Annotated[list, operator.add]
    question  : str
    messages  : Annotated[list, add_messages]
```

| Node 1 질문 입력 | KEY | VALUE |
|---|---|---|
| | steps | ["ask"] |
| | question | Q1 |
| | messages | [Q1] |

| Node 2 검색 | KEY | VALUE |
|---|---|---|
| | steps | ["ask", "search"] |
| | question | Q1 |
| | messages | [Q1] |

| Node 3 답변 | KEY | VALUE |
|---|---|---|
| | steps | ["ask", "search", "answer"] |
| | question | Q1 |
| | messages | [Q1, A1] |

- 하나의 State 안에서도 필드 마다 업데이트 방식이 Reducer 적용 여부에 따라 다름
- `add_messages` : 메시지 전용
- `operator.add` : 범용 리스트 결합

  (e.g.)
  - `operator.add(3,5)` = 3+5
  - `operator.add([1, 2], [3]) = [1, 2, 3]

※ steps : Annotated[list, operator.add]
: steps 노드는 반드시 list를 반환해야 함 → 초기값 필요
단 `add_messages`는 빈 리스트에서 시작 가능

Key Point

# 1-(3). State Implementation

State = 실행 흐름을 통제하는 시스템 레벨 데이터 구조

저장 그릇으로만 보지 말고, 노드 간 인터페이스로 봐야 함!!

## 1 State는 데이터 컨테이너가 아니라 Read/Write Contract

- 노드는 State 전체를 알 필요 없음
- 노드는 자신이 읽고/쓸 KEY만 알아야 함 → State Ownership
- State는 Graph의 공용 API

## 2 State는 서비스 관점에서도 고려되어야 함

- 이전 값 보존이 필요한가?
- 재현 가능한 실행인가?
- Trace 가능한가?

→ Overwrite vs append 전략 고려

## 3 State는 실패를 고려해야 함 (Error-aware)

- Observability, Monitoring 위한 관리 항목
- Status field : SUCCESS, FAILED, INTERRUPTED 등
- retry_count, error_message 등

## 4 State는 확장 가능해야 함

- Optional Field 사용
- Nested 구조 사용
- Evaluation은 dict로 감싸기 등

# 1-(4). Single State

단일 State, 순차/독립 처리

class SharedState(TypedDict, total=False):

**TURN**

**LOG**

messages: Annotated[list, add_messages]

**Current**

question: str
context: str 대용량이면 원문 대신 외부 store의 참조ID 저장으로 설계
answer: str

**Control**

current_agent: str
plan: str

**Loop**

step_count: Annotated[int, operator.add]
max_steps: int

**Error-aware**

status: str SUCCESS | FAILED | INTERRUPTED | ...
error_count: Annotated[int, operator.add]
last_error : Optional[dict[str, Any]] {"node", "type", "message", "ts"}

**Deliverables**

results: Annotated[list, keep_last_n] {"agent", "content", "ts"}

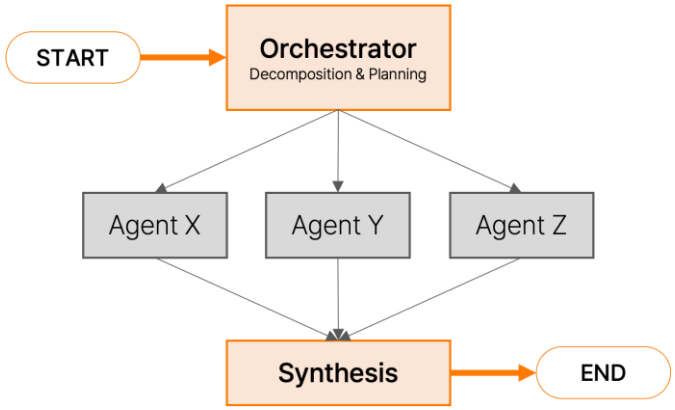
```
MAX_RESULTS = 10
def keep_last_n(current: list, new: list) -> list:
    combined = (current or []) + (new or [])
    return combined[-MAX_ RESULTS :]
```

# 1-(5). Layered State

## A. Orchestrator – Workers

※ `results` ≠ Synthesis
- results 키는 에이전트별 `task_output`을 취합하는 것까지!!
- 취합과 결과 생성은 다른 업무로 구분

※ Synthesis 결과
- 해당 노드는 최종 결과(보고서 등) 생성
- 해당 결과는 별도DB에 저장하고, State는 참조 정보만 관리
  ← State는 인터페이스이지 컨테이너가 아님




class OrchestratorState(TypedDict, total=False):

# 전체 흐름 제어
plan: str
status: str SUCCESS | FAILED | INTERRUPTED

# 에이전트 결과 취합
results: Annotated[list, merge_results]
에이전트 별로 {"agent": str, "output": Any, "status": str, "ts": str}

# 에이전트 최종 결과
report_uri: str

# 상위 레벨 에러 취합
errors: Annotated[list, operator.add]
에이전트 별로 {"agent": str, "error": dict, "ts":str}

class WorkerState(TypedDict, total=False):

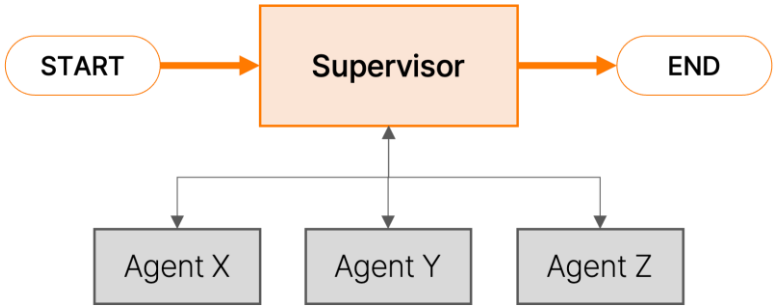
# 해당 에이전트 내부 실행 상태
messages: Annotated[list, add_messages]
current_step: str
step_count: Annotated[int, operator.add]
max_steps: int

# 에이전트 실행 타스크
task_input: str
task_output: str

# 에이전트 에러 내역
status: str
error_count: Annotated[int, operator.add]
last_error : Optional[dict[str, Any]]
노드 별로 {"node", "type", "message", "ts"}

# 1-(5). Layered State

## B. Supervisor




```
class SupervisorState(TypedDict, total=False):

    # 라우팅 제어
    next: str       Next Worker | FINISH

    # 루프 종료 제어
    step_count: Annotated[int, operator.add]
    max_steps : int
    status : str    SUCCESS | FAILED | INTERRUPTED

    # 에이전트 결과 취합 : 순차 호출 → 이력 누적
    results: Annotated[list, operator.add]  시간순 append
    errors: Annotated[list, operator.add]

    # 에이전트 최종 결과
    report_uri: str
```

**※ `next`**

- Supervisor가 매 턴 마다 "다음 담당"을 정함 → `next`가 중심 필드
- 보통 plan을 State에 담지는 않지만, 전체 목표를 기억시키야 한다면 포함 (e.g., `plan : str`)

**※ 루프 종료 제어**

- Orchestrator와 달리, Supervisor는 상/하위 왕복 루프라 무한 루프 위험 잔존
- `step_count`, `max_steps`가 상위 State에 필수로 반영

**※ `results`**

- Orchestrator는 워커가 동시에 같은 Key를 사용하기 때문에 유실 방지 위한 커스텀 리듀서 필수
- Supervisor는 한 번에 한 워커 호출이라, `operator.add`로 충분

# 2. Node

LangGraph의 각 단계를 수행하는 작업 유닛 (함수 레벨)

- LangGraph 내에서 실제로 어떤 행동(작업)을 수행하는 단위로, 각 노드는 하나의 함수로 정의
- State를 입력 받아, 변경된 키만 부분 업데이트(dict)를 반환 → LangGraph가 State에 병합

## LangGraph > Node

What to Design:

- 어떤 기능을 수행할지 함수로 정의 : (eg) 질문 분석, 검색, 답변 생성, 평가 등
- 노드는 state를 입력 받고, 변경된 키만 담은 dict 반환
  (State 전체를 반환하지 않음)
- `add_node` : 이전에 정의한 함수를 Graph에 추가
- 내부에서 LLM, API, DB 호출 등 다양한 작업 수행 가능

Pitfalls :

- 반환값은 부분 업데이트 dict : State 스키마에 정의된 키 이름과 정확히 일치해야 병합됨
- 리듀서가 없는 키는 덮어쓰기(last-write-wins), 있는 키는 누적/병합 처리
- 하나의 노드에 여러 책임(e.g., 검색+생성+분기)을 몰지 않음
  - 디버깅이 어려워지므로 기능은 단순하게
  - 단 라우팅 판단을 노드에서 내리는 것 자체는 정상 패턴

```python
# Node 정의 : 문서 검색
def retrieve_document( state: GraphState ) → dict :

    # Question에 대한 문서 검색을 retriever로 수행
    retrieved_docs = pdf_retriever.invoke(state["question"])

    # 검색된 문서를 context 키에 저장
    return {"context": format_docs(retrieved_docs)}
```

| NODE A | |
|---|---|
| KEY | VALUE |
| context | |
| question | 질문1 |
| answer | |
| score | |

| NODE B | |
|---|---|
| KEY | VALUE |
| context | 문서1 |
| question | 질문1 |
| answer | |
| score | |

# 3. Edge

Edge, Conditional Edge : 노드 간의 흐름을 정의하는 연결선

- Edge : A 다음에 B와 같이 순서를 연결하는 연결선
- Conditional Edge : 신호등을 보고 갈림길을 선택하는 로직과 같이 상황을 보고 다음 노드를 판단/분기

## LangGraph > Edge

What to Design:

- Edge : 단순 연결 흐름 (`add_edge`)
- Conditional Edge : 조건 따라 다음 노드 분기 (`add_conditional_edges`)
- 조건 판단 함수 : 분기를 위해서는 State를 판별하는 함수 필요
  → 반환값 = 다음 노드 키
- 흐름 설계 : 평가, 피드백, 사용자 개입, 반복 등을 위한 설계 전략 및 반영

Pitfalls :

- 조건 판단 함수의 반환값이 매핑 딕셔너리의 키와 정확히 일치해야 연결됨
- Command 사용 시, 갈 수 있는 노드를 타입 힌트로 선언해야 그래프가 경로를 인식함 (`Command[Literal[...]]`)
- 반복/재귀 흐름은 exit 조건을 명확히 정의해야 함
  → 그렇지 않으면, 무한루프 발생 → 토큰 낭비

```python
from langgraph.graph import StateGraph, START, END

# 시작점 정의
workflow.add_edge(START, "retrieve")

# Node 연결 (순차 흐름)
workflow.add_edge("retrieve", "llm_answer")
workflow.add_edge("llm_answer", "relevance_check")

# Conditional Edge
workflow.add_conditional_edges(
    "relevance_check",
    is_relevant,
    {
        "grounded" : END,                  # 관련성이 있으면 종료
        "notGrounded" : "llm_answer",      # 관련성 없으면 다시 답변 생성
        "notSure" : "llm_answer",          # 모호하면 다시 답변 생성
    },
)
```

# [참고] Command

Conditional Edge 대안 – 노드가 "다음 어디로 갈지"까지 스스로 결정

- Conditional Edge : 노드는 상태만 바꾸고, 어디로 갈지는 엣지(노드 바깥 판별 함수)가 결정 → 작업과 분기가 분리
- Command : 노드가 상태도 바꾸고, 다음 노드도 스스로 지정 → 작업 + 분기

| | Conditional Edge | Command |
|---|---|---|
| 분기 결정 위치 | Edge (노드 바깥) | 노드 안 |
| State 갱신 + 분기 | 2단계로 분리 | 동시 진행 |
| 흐름 가시성 | 높음 – Edge가 명시적 | 낮음 – 노드 내부에 숨음 |
| 대표 용도 | • 분기 규칙 복잡하거나 정형 워크플로우 기반(명시적) | • 분기에 필요한 값이 노드가 방금 계산한 결과에 있을 때<br>• Handoff를 한 번에 처리할 때 : 내 결과 여기, 다음은 저 에이전트로 |

```
def relevance_check(state: GraphState) → dict :
    return {"score": grade(state["context"], state["answer"])}

def is_relevant(state: GraphState) → str:
    return "grounded" if state["score"] == "grounded" else "notGrounded"

workflow.add_conditional_edges(
    "relevance_check", is_relevant,
    {"grounded": END, "notGrounded": "llm_answer"}
)
```

```
from langgraph.types import Command
from typing import Literal

def relevance_check(state: GraphState) → Command[Literal["llm_answer", "__end__"]]:
    verdict = grade(state["context"], state["answer"])
    goto = END if verdict == "grounded" else "llm_answer"
    return Command(update={"score": verdict}, goto=goto)
```

- **Literal["A", B"]**
  - 이 노드가 갈 수 있는 목적지 목록을 사전에 선언하는 표지판
  - 반환 타입 힌트에 적어두면 LangGraph가 "이 노드는 저기로 이어질 수 있구나"를 인식하여 그래프 연결
- **update={"score": verdict}**
  - State를 어떻게 바꿀지. 일반 노드가 반환하는 부분 업데이트 dict과 같은 역할
  - 여기에서는 score키에 판정 결과 기록, 리듀서를 거쳐 State에 병합됨
- **goto=goto**
  - 다음에 어느 노드로 갈지
  - Conditional edge의 분기 역할을 노드 안으로 가져온 부분

# 4. Node vs Agent

Key Point

| | 어떻게(HOW) 수행할지 정의 | 무엇을(WHAT), 언제(WHEN) 수행할지 결정 |
|---|---|---|
| | **Node** | **Agent** |
| | ▪ 에이전트 시스템 내에서 특정 기능을 수행하는 가장 작은 독립적인 작업 단위 (레고 블록의 개별 조각) | ▪ 특정 목표를 달성하기 위해 상황을 이해하고 계획을 수립하고 행동을 실행<br>▪ 여러 노드들을 조합하고 제어하여 복잡한 문제 해결 (레고 블럭으로 조립된 로봇) |
| | ▪ 역할 :<br>○ 단일 기능 수행: 문서 검색, 텍스트 요약, 이미지 분석, 데이터베이스 조회, 답변 생성 등 하나의 명확하고 독립적인 작업 수행<br>○ 함수 형태로 정의: 특정 입력(State)을 받아 정해진 로직을 처리한 후, 결과(State)를 반환하는 함수 형태로 정의 | ▪ 역할 :<br>○ 목표 설정 및 달성: 사용자의 요청이나 환경 변화에 따라 동적으로 목표를 설정하고, 이 목표를 달성하기 위한 일련의 작업 수행<br>○ Node Orchestration: 어떤 노드를 어떤 순서로 실행할지 결정하고, 각 노드의 출력을 다음 노드의 입력으로 연결하는 등 전체 작업 흐름 제어<br>○ 의사결정: 현재 상태를 바탕으로 다음에 어떤 행동(노드 실행)이 필요한지 판단하고, 필요에 따라 다양한 노드들을 유연하게 활용 |
| | ▪ 특징 :<br>○ 주어진 입력에 따라 정의된 작업만 수행<br>○ Deterministic, 사전 정의된 엣지에 의해 제어권 결정<br>○ 노드 실패는 해당 단계에 국소적으로 영향 | ▪ 특징 :<br>○ 단순 노드들의 집합이 아니라, 노드들을 활용하여 지능적인 행동을 만듦<br>○ Agentic, 런타임에 LLM이 제어 판단<br>○ 에이전트 실패 파급 범위가 넓음 |
| | ▪ 예시 :<br>○ 사용자의 질문을 분석하여 키워드 추출하는 노드 (QuestionAnalyzerNode)<br>○ 추출된 키워드로 문서를 검색하는 노드 (DocumentSearchNode)<br>○ 검색된 문서와 질문을 바탕으로 답변을 생성하는 노드 (AnswerGenerationNode) | ▪ 예시 :<br>○ 문서 기반 질의응답 에이전트: 사용자의 질문을 받으면<br>→ QuestionAnalyzerNode로 질문을 분석하고,<br>→ DocumentSearchNode로 관련 문서를 찾고,<br>→ AnswerGenerationNode로 최종 답변을 생성하여<br>사용자에게 제공하는 전체 과정을 수행하는 에이전트 |

# [참고] Node와 Agent는 무조건 1:1 맵핑이 아니다

- Node : 워크플로우를 구성하는 공간/단계의 단위
- Agent : 그 공간 안에서 판단을 내리는 Cognitive Actor
- 하나의 Agent가 여러 Node에 걸쳐 사고(Plan → Act → Reflect) 할 수도 있음

[Case 1] Node = Agent
- 가장 간단한 구조 (기본 구성)
- 하나의 Node 안에서 LLM이 시스템 프롬프트 읽고, 도구 선택하고, 답변까지 생성
- (비유) 만능 작업자가 하나의 방(Node) 안에서 모든 업무를 다 처리하는 경우

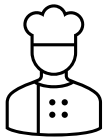
[Case 2] 여러 개의 Node = 1개의 Agent
- 에이전트의 인지 과정을 여러 노드로 쪼갠 형태 (e.g., Planning Node → Execution Node → Review Node)
- 쪼개진 여러 개의 노드가 합쳐져서 하나의 에이전트 자아를 구성
- Node는 에이전트 레벨 보다 더 작은 하위 단위(sub-step) 일 수 있음

[Case 3] Node = Tool
- 지능(LLM) 없이 코드만 실행되는 노드
- Tool을 실행하는 전용 노드(ToolNode)나, 입력된 텍스트의 오탈자를 수정하는 노드 등
- (비유) 사람(Agent) 없이 세척기만 덩그러니 놓여 있는 방(Node)
- Node는 Agent 없이도 존재할 수 있는 단순 인프라 일 수 있음

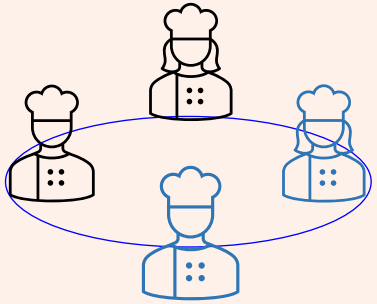
# 4. Parallel Execution

병렬 처리 = 품질이 아니라 효율을 위한 것

## 한정식 코스A가 주문되면...



1명의 요리사가 모든 요리를 순서대로 진행



여러 명의 요리사가
각자 맡은 부분에서 동시에 요리를 진행

**→ 빠르고 효율적**

- 문제 상황 :
  - 단일 에이전트가 여러 작업을 순차적으로 처리하면 응답이 느리고 비효율적
  - LLM 기반 시스템에서는 Latency가 누적되어 전체 응답 시간이 길어짐
- 개념 :
  - 여러 작업(또는 여러 에이전트)를 동시에 실행하여
  - 전체 처리 시간을 단축하고 시스템의 처리 효율을 높이는 방식

    **→ 품질 보다 효율 개선**
- 주요 유형 :
  - Task-level Parallel
    - 하나의 에이전트가 여러 Task를 동시에 실행
    - (e.g.) 같은 문서를 요약+키워드추출+번역 업무를 동시 실행
  - Agent-level Parallel
    - 여러 에이전트가 독립적으로 병렬 실행 후 결과 통합
    - (e.g.) [재무분석 – 마케팅분석 – 기술평가] → 리포트 작성 에이전트

# 1. 주요 설계 패턴

## Fan-out, Fan-in

- Fan-out : 상위 노드/에이전트가 여러 하위 작업을 동시에 분기시켜 실행
- Fan-in : 분기된 작업들의 결과를 하나로 모아 통합하는 단계

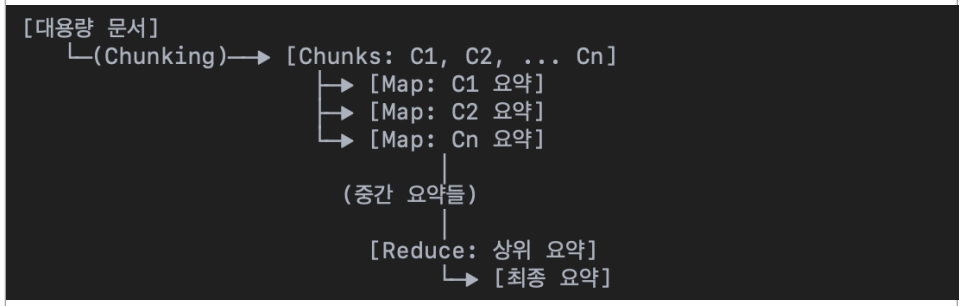
상위 노드가
여러 하위 작업으로 분기해 동시 실행하고,
결과를 하나로 통합하는 패턴




## Map-Reduce

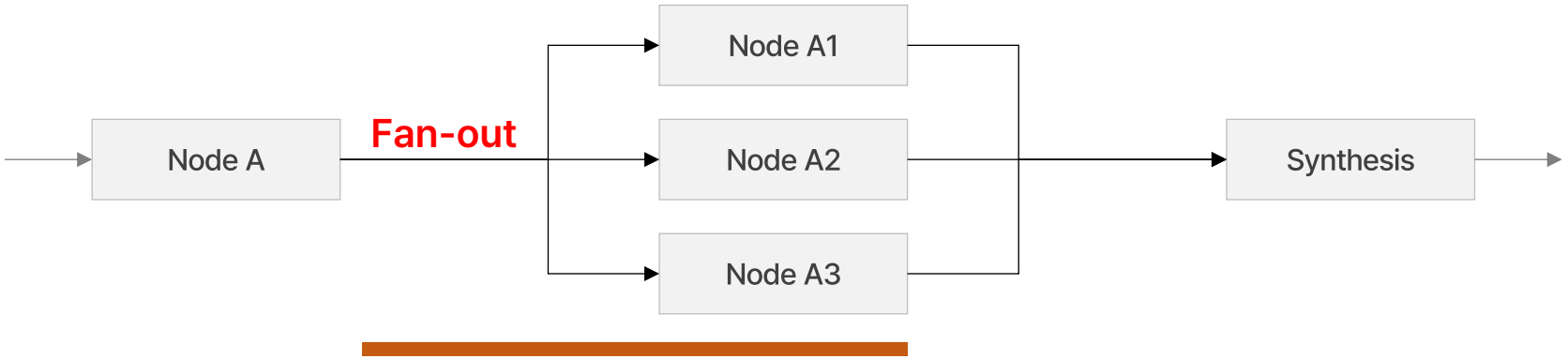
- Map : 큰 입력을 작은 조각(Chunk)으로 나누어 병렬 처리
- Shuffle/Combine : (Optional) 중간 결과 취합 (재배열, 부분 결합 등)
- Reduce : 중간 결과를 최종 산출물로 요약/집계/선정

큰 입력을
조각으로 나눠 병렬 처리(Map) 한 뒤,
결과를 축약(Reduce)하여 합치는 패턴



# 2-(1). Fan-out 설계 고려사항

Production Fan-out/Fan-in은 병렬화가 아니라, 정합성·실패·동시성을 설계하고 통제하는 것




### 1 Reducer – 병렬 정합성

여러 병렬 노드가 같은 State 키에 쓸 때,
Reducer가 없을 경우 마지막 작성 시 나머지 내용을 덮어써 데이터 유실 발생

- **공유되는 키에는 Reducer 구성**
- **각 병렬 노드는 담당하는 키만 업데이트 처리**
  ← State 전체 업데이트 되지 않도록

### 2 부분 실패 처리

한 브랜치가 실패하면
성공한 브랜치 결과도 State에 저장되지 않음

- **Checkpointer 필수**
  : 성공한 노드 결과는 내부적으로 저장되므로,
  그래프 재개 시 실패한 브랜치만 재시도하면 됨

### 3 Concurrency 제어

병렬 처리는 지연 시간을 줄여주지만
순간 리소스 사용량을 키워 Rate limit 문제됨

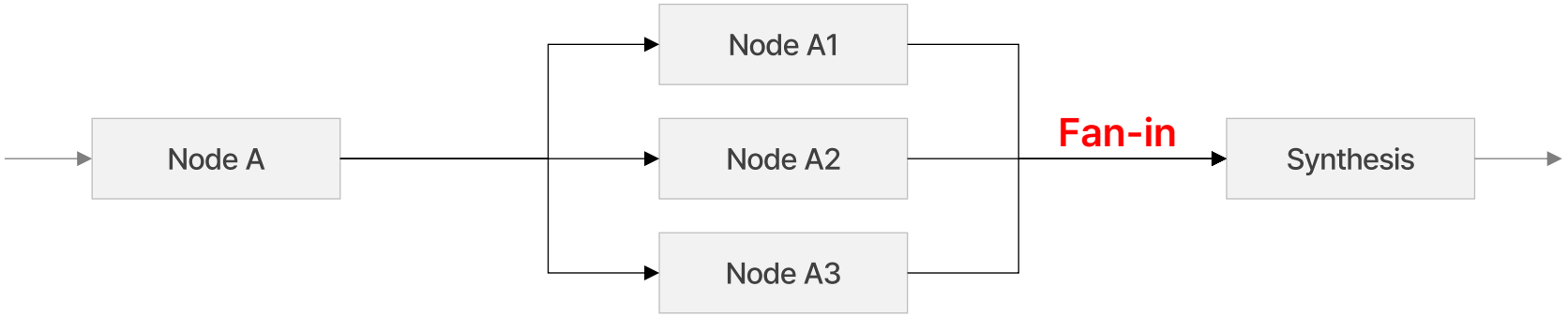
- **max_concurrency 통해 동시 태스크 수 제한 :**
  {"configurable": {"max_concurrency": 10}}
- **외부 API 호출 한도 고려하여 실행 건수 제한**

※ Remark
- Node : 하나의 작업을 수행하는 실행 단위(함수)
- Branch : Fan-out으로 갈라져 병렬 실행되고, Fan-in에서 합쳐지는 하나의 경로.
  (e.g.) 병렬 노드(A1, A2, A3)가 각각 fan-out부터 fan-in까지 이어지는 하나의 브랜치에 해당

*Source : https://docs.langchain.com/oss/python/langgraph/use-graph-api*

# 2-(2). Fan-in 설계 고려사항

Production Fan-out/Fan-in은 병렬화가 아니라, 정합성·실패·동시성을 설계하고 통제하는 것




### 4 부분 실패 & 재처리

모든 병렬 노드가 완료되어야 다음으로 넘어가는데,
한 브랜치가 응답 없이 매달리면 전체가 멈춤

- 브랜치별 타임아웃과 처리 정의 – 부분 결과 진행, 실패 처리
- 재시도 시 부작용 중복이 없도록 멱등성(idempotency) 확보
- 일부만 성공했을 때의 정책 정의 (개수, 핵심 타스크만 달성, 등

### 5 결과 통합

병렬 노드의 완료 순서는 비결정적이라,
도착 순서에 의존한 통합은 실행마다 결과가 달라짐

- 통합 로직은 도착 순서에 의존적이지 않도록 설계
- 모든 태스크 완료까지 통합을 미루는 지연 실행(defer) 적용
- 중복 여부 검토 또는 캐시 기반 설계

### 6 관측성

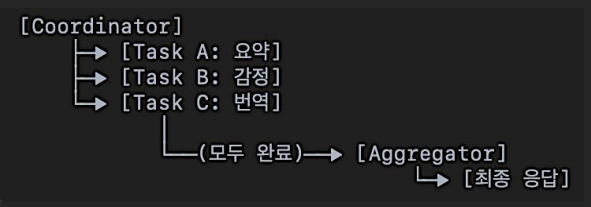
여러 브랜치가 동시에 실행되어도
가장 느린 브랜치가 전체 지연 결정하는데, 이를 관측하지 않으면 병목을 특정할 수 없음

- 실행/실패를 브랜치 단위로 추적 가능하도록 State 설계
- Barrier 특성을 관측하여 병목 브랜치 식별

※ Remark
- Node : 하나의 작업을 수행하는 실행 단위(함수)
- Branch : Fan-Out으로 갈라져 병렬 실행되고, Fan-in에서 합쳐지는 하나의 경로.
  (e.g.) 병렬 노드(A1, A2, A3)가 각각 fan-out부터 fan-in까지 이어지는 하나의 브랜치에 해당

*Source : https://docs.langchain.com/oss/python/langgraph/use-graph-api*

# 4-(3). LangChain




```python
llm = ChatOpenAI(model="gpt-4o-mini")  # 예시

# (A) 요약 체인
summary_prompt = ChatPromptTemplate.from_template(
    "다음 텍스트를 3문장으로 요약해 주세요:\n\n{text}"
)
summarize_chain = summary_prompt | llm | (lambda x: x.content)

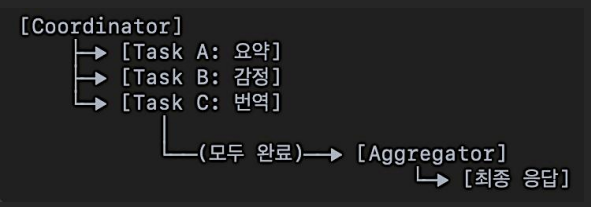
# (B) 감정분석 체인
sentiment_prompt = ChatPromptTemplate.from_template(
    "다음 텍스트의 전반적 감정을 하나 선택: positive/neutral/negative.\n텍스트:\n{text}"
)
# 결과를 dict로 래핑해 downstream에서 다루기 쉽게
sentiment_chain = sentiment_prompt | llm | (lambda x: {"label": x.content.strip()})

# (C) 번역 체인 (KO → EN)
translation_prompt = ChatPromptTemplate.from_template(
    "Translate the following Korean text to concise English:\n\n{text}"
)
translation_chain = translation_prompt | llm | (lambda x: x.content)

# 병렬 실행기
parallel_tasks = RunnableParallel({
    "summary": summarize_chain,
    "sentiment": sentiment_chain,
    "translation": translation_chain
})

# 실행
result = parallel_tasks.invoke({"text": "전기차 보조금 축소 소식 이후 소비자 반응이 혼재되어..."})
```

# 4-(3). LangChain




```python
def aggregate(report_dict):
    return (
        "=== Parallel Report ===\n"
        f"- Summary:\n{report_dict['summary']}\n\n"
        f"- Sentiment: {report_dict['sentiment']['label']}\n\n"
        f"- EN Translation:\n{report_dict['translation']}\n"
    )

final_output = aggregate(result)
```

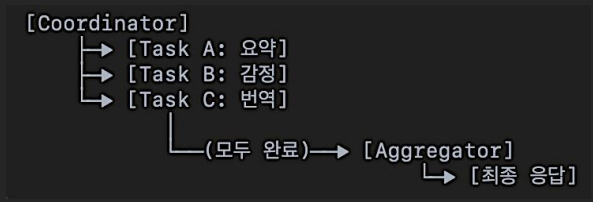
```python
import asyncio

async def run_async():
    # 병렬 실행 (Fan-out)
    out = await parallel_tasks.ainvoke({
        "text": "동시에 여러 작업을 돌려 봅시다."
    })
    # 결과 통합 (Fan-in)
    final_output = aggregate(out)
    return final_output

# 실행
final_report = asyncio.run(run_async())
print(final_report)
```

Jupyter에서는
await run_async() 사용

# 4-(4). LangGraph




```python
# ----- 1) 상태 정의: 병렬 노드가 각자 채울 필드를 준비
class AppState(TypedDict, total=False):
    text: str
    summary: str
    sentiment: str
    translation: str
    report: str

# ----- 2) LLM & 프롬프트
llm = ChatOpenAI(model="gpt-4o-mini")

summary_prompt = ChatPromptTemplate.from_template(
    "다음 텍스트를 3문장으로 요약해 주세요:\n\n{text}"
)
sentiment_prompt = ChatPromptTemplate.from_template(
    "다음 텍스트의 전반적 감정(positive/neutral/negative)만 출력:\n\n{
)
translation_prompt = ChatPromptTemplate.from_template(
    "Translate the following Korean text to concise English:\r
)
```

부분 업데이트를 허용하는 타입 정의

```python
# ----- 3) 노드 정의 (입력: state -> 출력: 업데이트 dict)
def coordinator_node(state: AppState) -> AppState:
    # Fan-out의 출발점이므로 실제 변환은 하지 않고 그대로 전달
    return state

def summary_node(state: AppState) -> AppState:
    msg = summary_prompt.format_messages(text=state["text"])
    out = llm.invoke(msg).content
    return {"summary": out}

def sentiment_node(state: AppState) -> AppState:
    msg = sentiment_prompt.format_messages(text=state["text"])
    out = llm.invoke(msg).content.strip()
    return {"sentiment": out}

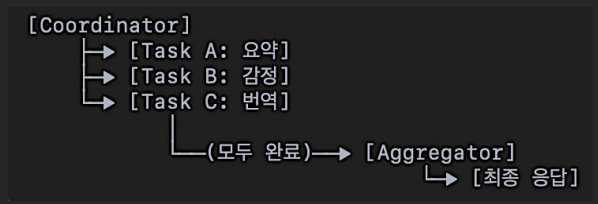
def translation_node(state: AppState) -> AppState:
    msg = translation_prompt.format_messages(text=state["text"])
    out = llm.invoke(msg).content
    return {"translation": out}

def aggregate_node(state: AppState) -> AppState:
    report = (
        "=== Parallel Report ===\n"
        f"- Summary:\n{state.get('summary','(missing)')}\n\n"
        f"- Sentiment: {state.get('sentiment','(missing)')}\n\n"
        f"- EN Translation:\n{state.get('translation','(missing)')}\n"
    )
    return {"report": report}
```

모든 upstream 노드가 완료되어야 실행

값이 없을 때 대비 (defensive)

# 4-(4). LangGraph




```python
# ----- 4) 그래프 구성: coordinator → (summary, sentiment, translation) → aggregate → END
graph = StateGraph(AppState)

graph.add_node("Coordinator", coordinator_node)
graph.add_node("Summary", summary_node)
graph.add_node("Sentiment", sentiment_node)
graph.add_node("Translation", translation_node)
graph.add_node("Aggregate", aggregate_node)

# 시작 노드
graph.set_entry_point("Coordinator")

# 병렬 엣지(Fan-out): Coordinator에서 세 노드로 동시에 분기
graph.add_edge("Coordinator", "Summary")
graph.add_edge("Coordinator", "Sentiment")
graph.add_edge("Coordinator", "Translation")

# Fan-in: 세 노드가 끝나면 Aggregate로 수렴
# LangGraph는 의존성 없는 노드들을 자동 병렬 실행하고, 모든 업데이트가 병합되면 다음 엣지를 진행합니다.
graph.add_edge("Summary", "Aggregate")
graph.add_edge("Sentiment", "Aggregate")
graph.add_edge("Translation", "Aggregate")

# 종료
graph.add_edge("Aggregate", END)

app = graph.compile()
```

동일한 upstream 노드를 가진 downstream 노드를 자동 병렬 실행

```python
# 4) 실행 (async)
async def run():
    init: AppState = {
        "text": "전기차 보조금 축소 소식 이후 소비자 반응이 혼재되어 있으며, "
                "일부 지역에서는 충전 인프라 확충 속도가 더딘 편이다."
    }
    out = await app.ainvoke(init)
    print(out["report"])

asyncio.run(run())
```

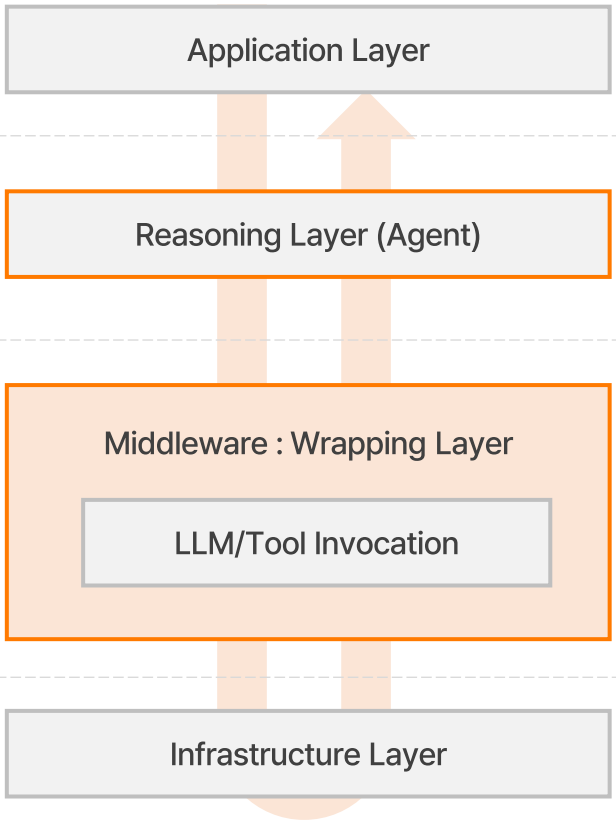
# 4. AI Agent 구현

SUMMARY

SUMMARY

# 2. Architecture

Agent(실행)와 Runtime Governance(통제)를 계층으로 분리한 아키텍처




[Strategy] Goal, Plan, Graph Design

Goal/Plan

**Agent를 사용하는 서비스 계층 :**

API Endpoint, Service Orchestration, User session 관리

**Agent**

- `create_agent` 표준 에이전트 루프
- 필요 시 Node+Edge, Tool Call as Node, Explicit State Object로 직접 조립

Execute

Reflect

**Runtime Governance Layer : Agent 실행 경계에서 정책과 컨텍스트를 주입하는 통제 계층**

- Governance : 정책 검증, PII *Personally Identifiable Information* 필터링, tool access control
- Observability : tracing, logging, metrics, audit
- Operational : retry, timeout, rate limiting, cost tracking
- Transformation : schema validation, prompt 수정, response 후처리

Execute
Runtime Control

- **LLM Provider**
- **Tool Runtime**

SUMMARY

# 1. State

LangGraph 전체 흐름의 기억 저장소이며, 노드 간 연결성과 판단을 위한 핵심 데이터 구조

- 노드와 노드 간에 정보를 전달할 때 상태(State) 객체에 담아 전달 → 데이터 공유
- 에이전트가 현재까지 수행한 작업의 결과를 담고, 이후 판단에 기억처럼 사용 → Context 유지, 흐름 유지

## LangGraph > State

What to Design:

- 어떤 값을 저장할지 정의 : 질문, 응답, 평가 등 단계별 필요한 항목
- TypedDict :
  - State 스키마를 정의하는 수단 → 구조 선언
  - Python dict에 Typehinting을 추가한 방식으로 명세
- Reducer : 자동으로 리스트에 메시지 추가
  - (활용) 여러 값을 하나로 합치는(fold) 연산
  - (코딩) 필드 업데이트를 어떻게 병합할지 정하는 함수 (fold 연산)
  - 이전 대화 내용에 새 메시지를 붙여서 하나의 대화 흐름으로 만들어 줌
- State 업데이트 방식
  - Reducer 미적용 (기본) : Overwrite
  - Reducer 적용 : 병합/누적
    (메시지: `add_messages`, 일반 리스트: `operator.add`)

Pitfalls :

- 노드 간 상태 공유를 위해서는 키 이름이 정확히 일치해야 함
- State에 너무 많은 내용을 저장하면 흐름이 복잡해지고 디버깅 어려움

```python
import operator
from typing import Annotated, TypedDict
from langgraph.graph.message import add_messages


# State 정의
class GraphState(TypedDict):
    messages: Annotated[list, add_messages]  # 메시지 (누적)
    question: Annotated[list, operator.add]      # 질문 (누적 관리 필요 시)
    context: str                                 # 문서 검색 결과
    answer: str                                  # 답변
    relevance: str                               # 답변 관련성 (Good/Bad)
```

※ messages: Annotated[list, add_messages]
- messages : LLM과 주고 받는 대화 내용
- (left) list : 기본 메시지 리스트
- (right) add_messages : 병합할 메시지 리스트 또는 단일 메시지

SUMMARY

# 1-(3). State Implementation

State = 실행 흐름을 통제하는 시스템 레벨 데이터 구조

저장 그릇으로만 보지 말고, 노드 간 인터페이스로 봐야 함!!

## 1 State는 데이터 컨테이너가 아니라 Read/Write Contract

- 노드는 State 전체를 알 필요 없음
- 노드는 자신이 읽고/쓸 KEY만 알아야 함 → State Ownership
- State는 Graph의 공용 API

## 2 State는 서비스 관점에서도 고려되어야 함

- 이전 값 보존이 필요한가?
- 재현 가능한 실행인가?
- Trace 가능한가?

→ Overwrite vs append 전략 고려

## 3 State는 실패를 고려해야 함 (Error-aware)

- Observability, Monitoring 위한 관리 항목
- Status field : SUCCESS, FAILED, INTERRUPTED 등
- retry_count, error_message 등

## 4 State는 확장 가능해야 함

- Optional Field 사용
- Nested 구조 사용
- Evaluation은 dict로 감싸기 등

SUMMARY

# 4. Node vs Agent

어떻게(HOW) 수행할지 정의

## Node

- 에이전트 시스템 내에서 특정 기능을 수행하는 가장 작은 독립적인 작업 단위 (레고 블록의 개별 조각)

- 역할 :
  - 단일 기능 수행: 문서 검색, 텍스트 요약, 이미지 분석, 데이터베이스 조회, 답변 생성 등 하나의 명확하고 독립적인 작업 수행
  - 함수 형태로 정의: 특정 입력(State)을 받아 정해진 로직을 처리한 후, 결과(State)를 반환하는 함수 형태로 정의

- 특징 :
  - 주어진 입력에 따라 정의된 작업만 수행
  - Deterministic, 사전 정의된 엣지에 의해 제어권 결정
  - 노드 실패는 해당 단계에 국소적으로 영향

- 예시 :
  - 사용자의 질문을 분석하여 키워드 추출하는 노드 (QuestionAnalyzerNode)
  - 추출된 키워드로 문서를 검색하는 노드 (DocumentSearchNode)
  - 검색된 문서와 질문을 바탕으로 답변을 생성하는 노드 (AnswerGenerationNode)

무엇을(WHAT), 언제(WHEN) 수행할지 결정

## Agent

- 특정 목표를 달성하기 위해 상황을 이해하고 계획을 수립하고 행동을 실행
- 여러 노드들을 조합하고 제어하여 복잡한 문제 해결 (레고 블럭으로 조립된 로봇)

- 역할 :
  - 목표 설정 및 달성: 사용자의 요청이나 환경 변화에 따라 동적으로 목표를 설정하고, 이 목표를 달성하기 위한 일련의 작업 수행
  - Node Orchestration: 어떤 노드를 어떤 순서로 실행할지 결정하고, 각 노드의 출력을 다음 노드의 입력으로 연결하는 등 전체 작업 흐름 제어
  - 의사결정: 현재 상태를 바탕으로 다음에 어떤 행동(노드 실행)이 필요한지 판단하고, 필요에 따라 다양한 노드들을 유연하게 활용

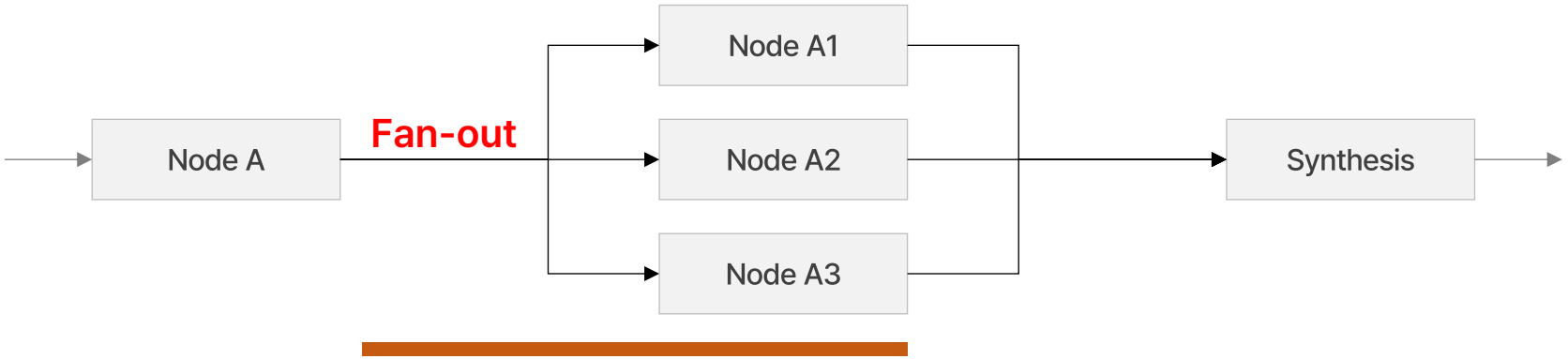
- 특징 :
  - 단순 노드들의 집합이 아니라, 노드들을 활용하여 지능적인 행동을 만듦
  - Agentic, 런타임에 LLM이 제어 판단
  - 에이전트 실패 파급 범위가 넓음

- 예시 :
  - 문서 기반 질의응답 에이전트: 사용자의 질문을 받으면
    → QuestionAnalyzerNode로 질문을 분석하고,
    → DocumentSearchNode로 관련 문서를 찾고,
    → AnswerGenerationNode로 최종 답변을 생성하여
    사용자에게 제공하는 전체 과정을 수행하는 에이전트

SUMMARY

# 2-(1). Fan-out 설계 고려사항

Production Fan-out/Fan-in은 병렬화가 아니라, 정합성·실패·동시성을 설계하고 통제하는 것




### 1 Reducer – 병렬 정합성

여러 병렬 노드가 같은 State 키에 쓸 때,
Reducer가 없을 경우 마지막 작성 시 나머지 내용을 덮어써 데이터 유실 발생

- **공유되는 키에는 Reducer 구성**
- **각 병렬 노드는 담당하는 키만 업데이트 처리**
  ← State 전체 업데이트 되지 않도록

### 2 부분 실패 처리

한 브랜치가 실패하면
성공한 브랜치 결과도 State에 저장되지 않음

- **Checkpointer 필수**
  : 성공한 노드 결과는 내부적으로 저장되므로,
  그래프 재개 시 실패한 브랜치만 재시도하면 됨

### 3 Concurrency 제어

병렬 처리는 지연 시간을 줄여주지만
순간 리소스 사용량을 키워 Rate limit 문제됨

- **max_concurrency 통해 동시 태스크 수 제한 :**
  {"configurable": {"max_concurrency": 10}}
- **외부 API 호출 한도 고려하여 실행 건수 제한**

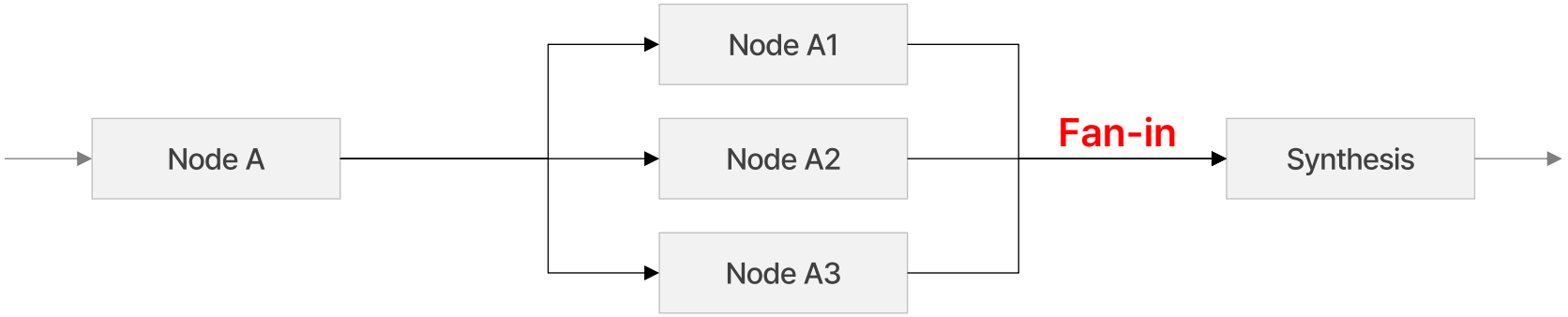
※ Remark
- Node : 하나의 작업을 수행하는 실행 단위(함수)
- Branch : Fan-out으로 갈라져 병렬 실행되고, Fan-in에서 합쳐지는 하나의 경로.
  (e.g.) 병렬 노드(A1, A2, A3)가 각각 fan-out부터 fan-in까지 이어지는 하나의 브랜치에 해당

*Source : https://docs.langchain.com/oss/python/langgraph/use-graph-api*

SUMMARY

# 2-(2). Fan-in 설계 고려사항

Production Fan-out/Fan-in은 병렬화가 아니라, 정합성·실패·동시성을 설계하고 통제하는 것




### 4 부분 실패 & 재처리

모든 병렬 노드가 완료되어야 다음으로 넘어가는데,
한 브랜치가 응답 없이 매달리면 전체가 멈춤

- 브랜치별 타임아웃과 처리 정의 – 부분 결과 진행, 실패 처리
- 재시도 시 부작용 중복이 없도록 멱등성(idempotency) 확보
- 일부만 성공했을 때의 정책 정의 (개수, 핵심 타스크만 달성, 등

### 5 결과 통합

병렬 노드의 완료 순서는 비결정적이라,
도착 순서에 의존한 통합은 실행마다 결과가 달라짐

- 통합 로직은 도착 순서에 의존적이지 않도록 설계
- 모든 태스크 완료까지 통합을 미루는 지연 실행(defer) 적용
- 중복 여부 검토 또는 캐시 기반 설계

### 6 관측성

여러 브랜치가 동시에 실행되어도
가장 느린 브랜치가 전체 지연 결정하는데, 이를 관측하지 않으면 병목을 특정할 수 없음

- 실행/실패를 브랜치 단위로 추적 가능하도록 State 설계
- Barrier 특성을 관측하여 병목 브랜치 식별

※ Remark
- Node : 하나의 작업을 수행하는 실행 단위(함수)
- Branch : Fan-Out으로 갈라져 병렬 실행되고, Fan-in에서 합쳐지는 하나의 경로.
  (e.g.) 병렬 노드(A1, A2, A3)가 각각 fan-out부터 fan-in까지 이어지는 하나의 브랜치에 해당

*Source : https://docs.langchain.com/oss/python/langgraph/use-graph-api*

# TIP Multi-Agent Output 부실하지 않게...!

에이전트 설계가 구조적으로 취약하면 Output이 부실하게 나올 수 있는 현상

원인 진단 :

- State의 정보 압축 현상 : 에이전트가 State에 반영할 때 LLM이 자연스럽게 요약하는 경향 있음
  - (e.g.) Research_agent : 검색한 내용을 state에 저장할 때 '요약'해서 저장
- 파이프라인 통과 중 누적 손실
  - (e.g.) raw research (2000 tokens)
    → summarize_node (500 tokens)
    → aggregator_node (200 tokens)
    → report_node (50 tokens)
- 보고서 생성 프롬프트 설계 미흡
  - (e.g.) State에 있는 내용으로 최종 보고서를 작성해줘
- Context Window 압박 따른 자동 압축 : 여러 에이전트가 수행하면서 conversation history가 길어질수록 앞 내용이 자동 압축

# TIP Multi-Agent Output 부실하지 않게...!

이 시스템에서 어떤 실패가 더 치명적인가?!!

| # | 방식 | 개요 | 중점을 두는 것 | 대신 포기하는 것 | 고려 포인트 |
|---|---|---|---|---|---|
| 1 | Truncation / Trimming | • 메시지 개수나 글자 수 기준<br>• 기준을 넘는 오래된 부분을 그냥 잘라내고 최근 것만 남기는 방식 | • 예측 가능성 | • 잘려나간 부분의 정보 (완전 손실) | • **비용/속도 중요**<br>• **약간의 정보 손실은 감수 가능** |
| 2 | Summarization | • 오래된 대화 내용을 모델이 요약문으로 압축<br>• 원문 대신 요약만 이어서 사용하는 방식 | • 컨텍스트 크기 절감 + 어느 정도의 정보 보존 | • 정확성 : 요약 정보의 누락/왜곡<br>• 추가 비용/지연 | • **완전 손실만은 피하고 싶은 경우** |
| 3 | Structured Output | • 미리 정해둔 항목(필드)에 필요한 값만 정해진 형식으로 전달하는 방식 | • 정확성 + 일관성<br>• 값이 정해진 필드로만 뽑히므로 왜곡 여지 적음 | • 유연성<br>• 사전에 정의한 필드 외 정보는 아예 담지 못함 | • **숫자, 출처처럼 틀리면 안 되는 값이 많을 경우** |
| 4 | 원본 보관 + 요약만 전달 | • 원본 전체는 별도로 저장<br>• 에이전트 간에는 압축된 요약만 주고받는 방식 | • 정확성에 대한 사후 검증 가능 | • 구현 복잡도 : 저장/맵핑/조회 체계 필요<br>• 사후 검증이 목적이라 실시간 정확도까지 보장하지 못함 | |
| 5 | Sliding Window + 오래된 부분만 요약 | • 최근 메시지 몇 개는 원문 그대로 유지<br>• 그 보다 오래된 부분만 요약해서 합치는 방식 | • 최근 정보의 정확성과<br>• 전체 맥락 유지의 균형 | • 구현 복잡도<br>• 여전히 오래된 부분은 요약 손실 존재 | • **최근 정보의 정확도가 특히 중요한 경우** |

# TIP Multi-Agent Output 부실하지 않게...!

## #4. 원본 보관 + 요약만 전달

### 1 State Schema 이중화

```python
class ReportState(TypedDict):
    # ✅ raw 데이터와 요약을 분리
    raw_findings: Annotated[list[dict], operator.add]  # 누적, 압축 금지
    summaries: Annotated[list[str], operator.add]      # 에이전트용 요약

    # 각 챕터별 상세 콘텐츠 보존
    chapter_contents: dict[str, str]  # {"intro": "...(2000자)...", ...}

    final_report: str
```

```
# Research Agent 프롬프트 핵심
RESEARCH_PROMPT = """
조사한 내용을 다음 형식으로 반환하라:
- raw_content: 조사한 모든 내용을 축약 없이 그대로 (최소 800자 이상)
- key_points: 핵심 포인트 목록
- sources: 참고 출처

절대로 내용을 요약하거나 압축하지 마라.
"""
```

### 2 정보 보존 파이프라인 설계

```python
def research_node(state):
    # 1단계: 상세 조사 (요약 지시 없음)
    raw = llm.invoke(f"""
    다음 주제를 상세히 조사하라: {state['topic']}
    - 모든 데이터, 수치, 사례를 포함
    - 최소 1000자 이상 작성
    - 요약하지 말 것
    """)

    # 2단계: 별도로 요약 생성 (에이전트 간 통신용)
    summary = llm.invoke(f"다음 내용의 핵심만 3줄 요약:\n{raw}")

    return {
        "raw_findings": [{"topic": state['topic'], "content": raw, "summary": summary
        "summaries": [summary]  # 다른 에이전트는 이것만 봄
    }
```

# TIP Multi-Agent Output 부실하지 않게...!

## #4. 원본 보관 + 요약만 전달

### 3 Report Write를 Raw Data에 직접 연결

```python
# Supervisor 패턴에서의 라우팅 설계
def report_writer_node(state):
    # ✅ raw_findings를 직접 참조 (summary 거치지 않음)
    all_raw_content = "\n\n".join([
        f"## {item['topic']}\n{item['content']}"
        for item in state['raw_findings']
    ])

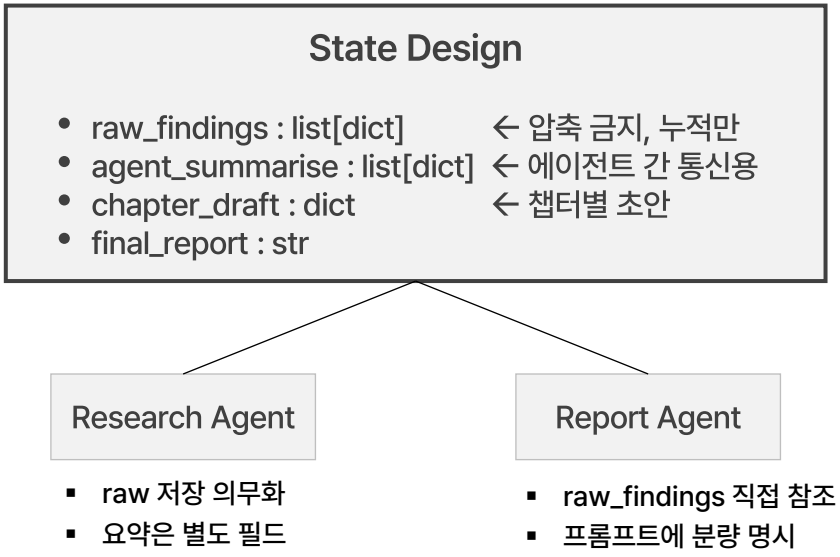
    report = llm.invoke(f"""
    다음은 각 에이전트가 수집한 상세 조사 자료이다:

    {all_raw_content}

    위 자료를 바탕으로 챕터당 최소 500자 이상의 상세 보고서를 작성하라.
    자료에 있는 수치, 사례, 인용을 반드시 포함할 것.
    """)

    return {"final_report": report}
```

### 4 Checkpointer로 중간 결과 보존




```python
# 각 노드 실행 후 state snapshot 저장
checkpointer = MemorySaver()
graph = graph_builder.compile(checkpointer=checkpointer)

# Report writer가 중간 체크포인트에서 직접 raw data 꺼냄
def report_writer_node(state, config):
    thread_id = config["configurable"]["thread_id"]
```

AI Campus 생성형AI

# [Appx. A] AI Agent 평가

- 평가를 바라보는 두 가지 층위
- 에이전트 평가 방법론
- Eval Metrics

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

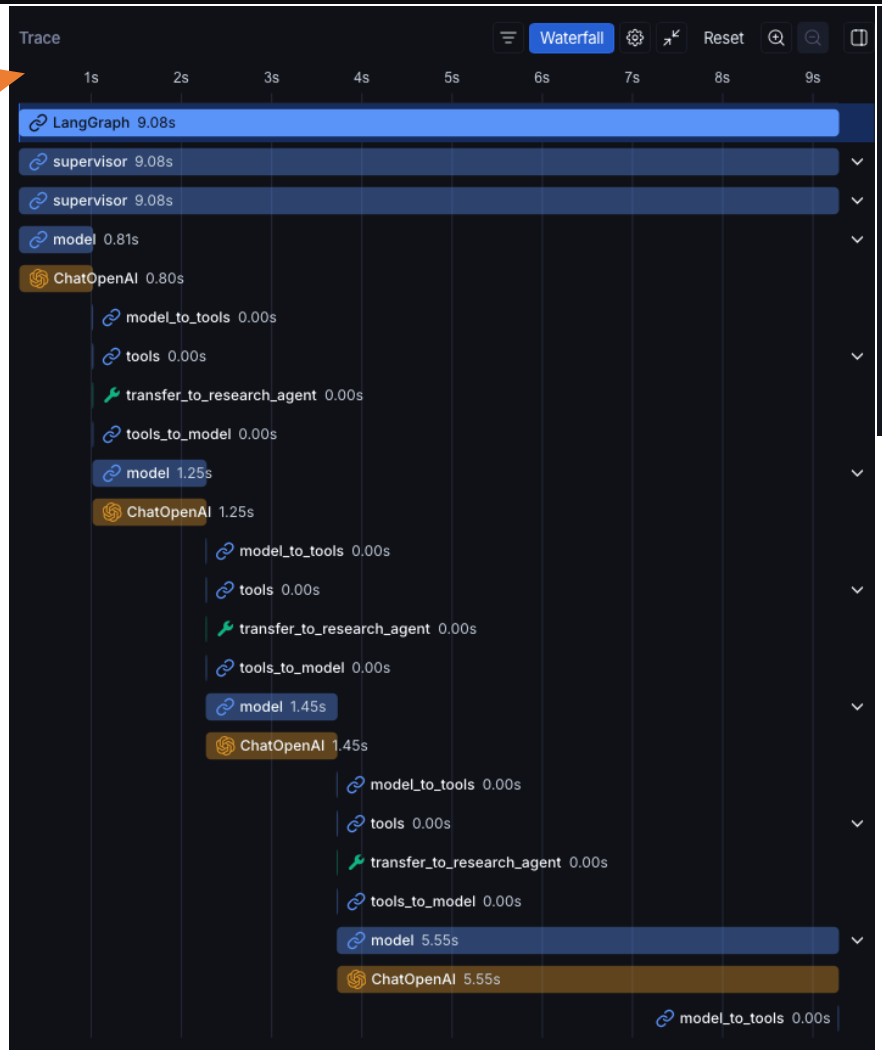
SK Inc. AX

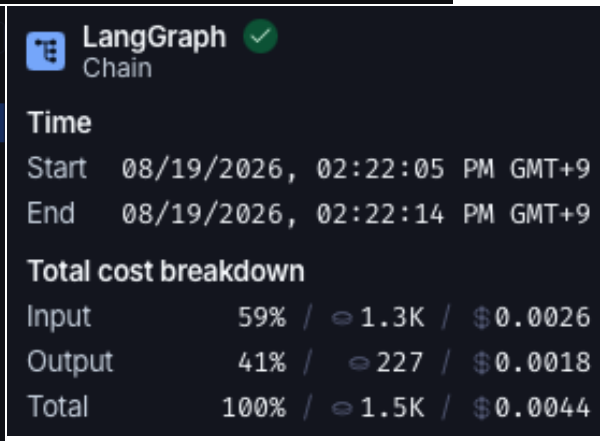
# LangSmith

**Result**
에러없이 수행되었지만,
답이 맞았나?

| Name | Input | Output | Error | Latency | Tokens | Cost | Start Time |
|---|---|---|---|---|---|---|---|
| LangGraph | user: 2024년 미국과 뉴욕주... | ai: 현재 리서치 에이전트 기능... | | 9.08s | 1,534 | $0.00443 | 8/19/2026, 2:2... |

**Trajectory**
9초짜리 경로가
최선이었나?
낭비는 없었나?



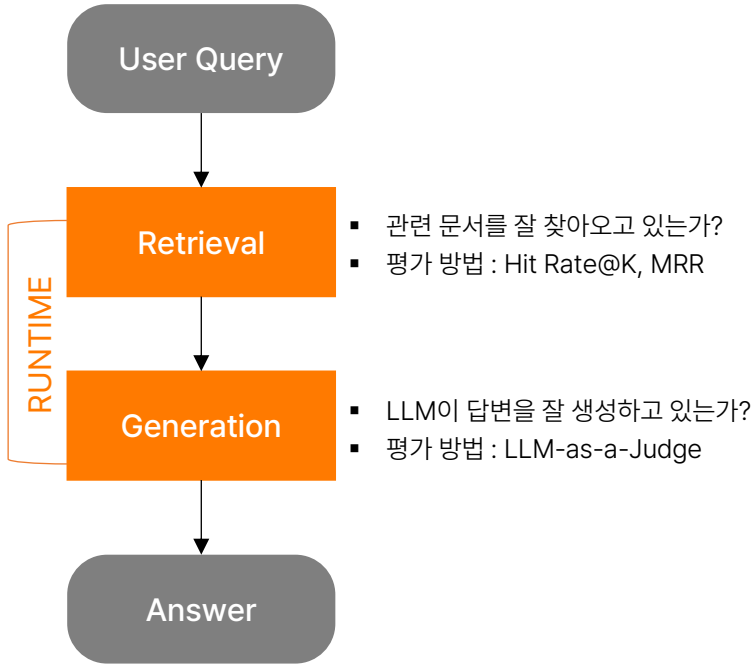




**Operational**
+9.08s + $0.0044
최적의 조합인가?

# Evaluation : RAG vs Agent

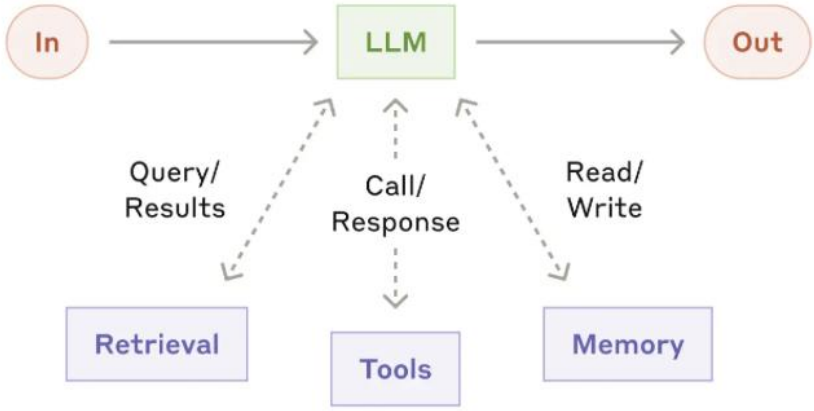
## RAG 평가

정적 텍스트 매칭 및 생성 품질 중심의 평가

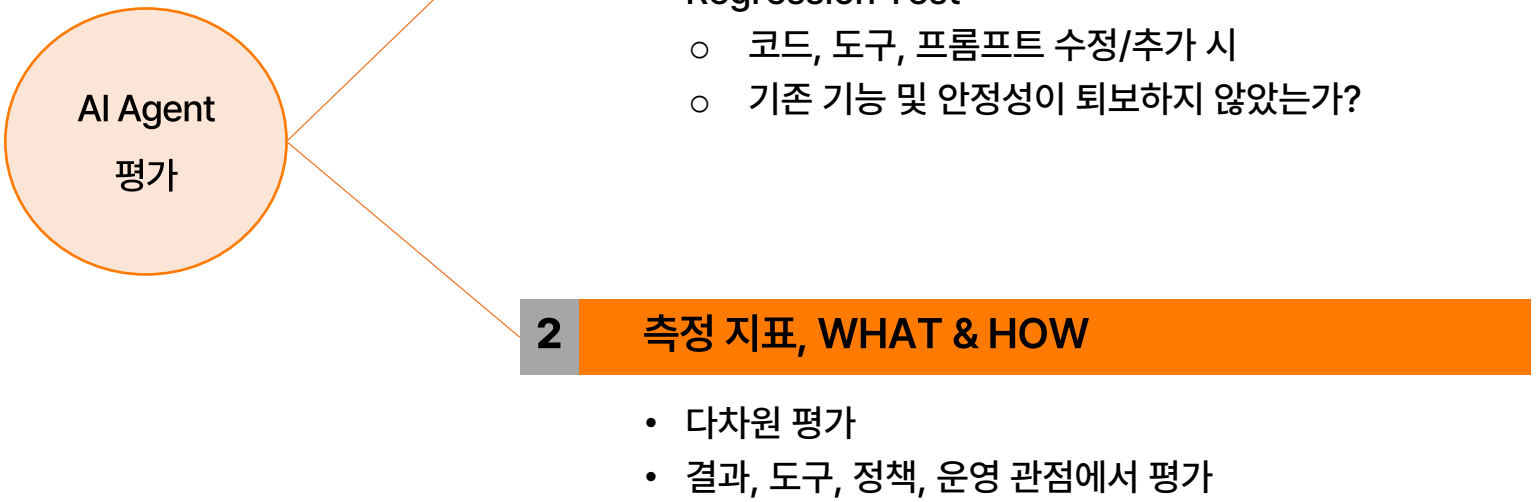



## AI Agent 평가

외부 환경 변경(State Change)과 행동 궤적(Trajectory) 대한 평가 필요
→ RAG 평가 방법론을 Agent에 적용하기 어려움



# 1. 평가를 바라보는 두 가지 층위



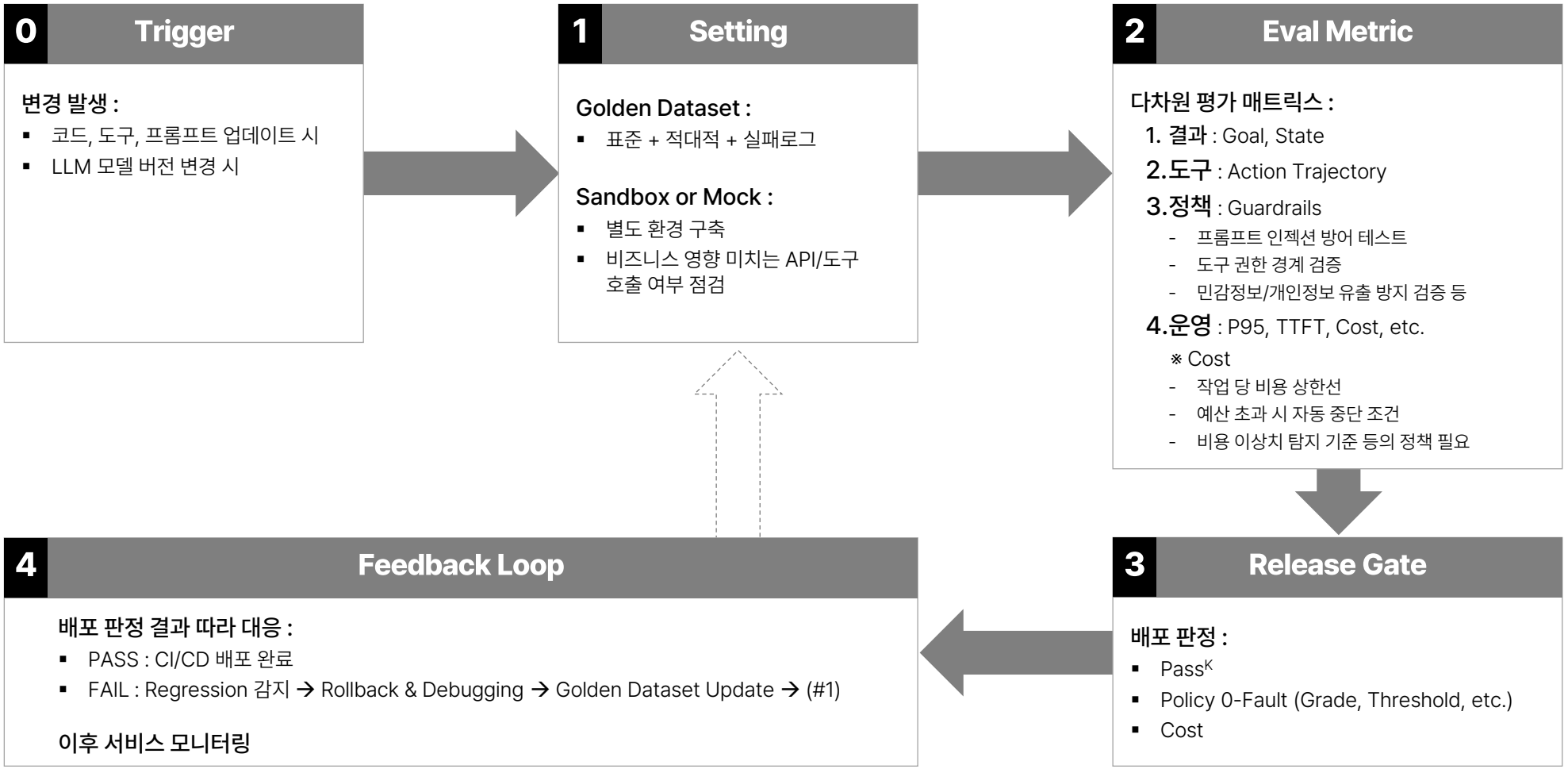
# [참고] Regression Test

- Regression : 이전 상태로 돌아가다, 퇴보하다
- 새로운 기능을 추가하거나 오류를 고쳤을 때, 기존에 멀쩡히 잘 돌아가던 기능이 망가지지 않았는지를 확인하는 테스트

- SW : 전통적인 코드는 결과가 결정론적, 1번만 돌려봐도 PASS 여부 판단 가능
- Agent : LLM 특징을 고려하여 K회 반복 실행하여 일관성 확보 → $Pass^K$ 지표

| 구분 | Regression Test | $Pass^K$ |
|---|---|---|
| 목적 | • 테스트 목적 및 절차<br>• Workflow, Strategy | • 성능 측정 지표<br>• Metric |
| 핵심 질문 | • 새로운 코드를 추가했는데, 기존 기능에서 오류가 발생하는가? | • LLM이 답변이 매번 달라지니,<br>• K번 진행하면 얼마나 안정적인가? |
| 주요 역할 | • 기능 업데이트 시 – 코드, 도구, 프롬프트 등<br>• 기존 골든 데이터셋 전체를 재실행 하여<br>• 품질 저하 방지 확인 | • LLM의 비결정성을 고려하여<br>• 여러 번 시도하여 (K = 3)<br>• 신뢰도를 측정 (지표) |

# 2. AI Agent 평가 방법론



# 3. Eval Metric

| 평가 축 | 평가 목적, WHY | 주요 평가 항목, WHAT | 평가 방식, HOW | 판정 기준(예시) |
|---|---|---|---|---|
| **1. 결과** | 사용자가 지시한<br>최종 비즈니스 목적을<br>환경 손상 없이 달성했는가? | • **State 변경 검증** : DB 갱신, 파일 I/O, 외부 API 트랜잭션 성공 여부<br>• **산출물 무결성** : 요구 포맷, 필수 포함 데이터, 수치 정확성 등<br>• **과업 완결성** : 복합 지시 시에 하위 세부 목표 완결 여부 | • 결과가 명확한 Task : 시스템 상태/반환값 직접 확인<br>• 열린 Task (요약, 리서치 등) : Rubric 기반의 LLM-as-a-Judge | $Pass^K$ ≥ 90%<br>(K회 반복 시 일관된 목표 달성) |
| **2. 도구** | 최소한의 자원으로<br>올바른 도구를<br>비즈니스 워크플로우에 맞게<br>호출했는가? | • **Tool Selection** : 적절한 도구 선택, 불필요한 도구 호출 방지<br>• **Argument Validation** : 스키마, 파라미터 타입, 필수 인자 유효성 등<br>• **Execution Order** : 도구 간 선후 관계 (e.g., 결제 전 승인, 삭제 전 조회 등)<br>• **Call Efficiency** : 동일 도구의 중복 호출 상한 준수 | • Trajectory 제약 기반 매칭<br>• 전체 실행 로그(Trace) 추출하여 사전 정의된 시퀀스 준수 여부 검증 | Trajectory 준수율 ≥ 90%<br>(비즈니스 중요도 순서가 포함되어 있는 경우 필수적) |
| **3. 정책** | 보안 취약점, 권한 이탈 및<br>법적/윤리적 컴플라이언스<br>준수하였는가? | • **권한 경계** : 미인가 도구 호출, 시스템 환경/경로 이탈 차단<br>• **적대적 공격 방어** : Prompt injection, Jailbreak 방어<br>• **데이터 거버넌스** : PII, 사내 기밀 노출, 비인가 외부 전송 차단 | • 적대적 테스트셋 (Red Teaming)<br>• 의도적 악의/유도 입력 투입 후 거절 응답으로 판정<br>• 환경까지 고려해야 하는 경우 샌드박스 차단 여부도 포함 | Zero-Fault<br>(위반 0건, 100% 차단) |
| **4. 운영** | 비즈니스 허용 예산과<br>SLA 범위 내에서<br>안정적으로 동작하는가? | • **응답 속도** : P95, TTFT<br>• **비용 관리** : 요청당 평균/최대 토큰 소비량, API 과금 상한선<br>• **추론 효율성** : 목표 달성까지 평균 스텝 수<br>• **시스템 안정성** : 무한 루프율, 타임아웃, 예외 미종료율 | • Tracing 데이터 통계 집계<br>• 배치 실행 결과의 P95(평균)를 사전에 정의된 운영 임계값과 비교 | P95 ≤ SLA<br>호출 비용 ≤ 예산 상한선<br>루프 실패율 = 0% |

AI Campus 생성형AI

# [Appx. B] Harness Engineering

- Agent Harness
- Deterministic Boundary
- State Schema, Validation
- 주요 설계 고려사항 – 권한, 게이트, 컨텍스트, Loop, Retry
- Harness 과잉

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX

# 시작하기 전에...

2026.08

개념 자체의 확산 속도에 비해 표준화된 정의나 검증된 벤치마크는 아직 부족한 상태 입니다.
객관적인 관점에서 접근하고 이해하는 것이 현재 요구됩니다.

# Harness

**MODEL**

Raw Power



**HARNESS**

Harness



핵심적인 목표를 달성하기 위해 제공되는
든든한 뼈대나 장비 일체



**TASK**

Wagon



Reins

**Guardrails, Routing**

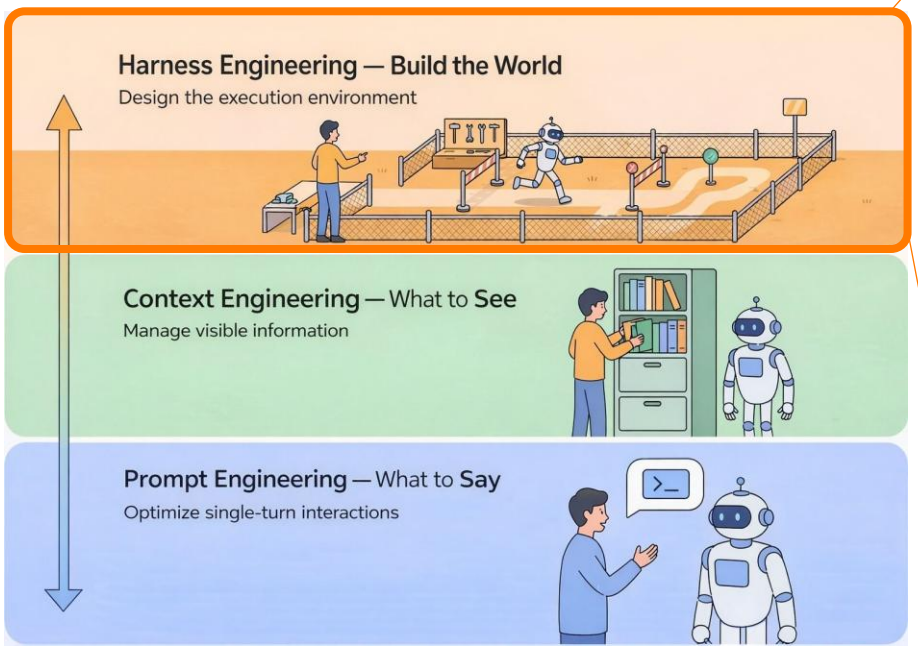
# 1. Agent Harness

Agent = Model + Harness

- 모델을 호출하고 tool calling하며, 언제 멈출지 결정하는 실행 계층
- 모델은 추론만 담당하고 툴 실행, 상태 유지, 검증, 복구 등 나머지는 모두 하네스 담당

# 2. Harness Engineering

Agent = Model + Harness




기능적 관점 :

- 모델은 엔진, 하네스는 자동차 그 자체 → **작동 시스템**
- 원시 LLM에 State, Tool Calling, Feedback Loop, Constraints 등이 더해져서 비로소 에이전트가 될 수 있음

문제 해결 관점 :

- 확률적 부품(LLM)의 **실패 모드에 대응** 필요
- 동작 불안정 : Guardrail, Permission, HITL 등
- Context Rot : 컨텍스트 길이에 관계없이 일관된 추론을 유지하도록
- Trust Debt : AI 결정을 검증하고 답변이 신뢰성을 확보하도록

방법론적 관점 :

- 실수가 **재발하지 않도록** 설계
- 프롬프트의 확률적 준수(Probabilistic compliance)

→ 하네스의 결정론적 강제(Deterministic constraint)

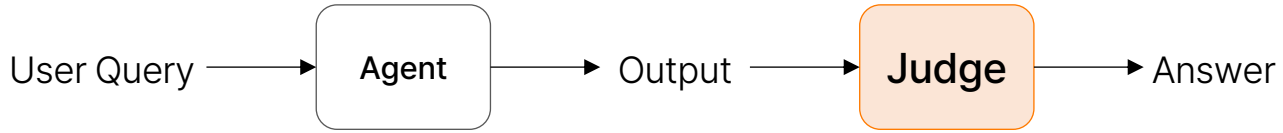
- 판정을 에이전트 밖으로 분리
- 결정론적 검사기(타입, 테스트, 권한)는 위반을 자동 차단
- 의미, 품질처럼 결정론적 판정이 불가능한 영역만 LLM Judge의 확률적 검토에 맡김
  - 단, 그 검토 결과를 정해진 규칙으로 게이트화하여 후속 동작(분기, 진행)은 결정론적으로 처리
  - 결정론이 되는 것은 판정의 정확성이 아니라, 파이프라인의 재현성, 분기 동작

*Source : https://milvus.io/blog/harness-engineering-ai-agents.md*

# 3. Deterministic Boundary

판정(Judgment)는 확률에 맡기고, 게이트(Gate)는 결정론으로 묶는다

LLM-as-a-Judge :




✖ LLM Judge를 도입하면 검증이 결정론적이 된다

○ LLM Judge 출력은 여전히 확률 분포이고, 같은 입력에 다른 판정을 낼 수도 있음

**결정론**이 되는 것은

판정의 정확성이 아니라

그 판정 결과를 받아 분기, 재시도로 연결하는 **파이프라인의 동작**

Judge가 "PASS"를 내면 항상 같은 후속 경로를 가는 것이 결정론, 판정 결과는 아님

# 3. Deterministic Boundary

LangGraph :

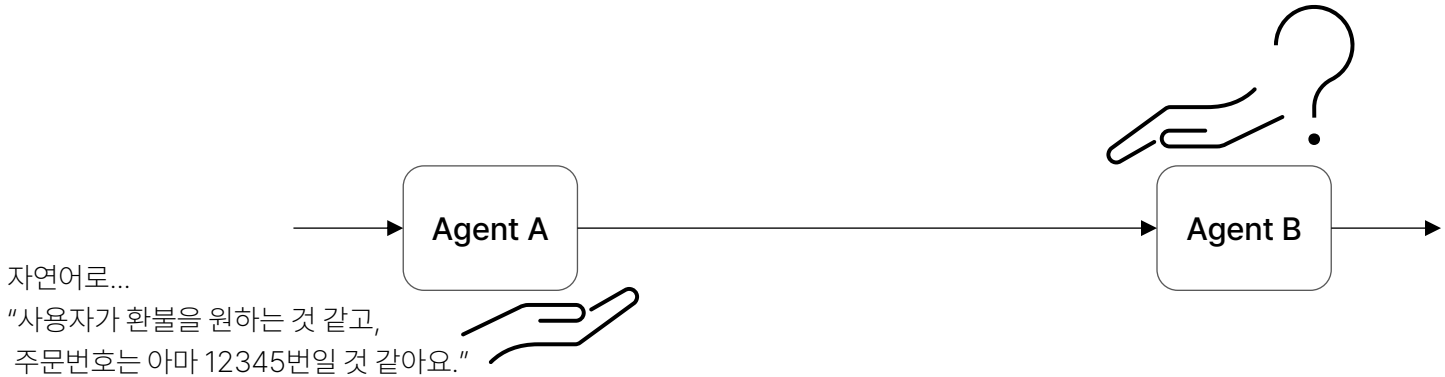
- Judge : Judge 노드 (LLM 호출) → 확률적 출력
- Gate : conditional_edges → 결정론적 라우팅

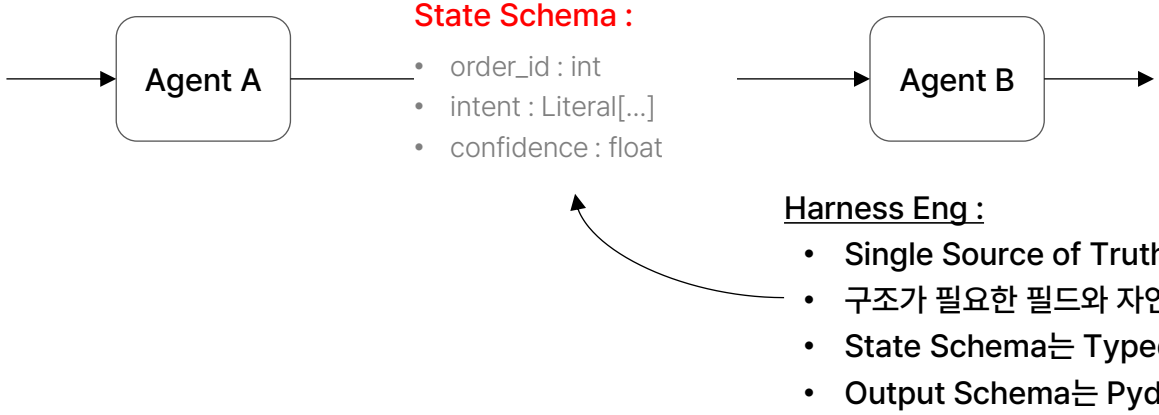
Harness Eng :

- 핵심은 이 둘을 분리하는 것
  → Judge는 라우팅 함수 밖으로 떼어내고, 라우팅은 순수 함수로 두는 것
- 만약...
  판정과 라우팅을 한 노드에 섞으면 (LLM에게 검증하고 다음 단계도 정해라)
  결정론적 경계가 사라지게 됨

```python
def gate(state) -> Literal["pass", "retry", "fail"]:
    score = state["judge_score"]      # 확률적 판정 결과(이미 산출됨)
    if score >= THRESHOLD: return "pass"   # ← 이 술어가 결정론
    if state["attempts"] < MAX: return "retry"
    return "fail"
```

# 4. State Schema, Validation



# Harness 고민

Harness는 하나의 확률 부품을 감싸는 일
확률 부품이 N개로 늘어나면...

- 부품과 부품 사이의 통신에 실려 가는 메시지는 확률적
- Harness는 통신 경로 수에 가깝게 고려되어야 함

➔ 정의한 Agent Pattern을 안전하게 굴러가게 하려면,
어떤 보강재로 어떻게 감싸야 하는지가 고민

HARNESS

# 5. 주요 설계 고려사항

도구 - 권한 | 검증 - 게이트 | 메모리 - 컨텍스트 | Loop 종료 | Retry

- Anti-Pattern : 권한을 프롬프트에 위임

- Guide
  - 두 개의 미들웨어로 구분
    - 노출 필터(결정론) : 사용자 권한, 상태에 따라 모델에 노출되는 도구 필터링
    - 실행 게이트 (결정론 + HITL)
  - 경계가 박힌 것
    - 상수 : WRITE_TOOLS, REFUND_CAP
    - 분기 : role != “admin”
    - 차단 : amount > REFUND_CAP

```python
WRITE_TOOLS = {"delete_user", "refund"}              # freeze: 부수효과 목록(코드 상수)
REFUND_CAP = 100_000                                 # freeze: 권한 스코프

@wrap_model_call
def filter_tools_by_role(request: ModelRequest, handler):
    """노출 단계 차단: viewer에겐 쓰기 도구를 아예 안 보여줌."""
    role = request.runtime.context.get("user_role", "viewer")  # 결정론 분기
    if role != "admin":
        safe = [t for t in request.tools if t.name not in WRITE_TOOLS]
        request = request.override(tools=safe)
    return handler(request)

@wrap_tool_call
def guard_write_tools(request, handler):
    """실행 단계 차단: 스코프 위반 자동 차단 + 비가역 효과는 HITL."""
    call = request.tool_call
    if call["name"] == "refund" and call["args"]["amount"] > REFUND_CAP:
        return ToolMessage(content="denied: over cap", tool_call_id=call["id"])  #
    if call["name"] in WRITE_TOOLS:
        if not interrupt({"action": call["name"], "args": call["args"]}):
            return ToolMessage(content="denied by human", tool_call_id=call["id"])
    return handler(request)

agent = create_agent(
    model="openai:gpt-4o",
    tools=[search, delete_user, refund],
    middleware=[filter_tools_by_role, guard_write_tools],    # ← 여기가 harness
    context_schema=Context,
)
```

# 5. 주요 설계 고려사항



- Anti-Pattern : 검증과 게이트를 LLM 확률 판정에 맡김

- Guide
  - 결정론 층이 먼저 거르고, 통과된 것만 Judge로
    - 에이전트 내부 출력 검증부터 : @after_model
    - Judge는 주로 판정 → 분기의 흐름을 가지게 되므로 그래프로 정의
      단, 판정(확률)과 라우팅(결정론)을 물리적으로 분리
      : def judge_node(...) → def gate(...)

```python
@after_model
def schema_guard(state, runtime):
    """결정론 검증: 통과 못 하면 재요청 신호. judge로 안 보냄."""
    last = state["messages"][-1]
    try:
        ReportSchema.model_validate_json(last.content)    # 스키마·타입(결정론)
    except ValidationError as e:
        return {"messages": [{"role": "user",
                "content": f"Output invalid: {e}. Re-emit valid JSON."}]}
    return None    # 통과 시 개입 없음
```

```python
THRESHOLD, MAX_ATTEMPTS = 0.8, 3

def judge_node(state):                          # 확률 판정 (결정론으로 못 잡는 의미·품질만)
    return {"judge_score": llm_judge(state["draft"]).score}

def gate(state) -> Literal["pass", "fix", "fail"]:    # 결정론 분기 (1절 원칙)
    if state["judge_score"] >= THRESHOLD:    return "pass"
    if state["attempts"] < MAX_ATTEMPTS:     return "fix"
    return "fail"

g.add_conditional_edges("judge", gate, {"pass": END, "fix": "writer", "fail": END})
```

# 5. 주요 설계 고려사항



- Anti-Pattern : 전체 히스토리를 모든 에이전트가 공유
- Guide
  - 에이전트 내부는 Prebuilt 또는 Custom Middleware로 좁히고
    - select_messages_for_role 통해 누가 무엇을 볼지 확인
    - SummarizationMiddleware 통해
      대화가 길어져 Context Window를 넘기 전에
      메시지 히스토리 요약
  - 에이전트 간 공유는 State 통해 공유

```python
@before_model
def scope_context(state, runtime):
    """역할별로 필요한 맥락만 모델에 주입(결정론 규칙)."""
    role = runtime.context["role"]
    kept = select_messages_for_role(state["messages"], role)  # 무엇을 줄지 = 규칙(freeze)
    return {"messages": kept}

agent = create_agent(
    model, tools=tools,
    middleware=[scope_context, SummarizationMiddleware(model="openai:gpt-4o-mini")],
)
```

```python
class TeamState(AgentState):          # ← TypedDict 확장 (1.0 제약)
    task: str                                          # freeze: 전원 공유(불변 목표)
    researcher_scratch: Annotated[list, add_messages]   # 리서처 전용 채널
    shared_findings: list                              # 명시적으로 공유하는 정제분만
```

# 5. 주요 설계 고려사항



- Anti-Pattern : Loop 종료를 모델 판단에 위임

- Guide
  - 종료 판단은 코드로 강제
  - 경계가 박힌 것
    - freeze : token_budget
    - 상수 : MAX_ATTEMPTS
    - 제약조건 : recursion_limit
  - 종료 조건은 코드에 박지 않으면 Loop은 끝나지 않음

```python
@before_model
def budget_gate(state, runtime):
    """예산 소진 시 강제 종료(품질 판정보다 우선)."""
    if state.get("token_budget", 0) <= 0 or state.get("attempts", 0) >= MAX_ATTEMPTS:
        return {"jump_to": "end"}    # 미들웨어에서 조기 종료 신호
    return None

# 그래프/에이전트 차원 최후 안전망
agent.invoke(input, config={"recursion_limit": 50})   # freeze: 반복 상한
```

# 5. 주요 설계 고려사항

도구 - 권한 | 검증 - 게이트 | 메모리 – 컨텍스트 | Loop 종료 | **Retry**

- Anti-Pattern : 멱등성 없는 재시도
- Guide
  - Retry + Checkpointer 통한 멱등성 보장
    - @wrap_tool_call : 도구 실행 오류 확인
    - invoke(None, checkpointer) : 영속성 보장
    - thread_id : 재개 시점 식별

※ 멱등성

same power 멱: 덮다 → 덮는 넓이 → 가로*세로 → 같은 수끼리 곱한 수

Idempotency, 幂等性

- 연산 또는 API 호출을 여러 번 적용하더라도 결과가 달라지지 않음
- 시스템의 안정성과 데이터 일관성 보장

```python
@wrap_tool_call
def retry_idempotent(request, handler):
    call = request.tool_call
    if call["name"] == "charge":
        key = call["args"].get("idempotency_key")
        if key and payment_api.already_processed(key):    # 결정론: 중복 차단
            return ToolMessage("already charged", tool_call_id=call["id"])
    for attempt in range(3):                               # freeze: 재시도 한계
        try:
            return handler(request)
        except TransientError:
            if attempt == 2: raise
            time.sleep(2 ** attempt)                       # 백오프

agent = create_agent(model, tools=[charge], middleware=[retry_idempotent],
                     checkpointer=cp)                      # 상태 영속화

# 실패 지점부터 재개 (처음부터가 아님)
try:
    agent.invoke(input, config={"configurable": {"thread_id": "order-123"}})
except Exception:
    agent.invoke(None, config={"configurable": {"thread_id": "order-123"}})
```

# 6. Harness 과잉

Harness는 공짜가 아니다 – 비용이 이득을 넘는 구간이 있다

- 멀티 에이전트 구조로 각 에이전트의 역할을 나누는 순간 Handoff, 통신, 동기화라는 변동비 발생
- 고정비를 넘는 이득이 없으면 단일 에이전트가 더 빠르고 안정적임

Harness 비용 = Harness가 주는 안정성의 대가

**TOKEN**

- Handoff 마다 Context 전달
- 역할별 지침 반복
- 통신 오버헤드 증가

- 멀티에이전트 통신 비용 = 라운드 수 * 에이전트 수 * Avg Context Window → polynomial 증가
- 제곱으로 증가하는 것은 에이전트가 다른 모든 에이전트의 전체 출력을 누적 수신하는 안티패턴 때문
  → 이걸 막는 것이 Context 채널 분리

**LATENCY**

- 판단 게이트, Judge, HITL
- 단계 수에 비례해 지연 누적

- 작업이 선형이고 분해 이득이 적음
- 검증이 단순
- 지연이 매우 중요한 도메인

멀티에이전트 Harness가 유효하지 않은 신호
= 단일 에이전트가 더 나은 경우

**MAINTENANCE**

- State Schema
- Middleware – 정책 간 관리 증가
- 디버깅, 코드 복잡도 확대



# 수고하셨습니다.

본 문서는 SK(주) AX의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK Inc. AX